

[OpenReview.net](#)

[Login](#)



[Back to the profile of Zhijian Liu](#)

DFlash: Block Diffusion for Flash Speculative Decoding



Jian Chen, Yesheng Liang, Zhijian Liu

Published: 01 May 2026, Last Modified: 25 Jun 2026 [ICML 2026 regular](#) [EveryoneRevisions](#) [BibTeX](#) [CC BY 4.0](#)

Abstract:

Autoregressive large language models (LLMs) deliver strong performance but require inherently sequential decoding, leading to high inference latency and poor GPU utilization. Speculative decoding mitigates this bottleneck by using a fast draft model whose outputs are verified in parallel by the target LLM. However, existing methods still rely on *autoregressive drafting*, which remains sequential and constrains practical speedups. Diffusion LLMs offer a promising alternative by enabling parallel generation, but current diffusion models typically underperform compared with autoregressive models. In this paper, we introduce **DFlash**, a speculative decoding framework that employs a lightweight block diffusion model for parallel drafting. We show that speculative decoding provides a natural and effective setting for diffusion models. By generating draft tokens in a single forward pass, DFlash enables efficient drafting, and by conditioning the draft model on context features extracted from the target model, it achieves high-quality drafts with higher acceptance rates. Experiments show that DFlash achieves over 6 lossless acceleration across a range of models and tasks, delivering up to 2.5 higher speedup than the state-of-the-art speculative decoding method EAGLE-3.

Lay Summary:

Large language models, such as chatbots and coding assistants, are powerful but often slow because they generate text one word at a time. This sequential process makes inference expensive and prevents modern GPUs from being fully used, especially when models need to produce long answers or reasoning traces.

This paper introduces DFlash, a method that makes large language model generation faster without changing the final output quality. DFlash uses a lightweight diffusion model to propose a block of future words in parallel. The large model then checks these proposed words in parallel and keeps only the correct ones. To make these guesses more accurate, DFlash also uses internal information from the large model to guide the smaller draft model.

Our experiments show that DFlash can significantly speed up generation across different models and tasks, including math, coding, and chat. By reducing inference time and serving cost, DFlash can make large language models more efficient, more accessible, and easier to deploy in real-world applications.

Originally Submitted Supplementary Material: [zip](#)

Link To Code: <https://github.com/z-lab/dflash>

Primary Area: Deep Learning->Large Language Models

Keywords: Large Language Model, Speculative Decoding, Diffusion Language Model

Originally Submitted PDF: pdf

Submission Number: 22495

Filter by reply type...

Filter by author...

Sort: Newest FirstSort: Oldest First

16 / 16 replies shown

Paper Decision

Decisionby Program Chairs30 Apr 2026, 21:40 (modified: 25 Jun 2026, 05:10)EveryoneRevisions

Decision: Accept (regular)

Comment:

This paper introduces DFlash, a speculative decoding framework that replaces traditional sequential autoregressive drafting with a lightweight block diffusion model. By leveraging parallel generation, DFlash reduces the drafting complexity from $O(n^2)$ to $O(n)$, effectively breaking the latency bottleneck inherent in current state-of-the-art methods like EAGLE. A key technical contribution is the KV injection mechanism, which fuses hidden features from the target model directly into the draft model's layers to improve acceptance rates without significant computational overhead.

The reviewers are in consensus that the paper is technically solid and offers a significant practical advancement for LLM inference. The authors have committed to including the additional ablation studies and long-context results in the final version. Given the high impact on inference efficiency and the thoroughness of the response, I recommend this paper for Acceptance.

Reference Correctness Check:

Automated checker did not flag any references in the submitted paper.

Official Review of Submission22495 by Reviewer sNrv

Official Reviewby Reviewer sNrv10 Mar 2026, 08:49 (modified: 25 Jun 2026, 05:31)EveryoneRevisions

Summary:

This paper proposes a new large language model inference acceleration framework named DFlash. This method uses a 5-layers block diffusion model to replace the traditional autoregressive draft model to achieve parallel draft generation. To improve the quality and acceptance rate of the diffusion model's generation, DFlash extracts and fuses the hidden layer features of the target model and injects them into the draft model's KV cache as contextual conditions. In terms of training, the paper abandons the standard random masking and instead adopts an "anchor" sampling strategy that conforms to the speculative sampling logic, and introduces exponentially decaying loss weights to emphasize the accuracy of early tokens. Experiments show that DFlash achieves significant lossless acceleration on multiple models.

Strengths And Weaknesses:

Strengths:

1. Clear and innovative motivation: Combining the parallel generation capability of diffusion models with the verification mechanism of speculative decoding is a highly promising research direction. This framework cleverly circumvents the poor standalone generation quality typical of diffusion models.
2. Well-designed training strategy: The anchor-based masking strategy tailored for speculative decoding and the position-weighted loss considering error accumulation are logically consistent. Ablation studies prove their effectiveness in accelerating convergence.

3. Significant single-batch acceleration: Under greedy decoding (temperature = 0), it achieves a higher single-batch speedup on the Qwen3-8B model than current state-of-the-art methods like EAGLE-3.

Weaknesses:

1. The paper lacks sufficient transparency regarding the mathematical and engineering implementation details of the draft model's internal mechanisms, which hinders reproducibility:
 - Specific implementation of layer-specific projection: The paper mentions that the fused target context features are "directly inject[ed]... into the Key and Value projections of every draft model layer." However, do the layers share the same projection matrix, or does each layer have independent projection weights?
 - Unclear KV Cache maintenance mechanism: If the draft model uses KV pairs projected from target hidden states, how is the KV generated during the draft model's own self-attention calculation handled? The main text lacks rigorous mathematical formulation explaining how these two coordinate (e.g., via concatenation or gating) to complete the computation.
2. DFlash defaults to a 5-layer draft model, whereas the baseline EAGLE-3 uses only a single-layer Transformer.
 - Core mechanism vs. Model capacity: It is difficult for readers to intuitively judge how much of DFlash's advantage over EAGLE-3 is attributable to the "block diffusion and KV injection mechanism," and how much is simply because "the draft model is deeper and has more parameters."
 - Misplacement of crucial ablations: The ablation study proving the effectiveness of KV injection (Table 8) is placed in Appendix A.2, which severely weakens the coherence of the main text's argumentation. Furthermore, this experiment only evaluates the removal of the Target Feature and lacks a strict controlled comparison against an autoregressive draft model of the same depth (e.g., 5 layers).
3. The largest model tested in the paper is the 30B-level Qwen3-Coder.
 - Hidden risks of compute waste: In industrial applications, massive models of 100B or even 200B+ (e.g., 235B) have the most urgent need to solve inference bottlenecks. For such massive models, the cost of verifying large draft blocks in compute-bound scenarios is extremely high. DFlash generates relatively large blocks (e.g., 16 tokens) in parallel. If early tokens are rejected, causing massive subsequent failures, will this aggressive parallel generation result in severe redundant FLOPs overhead? The paper lacks a theoretical discussion on compute utilization.
4. High-Concurrency Evaluation:
 - Misaligned core baselines: When evaluating high-concurrency scenarios using SGLang (Table 3), the paper presents the throughput of the Qwen series models but completely omits the comparison with EAGLE-3.
 - Lack of head-to-head comparison for Qwen: The paper only provides a high-concurrency comparison between DFlash and EAGLE-3 on LLaMA-3.1-8B (Table 4). Given the widespread application of the Qwen series models in the industry (especially in high-concurrency settings), the absence of this critical comparison weakens the method's practical persuasiveness.

Soundness: 3: good

Presentation: 3: good

Significance: 3: good

Originality: 3: good

Key Questions For Authors:

1. Regarding the KV injection mechanism: Please provide the exact mathematical formulation for the KV injection in each draft model layer. How do the draft model's own KVs and the injected Target KVs work together?

- Regarding the decoupling of performance gains: To more clearly demonstrate the superiority of the DFlash architecture, could the authors provide a direct comparison between a "5-layer autoregressive draft model (or 5-layer EAGLE-3)" and the "5-layer DFlash model"? We need to confirm that DFlash maintains a significant advantage under strictly aligned draft model capacities. I also strongly recommend moving the results from Appendix A.2 to the main text.
- Regarding scaling to ultra-large models: Considering the compute bound, how do the authors view DFlash's performance on ultra-large models (e.g., 100B to 235B)? Is there any quantitative analysis of the extra compute overhead (FLOPs) caused by rejection rates?
- Regarding SGLang high-concurrency comparisons: Please supplement the direct comparison experimental data between the Qwen3 series models (especially 8B and 30B) and EAGLE-3 regarding high-concurrency throughput under the SGLang framework. Why was the EAGLE-3 baseline omitted from Table 3?

Limitations:

yes

Overall Recommendation: 5: Accept: Technically solid paper, with high impact on at least one sub-area of AI or moderate-to-high impact on more than one area of AI, with good-to-excellent evaluation, resources, reproducibility, and no unaddressed ethical considerations.

Confidence: 5: You are absolutely certain about your assessment. You are very familiar with the related work and checked the math/other details carefully.

Compliance With LLM Reviewing Policy: Affirmed.

Code Of Conduct Acknowledgement: Affirmed.

Final Justification:

Thank you for the detailed rebuttal. However, my concerns are fully resolved. I will keep my score.

Rebuttal by Authors

Rebuttal by Authors 31 Mar 2026, 14:09 (modified: 25 Jun 2026, 09:16) Everyone [Revisions](#)

Rebuttal:

We thank the reviewer for the thorough review, the recognition of DFlash's innovative motivation, well-designed training strategy, significant acceleration, and the detailed suggestions that help strengthen the paper.

Weakness 1 & Question 1: KV Injection Mechanism

A **shared** `fc` layer fuses concatenated multi-layer target hidden states into the draft hidden dimension once before the layer stack. Each draft layer uses its **own** `k_proj/v_proj` to project **both** target and draft representations — shared K/V projection weights within each layer. Results are concatenated along the sequence dimension. Target hidden states only pass through `k_proj/v_proj`, bypassing `q_proj`, `o_proj`, and FFN entirely.

```
# Shared, once before layers
target_hidden = RMSNorm(Linear(concat(target_hiddens[selected_layers],
dim=-1)))
draft_hidden = embed_tokens(mask_block_ids) # [B, block_size, D]

# Per draft layer i
Q = Q_proj_i(draft_hidden) # [B, block_size, ...]
K = concat([K_proj_i(target_hidden),
            K_proj_i(draft_hidden)], dim=seq) # [B,
ctx_len+block_size, ...]
V = concat([V_proj_i(target_hidden),
            V_proj_i(draft_hidden)], dim=seq)
Q, K = RoPE(Q, K)
```

```

attn_out = Attention(Q, K, V, mask, causal=False) # [B, block_size, ...]
draft_hidden = draft_hidden + O_proj_i(attn_out)
draft_hidden = draft_hidden + FFN_i(RMSNorm(draft_hidden))
# target_hidden bypasses Q_proj, O_proj, FFN - only enters via K/V

```

We will opensource training code for full reproducibility. For inference, we have implemented DFlash in SGLang, vLLM and MLX-LLM, which will also be openourced.

Weakness 2 & Question 2: Decoupling Performance Gains

We trained three 5-layer draft models for Qwen3-4B (same data, setting, effort, block size 8):

	GSM8K		Humaneval		MT-Bench	
	AL	Spd	AL	Spd	AL	Spd
EAGLE-3-5L	4.2	2.1x	4.3	2.2x	3.1	1.4x
DFlash-AR-5L (KV inj.)	4.8	2.4x	4.6	2.3x	3.4	1.5x
DFlash-5L (KV inj.)	4.2	3.3x	4.0	3.2x	3.0	2.2x

DFlash-AR-5L vs. EAGLE-3-5L: higher acceptance length across all tasks, **showing KV injection is more effective than input fusion, independent of capacity**. DFlash-5L achieves comparable acceptance length with EAGLE-3-5L but significantly higher speedup (3.3x vs. 2.1x) via parallel drafting, breaking the acceptance-latency trade-off. For Table 8, we agree with the suggestion and will move it to the main text in the revised version.

Weakness 3 & Question 3: Ultra-Large Models & Compute Efficiency

We trained DFlash for larger targets. Results at Concurrency = 8 on SGLang:

Model	Math500 AL/Spd	Humaneval AL/Spd	MT-Bench AL/Spd
Qwen3.5-27B	7.7/3.8x	9.1/3.9x	5.5/2.5x
Qwen3-Coder-Next-80B-FP8	6.0/1.9x	7.2/1.9x	3.9/1.2x
GPT-OSS-120B (B10)	5.4/1.6x	4.4/1.7x	3.7/1.3x

DFlash maintains high acceptance length up to 120B. Notably, large target models are typically MoE architectures, which are less compute-bound than dense models at high concurrency, making them more friendly to DFlash's larger block sizes.

Verification compute waste is a concern shared by all speculative decoding methods, but DFlash achieves better acceptance length / draft cost trade-off. Moreover, DFlash trained with B=16 adapts to smaller block sizes without retraining. We define utilization as accepted / block size. Below is the DFlash draft model for Qwen3.5-35B-A3B with block size 4, 8 and 16:

B	Math500	HumanEval	MT-Bench	Avg AL	Util.	Wasted
---	---------	-----------	----------	--------	-------	--------

4	3.5	3.5	3.1	3.4	84%	16%
8	5.5	5.7	4.4	5.2	65%	35%
16	7.2	7.9	5.3	6.8	43%	57%

At B=4, 84% of verification FLOPs produce accepted tokens — a practical knob to trade off acceptance length and compute based on the serving scenario.

Weakness 4 & Question 4: High-Concurrency EAGLE-3 Comparison

Due to the character limit, we provide SGLang results on the production-level 30B MoE model (Qwen3-Coder-30B-A3B) here. EAGLE-3: depth 8, top_k=10, tokens 16/60. DFlash: block size 16.

C	Method	Humaneval AL/Spd	MBPP AL/Spd
1	EAGLE-3 (16)	6.2/1.9x	6.5/2.0x
1	EAGLE-3 (60)	6.9/1.9x	7.2/2.3x
1	DFlash (16)	8.1/3.5x	7.2/3.2x
8	EAGLE-3 (16)	6.2/1.8x	6.5/2.0x
8	EAGLE-3 (60)	6.8/1.5x	7.2/1.9x
8	DFlash (16)	8.1/3.2x	7.2/3.2x
16	EAGLE-3 (16)	6.2/1.5x	6.5/1.9x
16	EAGLE-3 (60)	6.8/1.2x	7.2/1.6x
16	DFlash (16)	8.1/3.2x	7.2/3.3x
32	EAGLE-3 (16)	6.2/1.2x	6.5/1.5x
32	EAGLE-3 (60)	6.8/1.0x	7.2/1.3x
32	DFlash (16)	8.1/3.1x	7.2/3.1x

DFlash maintains ~3x speedup across all concurrency levels, while EAGLE-3 degrades significantly. We will add more Qwen model comparisons on SGLang in the updated paper.

Replying to Rebuttal by Authors

Rebuttal Acknowledgement by Reviewer sNrv

Rebuttal Acknowledgement by Reviewer sNrv03 Apr 2026, 15:17 (modified: 25 Jun 2026, 10:36)EveryoneRevisions

Acknowledgement: (a) Fully resolved - My concerns have been adequately addressed. If you select this option, please consider adjusting your score accordingly.

Reasons:

No more questions.

Official Review of Submission22495 by Reviewer erZc

Official Review by Reviewer erZc08 Mar 2026, 14:25 (modified: 25 Jun 2026, 05:31)EveryoneRevisions

Summary:

Autoregressive LLMs are bottlenecked by sequential decoding, and while speculative decoding uses a fast draft model to mitigate this, existing methods still rely on sequential autoregressive drafting. DFlash introduces a lightweight block diffusion model that generates draft tokens in a single parallel forward pass, overcoming this limitation. By conditioning on context features from the target model, it produces high-quality drafts with improved acceptance rates. Experiments show DFlash achieves over 6× lossless speedup, outperforming the state-of-the-art EAGLE-3 by up to 2.5×.

Strengths And Weaknesses:

The paper has several notable strengths. The core idea of using diffusion models for speculative drafting is well motivated and clearly explained. The authors provide a nice analysis of why autoregressive drafters are fundamentally limited by their linear scaling with speculation budget, while diffusion drafters can maintain constant drafting cost regardless of block size. This insight is both theoretically sound and practically important.

The experimental results are quite impressive. Achieving 4-5x speedups over baseline and consistently outperforming EAGLE-3 by a factor of 2-3x across multiple benchmarks is a significant advance. The authors evaluate on diverse tasks including math reasoning, code generation, and general chat, and they test both greedy decoding and sampling-based generation. The integration with SGLang and validation under varying concurrency levels shows that the method works in realistic serving scenarios, not just in controlled experiments.

The ablation studies are thorough and informative. The authors systematically investigate the impact of draft model depth, number of target features, and training-inference block size mismatch. The finding that models trained with larger block sizes generalize well to smaller inference-time blocks is practically useful and suggests interesting directions for adaptive scheduling.

The paper is generally well written and easy to follow. The figures and tables are clear, and the authors do a good job explaining the key concepts without getting bogged down in unnecessary technical details.

As for weaknesses, there are only a couple of minor points worth mentioning. The paper could provide more architectural details about the block diffusion model itself, such as the exact denoising schedule and how discrete tokens are handled in the continuous diffusion space. While this doesn't undermine the core contribution, additional clarity would help with reproducibility. Also, the performance on MT-Bench is noticeably lower than on math and code tasks, which is worth discussing but doesn't detract from the overall strong results.

Soundness: 3: good

Presentation: 3: good

Significance: 4: excellent

Originality: 3: good

Key Questions For Authors:

Could you provide more details about the block diffusion architecture? Specifically, how many denoising steps are used during inference, and how are discrete tokens mapped to and from the continuous diffusion

space? Understanding these implementation details would help readers better appreciate the computational trade-offs.

Limitations:
See questions.

Overall Recommendation: 4: Weak accept: Technically solid paper that advances at least one sub-area of AI, with a contribution that others are likely to build on, but with some weaknesses that limit its impact (e.g., limited evaluation). Please use sparingly.

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Compliance With LLM Reviewing Policy: Affirmed.

Code Of Conduct Acknowledgement: Affirmed.

Final Justification:
My final rating is 4. My concern is solved.

Rebuttal by Authors

Rebuttalby Authors31 Mar 2026, 14:10 (modified: 25 Jun 2026, 09:16)EveryoneRevisions

Rebuttal:
We thank the reviewer for the positive assessment, particularly the recognition of DFlash's well-motivated core idea, impressive experimental results, thorough ablation studies, and clear presentation. We address the two minor points below.

Architectural Details

DFlash is a **discrete masked diffusion model**, not a continuous diffusion model. The corruption and denoising processes operate entirely in discrete token space. Tokens are either masked or unmasked, and the model directly predicts logits over the vocabulary at each masked position. There is no continuous noise schedule or iterative denoising in embedding space. To minimize draft cost, which is a critical factor for speculative decoding speedup, we apply **one-step denoising** for the entire block of mask tokens. The draft model is forwarded only once to denoise the whole block, which is then submitted for verification. This aligns with our training setup, where we always input blocks of mask tokens starting from a clean anchor token (the bonus token from the target model) and predict the entire block in a single pass. We experimented with multi-step denoising, but the marginal acceptance length improvement did not compensate for the increased draft cost of multiple forward passes, so we use one-step denoising.

MT-Bench Performance

The relatively lower performance on chat benchmarks is due to two factors. First, our 800K training data mixture contains only 150K chat samples, limiting chat performance. We recently trained DFlash draft models for Qwen3.5 series with a rebalanced mixture (~30% chat data), and MT-Bench acceptance length increased significantly to ~5.5. Second, chat is inherently less structured than coding and math, making it harder for any draft model to speculate. Even the native MTP layer of Qwen3.5 models, trained jointly during pre-training, shows the same trend. Below are SGLang results (batch size 1, block size 16) comparing DFlash and MTP on Qwen3.5 models:

SGLang	MT-Bench		Alpaca	
	Acc Len	Speedup	Acc Len	Speedup
Qwen3.5-9B-MTP	5.3	1.5x	5.0	1.4x

Qwen3.5-9B-DFlash	5.5	3.0x	5.1	2.8x
Qwen3.5-27B-MTP	5.4	2.0x	5.2	1.5x
Qwen3.5-27B-DFlash	5.5	3.3x	5.2	3.0x

DFlash achieves similar acceptance length to MTP but **2x higher speedup** thanks to its parallel drafting with much lower draft cost.

Replying to Rebuttal by Authors

Rebuttal Acknowledgement by Reviewer erZc

Rebuttal Acknowledgementby Reviewer erZc03 Apr 2026, 13:43 (modified: 25 Jun 2026, 10:36)EveryoneRevisions

Acknowledgement: (a) Fully resolved - My concerns have been adequately addressed. If you select this option, please consider adjusting your score accordingly.

Reasons:

Thanks for the experiment and response. My concerns are addressed, and I suggest that the author add the experiments to the main paper.

Replying to Rebuttal Acknowledgement by Reviewer erZc

Reply Rebuttal Comment by Authors

Reply Rebuttal Commentby Authors03 Apr 2026, 14:20 (modified: 25 Jun 2026, 12:03)EveryoneRevisions

Comment:

Thank you very much for the positive feedback and for confirming that your concerns have been addressed. We also appreciate the suggestion to include these results in the main paper. In the revised version, we will add the chat and other results from our recently trained DFlash models, include more details on the rebalanced data mixture, and more clearly describe the architecture and diffusion process. If you feel these additions further strengthen the paper, we would be very grateful if you would consider this in your final assessment.

Official Review of Submission22495 by Reviewer jQ64

Official Reviewby Reviewer jQ6402 Mar 2026, 17:48 (modified: 25 Jun 2026, 05:31)EveryoneRevisions

Summary:

This paper introduces DFlash, a speculative decoding framework. Unlike existing methods (e.g., EAGLE-3) that rely on sequential autoregressive drafting, DFlash employs a lightweight block diffusion model for parallel draft generation. By injecting context features from the target model into the KV cache of every draft layer , DFlash achieves high acceptance rates and acceleration without compromising output quality. Results demonstrate a >6x speedup on models like Qwen3-8B.

Strengths And Weaknesses:

Strengths

- Reduces drafting complexity from $O(\gamma)$ to $O(1)$, generating an entire block of tokens in a single forward pass.
- Innovatively injects target model features directly into the Key and Value projections of every draft layer.
- Demonstrates superior acceleration (up to 6.08x) across multiple benchmarks and shows robust performance under various concurrency levels.
- Introduces position-dependent Loss Weighting and random anchor sampling, effectively simulating the speculative environment during inference and accelerating convergence.

Weaknesses

- While diffusion models reduce forward pass counts, projecting and injecting features from 5 target model layers into all layers of the draft model increases memory footprint and computational burden. The authors should more precisely quantify the impact of this "heavy conditional injection" on inference VRAM.
- Experiments focus on sequences up to 2048 tokens. In ultra-long contexts (e.g., 32k+), are the extracted fixed-layer target features still sufficiently representative? Does the acceptance rate drop sharply due to shifts in feature distribution?

Soundness: 3: good
Presentation: 3: good
Significance: 4: excellent
Originality: 3: good
Key Questions For Authors:
NA

Limitations:
Yes

Overall Recommendation: 5: Accept: Technically solid paper, with high impact on at least one sub-area of AI or moderate-to-high impact on more than one area of AI, with good-to-excellent evaluation, resources, reproducibility, and no unaddressed ethical considerations.
Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.
Compliance With LLM Reviewing Policy: Affirmed.
Code Of Conduct Acknowledgement: Affirmed.

Rebuttal by Authors

Rebuttal by Authors31 Mar 2026, 14:13 (modified: 25 Jun 2026, 09:16)EveryoneRevisions

Rebuttal:

We thank the reviewer for the careful evaluation and the recognition of DFlash's well-motivated core idea, impressive speedup results, thorough ablation studies, and clear presentation. We address the two raised concerns below.

Weakness 1: Memory Overhead of Conditional Injection

The memory overhead of DFlash's conditional injection is minimal. The only extra component beyond a standard draft model is a shared `fc` layer that projects the concatenated 5-layer target hidden states into the draft model's hidden dimension: `Linear(5 × D, D)`. For Qwen3.5-35B-A3B where $D=2048$, this adds $5 \times 2048 \times 2048 \times 2$ bytes ≈ 42 MB in weight parameters — negligible compared to the 70 GB target model.

Activation memory for the projection ($bsz=1, D=2048$):

	Prefill (seq_len=2048)	Decode (accepted ≤ 16)
5-layer target hidden [bsz, L, 5×D]	$5 \times 2048 \times 2048 \times 2 = 40$ MB	$5 \times 16 \times 2048 \times 2 = 320$ KB
fc output [bsz, L, D]	$2048 \times 2048 \times 2 = 8$ MB	$16 \times 2048 \times 2 = 64$ KB
Peak activation (input + output)	~48 MB	~384 KB

The heaviest VRAM usage occurs during prefilling, where all prefix tokens' target hidden states must be stored and projected — but at seq_len=2048, this peaks at only ~48 MB, trivial against the 70 GB target model. During decoding, the VRAM usage is minimal. All prefix tokens have been stored in draft model's KV cache, so the target hidden states correspond only to the new accepted tokens from the previous verification round, which is at most block_size (16 tokens), resulting in under 400 KB of activation memory.

Once projected, the fused target hidden is simply passed into the `k_proj` and `v_proj` of each draft layer — the same KV projections that a standalone draft model or an EAGLE-3 style input fusion draft model would use anyway. However, unlike these approaches, the target context tokens in DFlash:

- 1. Only pass through `k_proj` and `v_proj` — they bypass `q_proj`, `o_proj`, and the entire FFN at each layer.
- 2. Do not participate in self-attention — they serve only as KV entries that the mask-block tokens query against.

This means DFlash actually reduces per-layer computation compared to an EAGLE-3 style input fusion draft model or a standalone draft model, where the fused context tokens go through full transformer layers (Q/K/V projections, self-attention among all tokens, output projection, and FFN). In DFlash, the context tokens are **passive KV providers**, and only the small block of mask tokens (e.g., 16) undergoes the full forward pass.

Weakness 2: Long Context Performance

We evaluated DFlash on LongBench with Qwen3.5-27B across context lengths from 1K to 32K. The base DFlash draft model (trained on 4K context) shows acceptance length degradation as context grows beyond the training length. However, we found that **DFlash adapts to long contexts very easily**. Fine-tuning with only 1.6K samples from LongAlign-10k for 3 epochs (DFlash-Long) significantly **recovers and even improves long-context performance**. Below are the acceptance length results on LongBench:

Context	hotpotqa		qasper		gov_report	
	DFlash	DFlash-Long	DFlash	DFlash-Long	DFlash	DFlash-Long
1K	4.91	4.99	5.27	5.38	4.53	4.53
2K	4.97	5.06	5.50	5.67	4.35	4.38
4K	4.91	5.41	5.17	5.80	3.93	4.25
8K	4.46	5.76	4.17	5.62	3.32	4.04
16K	3.61	6.05	3.57	6.00	2.67	3.81
32K	—	—	—	—	2.09	3.56

Note: only gov_report contains samples reaching 32K context length in LongBench.

At short contexts (1K), DFlash and DFlash-Long perform similarly. As context grows to 16K–32K, the base DFlash draft model degrades (e.g., hotpotqa: 4.91 → 3.61, gov_report: 4.53 → 2.09), while DFlash-Long maintains or even improves acceptance length. This demonstrates that the extracted fixed-layer target

features remain representative at long contexts. The draft model simply needs lightweight context scaling training to learn the longer-range patterns.

This context adaptation requires minimal effort (1.6K samples, 3 epochs), and **we will incorporate this lightweight context scaling training for all future DFlash draft models.**

Replying to Rebuttal by Authors

Rebuttal Acknowledgement by Reviewer jQ64

Rebuttal Acknowledgement by Reviewer jQ64 01 Apr 2026, 12:49 (modified: 25 Jun 2026, 10:36) Everyone [Revisions](#)

Acknowledgement: (a) Fully resolved - My concerns have been adequately addressed. If you select this option, please consider adjusting your score accordingly.

Reasons:

I have no further questions. Good work.

Replying to Rebuttal Acknowledgement by Reviewer jQ64

Reply Rebuttal Comment by Authors

Reply Rebuttal Comment by Authors 03 Apr 2026, 12:23 (modified: 25 Jun 2026, 12:03) Everyone [Revisions](#)

Comment:

Thank you for the positive feedback and for confirming that your concerns have been fully addressed. We will add more details on the conditional injection design and expand the discussion of long-context generalization in the revised paper.

Official Review of Submission22495 by Reviewer Wtpu

Official Review by Reviewer Wtpu 17 Feb 2026, 18:29 (modified: 25 Jun 2026, 05:31) Everyone [Revisions](#)

Summary:

While speculative decoding (SD) can accelerate inference of autoregressive LLMs, most of the existing SD methods rely on autoregressive drafting. Recently, diffusion LLMs have gained interest due to the capability of parallel decoding and there are several works that use diffusion language models as a drafter. This paper proposes DFlash, a speculative decoding method that uses a small block diffusion model for parallel drafting. DFlash generates draft tokens in a single forward pass, by conditioning the draft model on hidden features from the target model. DFlash demonstrates superior speedup over existing SD methods.

Strengths And Weaknesses:

Strengths

1. The paper addresses an important research topic in speculative decoding with a dLLM drafter.
2. The paper is well-written and easy to follow.
3. Experimental results demonstrate the superior performance of DFlash.

Weaknesses

1. The contribution of this paper seems incremental. While it offers a valuable engineering solution, algorithmic novelty is limited. As the paper noted, parallel drafting (e.g. Medusa) and conditioning the draft model on hidden features from the target model (e.g. EAGLE) are not new.
2. This paper claims that existing methods like DiffuSpec and SpecDiff-2 are impractical due to the massive (e.g. 7B) draft models, but I believe it is an overclaim because they are using much larger target models (e.g. 32B, 70B) than this paper (e.g. 4B, 8B). If this paper wants to claim that they are impractical, it is necessary to demonstrate that DFlash shows superior speedup over DiffuSpec and SpecDiff-2 for the same target model size. Furthermore, even though there is no available implementation, I believe this paper should include more detailed discussion and comparison with these prior works using dLLM drafters to clarify the contributions of DFlash.

3. DFlash's speedup results are impressive. However, since it exploits existing ideas (parallel drafting, hidden features from the target model), I wonder what the critical factor is for this speedup. Medusa and SpecDiff also use parallel drafting, and EAGLE also conditions the draft model on hidden features from the target model. Then, is the key factor KV injection? Do you have results on DFlash (KV injection) vs DFlash (EAGLE-style condition) and DFlash (parallel drafting) vs DFlash (one-by-one drafting)?

I am open to raising my score if these points are addressed.

Soundness: 3: good

Presentation: 3: good

Significance: 2: fair

Originality: 2: fair

Key Questions For Authors:

Please refer to the Weaknesses section above.

Limitations:

No. This paper should discuss limitations of DFlash.

Overall Recommendation: 4: Weak accept: Technically solid paper that advances at least one sub-area of AI, with a contribution that others are likely to build on, but with some weaknesses that limit its impact (e.g., limited evaluation). Please use sparingly.

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Compliance With LLM Reviewing Policy: Affirmed.

Code Of Conduct Acknowledgement: Affirmed.

Final Justification:

Additional experiments for weakness 3 are insightful. While I still have a concern about the novelty, I decided to increase my rating from 3 (weak reject) to 4 (weak accept) as my final rating.

Rebuttal by Authors

Rebuttal by Authors31 Mar 2026, 14:22 (modified: 25 Jun 2026, 09:16)EveryoneRevisions

Rebuttal:

We thank the reviewer for recognizing the importance of the topic, clear writing, superior results, and for the openness to raising the score.

Weakness 1: Novelty

We appreciate this concern and would like to highlight three aspects of novelty:

1. First practical diffusion drafter across diverse models and frameworks. Unlike Medusa's independent prediction heads, DFlash models inter-token dependencies via block diffusion. It works across Qwen3/3.5 (4B–80B), LLaMA3.1, and GPT-OSS (20B–120B) on SGLang and vLLM, outperforming EAGLE-3 and native MTP. Below are SGLang concurrency=8, block size 16 results on one B200:

Model	Method	Math500 AL/Spd	Humaneval AL/Spd	MT-Bench AL/Spd
Qwen3.5-4B	MTP	6.5/1.5x	6.3/1.6x	5.3/1.3x
	DFlash	7.1/3.0x	7.3/2.9x	5.6/2.3x

Qwen3.5-9B	MTP	6.7/1.7x	6.6/1.7x	5.3/1.3x
	DFlash	7.3/3.5x	7.9/3.4x	5.5/2.5x
Qwen3.5-35B-A3B	MTP	6.9/1.7x	7.1/1.6x	5.2/1.2x
	DFlash	7.2/2.4x	7.9/2.3x	5.4/1.7x
Qwen3.5-27B	DFlash	7.7/3.8x	9.1/3.9x	5.5/2.5x
Qwen3-Coder-Next-80B-FP8	DFlash	6.0/1.9x	7.2/1.9x	3.9/1.2x
GPT-OSS-20B (B8)	DFlash	5.1/2.2x	4.3/2.2x	4.2/2.0x
GPT-OSS-120B (B10)	DFlash	5.4/1.6x	4.4/1.7x	3.7/1.3x

vLLM results for Qwen3.5-9B (DFlash tok/s, speedup vs AR):

C	Math500	HumanEval	MT-Bench
1	849 (4.0x)	969 (4.6x)	627 (3.0x)
8	5096 (3.2x)	5434 (3.4x)	3536 (2.2x)
16	7669 (2.5x)	8131 (2.7x)	5297 (1.7x)
32	9836 (1.9x)	10258 (2.1x)	6787 (1.3x)

2. KV injection is architecturally novel. It outperforms input fusion in acceptance length (see Weakness 3 Table) with less compute — **context tokens only pass through `k_proj/v_proj`, bypassing `q_proj`, `o_proj`, FFN, and self-attention.**

3. Breaking the acceptance-latency trade-off. AR drafters' latency scales linearly with speculation budget. DFlash achieves high acceptance length with constant drafting cost — a conceptual insight validated by Section 3's analysis and experiments.

Weakness 2: Comparison with DiffuSpec and SpecDiff-2

As neither method has publicly available code, direct experimental comparison is infeasible. We compare on similar-scale targets via acceptance length, which reflects draft quality independent of system differences. DFlash with block size 16 results:

Dataset	Qwen3.5-27B	Qwen3-Coder-Next-80B-FP8
Math500	7.7	6.1

GSM8K	7.2	5.5
HumanEval	9.1	7.3
MBPP	7.3	5.5
MT-Bench	5.5	4.1

vs. DiffuSpec: Dream-7B drafter for Qwen2.5-32B, avg acceptance length 6.99 with block sizes 20–30. DFlash for Qwen3.5-27B achieves higher acceptance length with smaller block size 16 and 1.7B draft parameters.

vs. SpecDiff-2: 7B DiffuLLAMA for LLaMA-2-70B, reports 5.04/6.57 acceptance length on Math500/HumanEval with block size 16. DFlash for Qwen3-Coder-Next-80B achieves **6.1/7.3** with 0.5B draft parameters.

With a much smaller draft model, DFlash achieves higher acceptance length than both 7B-based approaches, suggesting a more favorable draft-quality–cost trade-off for accelerating large target models. At the same time, we recognize that DiffuSpec’s training-free design and SpecDiff-2’s train-time/test-time alignment are valuable contributions to diffusion drafting. These directions are complementary to DFlash, and we will add a more detailed discussion in the revised paper.

Weakness 3: Critical Factor for Speedup

To isolate each factor, we trained four draft models for Qwen3-4B under the same data, setting, and effort (block size 8):

	GSM8K		Humaneval		MT-Bench	
	AL	Spd	AL	Spd	AL	Spd
EAGLE-3-5L	4.2	2.1x	4.3	2.2x	3.1	1.4x
DFlash-AR-5L (KV inj.)	4.8	2.4x	4.6	2.3x	3.4	1.5x
DFlash-inputfusion-5L	3.5	2.9x	3.5	2.9x	2.6	2.0x
DFlash-5L (KV inj.)	4.2	3.3x	4.0	3.2x	3.0	2.2x

KV injection vs. input fusion: Under both AR and parallel drafting, KV injection consistently achieves higher acceptance length — DFlash-AR-5L vs EAGLE-3-5L and DFlash-5L vs DFlash-inputfusion-5L. KV injection also simplifies training for AR drafters: no train-time-test with multiple forward passes as in EAGLE-3.

Parallel vs. AR drafting: DFlash-5L has moderately lower acceptance length than DFlash-AR-5L and comparable acceptance length with EAGLE-3-5L, but achieves dramatically higher speedup (3.3x vs 2.4x and 2.1x) due to constant drafting cost. **This advantage grows with larger block sizes (e.g., 16).**

The speedup stems from: (1) parallel drafting for constant cost, (2) KV injection for better quality with less compute, (3) lightweight drafter for minimal draft cost.

Limitations

We agree and will add a limitations section in the revised paper discussing compute-bound scenarios and context length generalization.

Replying to Rebuttal by Authors

Rebuttal Acknowledgement by Reviewer Wtpu

Rebuttal Acknowledgement by Reviewer Wtpu01 Apr 2026, 21:35 (modified: 25 Jun 2026, 10:36)EveryoneRevisions

Acknowledgement: (a) Fully resolved - My concerns have been adequately addressed. If you select this option, please consider adjusting your score accordingly.

Reasons:

Thank you for the response. Additional experiments for weakness 3 are insightful. While I still have a concern about the novelty, I decided to increase my rating from 3 (weak reject) to 4 (weak accept) as my final rating.

Replying to Rebuttal Acknowledgement by Reviewer Wtpu

Reply Rebuttal Comment by Authors

Reply Rebuttal Comment by Authors03 Apr 2026, 12:19 (modified: 25 Jun 2026, 12:03)EveryoneRevisions

Comment:

Thank you very much for your thoughtful review, and for raising the score and recognizing the value of the additional experiments.

We also appreciate the discussion on novelty. We acknowledge that prior works such as Medusa and SpecDiff-2 explore parallel drafting, and that EAGLE-3 conditions on target hidden states. We believe DFlash's novelty lies in the combination of block diffusion and KV injection. In our experiments, KV injection achieves higher acceptance length than EAGLE-3-style input fusion with lower compute cost, while block diffusion enables parallel drafting with constant cost. Together, they allow DFlash to achieve higher acceptance length than prior diffusion drafting methods with a much smaller drafter, and higher acceptance length than Medusa by modeling inter-token dependencies while keeping draft cost low. DFlash also demonstrates strong practical speedups for various models in real serving frameworks like vLLM and SGLang. We will clarify these novelty aspects more clearly in the revised paper. If this clarification helps address your remaining concern, we would be grateful if you would consider it in your final assessment.

[About OpenReview](#)

[Contact](#)

[FAQ](#)

[Hosting a Venue](#)

[Sponsors](#)

[Terms of Use / Privacy Policy](#)

[All Venues](#)

[Donate](#)

[News](#)

OpenReview is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026 OpenReview

[OpenReview.net](#)

[Login](#)



[Go to ICLR 2026 Conference homepage](#)

BlockSpec: Blockwise Speculative Decoding for Diffusion LLMs



Tianxiang Pan, Baitao Gong, Mo Guang, Hongwei Yong, Tianpeng Jiang, Yaqian Li, Zheng Cao, Kaiwen Long

16 Sept 2025 (modified: 11 Feb 2026)Submitted to ICLR 2026EveryoneRevisionsBibTeXCC BY 4.0

Keywords: speculative decoding, diffusion LLMs

TL;DR: This paper propose BlockSpec, a novel trajectory speculative decoding framework tailored for diffusion-based LLMs, which significantly reduces inference steps and accelerates parallel decoding.

Abstract:

In diffusion-based Large Language Models (dLLMs), parallel decoding is usually realized through threshold-based or top-k strategies. While effective in high-confidence tokens, these strategies often collapse on low-confidence tokens, forcing the model into inefficient single-token decoding. To address this limitation, we propose Block Speculation (BlockSpec), a novel training-free blockwise speculative decoding method that explores multiple future decoding trajectories in parallel. Our method introduces a new tree-based trajectory generation strategy and a blockwise parallel verification module, where decoding tokens are organized into tree exploration paths and then multiple decoding trajectories can be simultaneously verified. Unlike traditional speculative decoding that focuses only on fixed-order left-to-right token speculation, our approach is the first attempt to introduce block-level speculation, which jointly explores both token choices and decoding trajectories for dLLMs. We also design two complementary speculation formulations—*intra-block* and *inter-block* speculation—that jointly accelerate dLLMs within and across blocks. Extensive experiments show that the proposed BlockSpec model reduces iteration steps by up to 40%, accelerating over 80% of decoding steps. As a result, our model achieves up to 7–14× speedup over vanilla dLLMs, together with an additional 1.3× improvement over state-of-the-art methods.

Primary Area: foundation or frontier models, including LLMs

Submission Number: 6926

[Filter by reply type...](#)

[Filter by author...](#)

Sort: Newest FirstSort: Oldest First

27 / 27 replies shown

Add:

Paper Decision

[Decision](#)by Program Chairs26 Jan 2026, 14:08 (modified: 06 Feb 2026, 10:25)EveryoneRevisions

Decision: Reject

Add:

Meta Review of Submission6926 by Area Chair kQ1i

[Meta Review](#)by Area Chair kQ1i07 Jan 2026, 13:29 (modified: 09 Feb 2026, 12:08)EveryoneRevisions

Summary:

Reviewers raised concerns primarily around the conceptual novelty of the proposed blockwise speculative decoding framework relative to existing speculative decoding and diffusion decoding strategies. In particular, several reviewers questioned whether the proposed tree-based trajectory exploration constitutes a fundamentally new decoding paradigm or a heuristic extension of prior approaches. Additional concerns were raised regarding the clarity of the method description, the strength of theoretical justification, and whether the empirical speedups generalize across models, tasks, and decoding settings beyond the presented benchmarks.

Reviewer Concerns:

The rebuttal helped clarify implementation details, decoding procedures, and experimental settings, and partially addressed questions about how BlockSpec differs operationally from prior speculative decoding methods. The authors also provided reasonable explanations for some design choices and acknowledged limitations candidly. However, the core concerns regarding conceptual novelty, the lack of strong theoretical grounding, and insufficient evidence that the gains are robust and broadly applicable remain largely unresolved. In particular, the rebuttal did not convincingly demonstrate that the proposed approach represents a principled advance rather than an engineering-driven heuristic tailored to specific diffusion decoding setups.

Reviewer Scores:

4446. Had reviewers been able to participate fully in post-rebuttal discussion, some might have slightly increased their scores to reflect improved clarity and responsiveness from the authors. However, it is unlikely that these changes would have been sufficient to cross the acceptance threshold, as the main concerns about novelty and generality would likely have persisted.

Add:

General Response (part 1)

[Official Comment](#) by Authors 02 Dec 2025, 13:59 Everyone

Comment:

We sincerely thank all reviewers for their thoughtful and constructive feedback. We are glad that several key strengths of the paper were consistently recognized, including:

- A training-free speculative decoding framework tailored for diffusion LLMs;
- Substantial decoding acceleration (7–14× over vanilla dLLMs and ~30% latency reduction over Fast-dLLM (Dual Cache), reduces the number of denoising iterations by 30–40%, and increases effective tokens-per-step from 2.6 to 4.3);
- A tree-structured parallel verification mechanism that operates under bidirectional attention;
- A new inter-block speculation strategy enabling segmented lookahead beyond single-block decoding;
- Clear experimental validation across models (LLaDA, Fast-dLLM), datasets, and hardware (A800/H800).
- Improved clarity and presentation in the revision. The updated manuscript now includes a formal exactness theorem, expanded failure-case analysis, precise attention-mask diagrams, clearer prefix/suffix KV caching explanations, and expanded presentation of interblock speculation in Appendix.

Below we address the main cross-reviewer concerns and summarize the clarifications and revisions incorporated into the updated manuscript.

1. Novelty and Contribution

We appreciate that most reviewers recognized the contributions of BlockSpec. In response to requests for additional clarification, we have refined the presentation to more explicitly distinguish BlockSpec from both Fast-dLLM (Dual Cache) and existing autoregressive speculative decoding methods. While Fast-dLLM

(Dual Cache) provides a systems-level optimization—reducing per-step FLOPs through blockwise KV caching—BlockSpec introduces a complementary and algorithmic-level speculative decoding framework designed specifically for diffusion LLMs. This framework enables parallel multi-token denoising under bidirectional attention, a setting fundamentally different from AR models. The key innovations, now clarified more explicitly in the revision, include:

- Blockwise speculative decoding for arbitrary-order denoising. Unlike AR speculative decoding, dLLM decoding operates over masked multi-token trajectories, requiring a redesign of draft construction, trajectory ordering, and correctness criteria under bidirectional attention.
- A principled analysis and remedy for low-confidence degeneration. We identify the collapse of parallelism under low confidence as a unique bottleneck in diffusion LLMs and design draft collection & verification specifically targeting this issue.
- A carefully explored tree-structured draft scheme adapted to dLLM verification cost. Multiple expansion policies (top-k at root, top-1 at deeper layers, duplication control, depth–width trade-offs) were empirically explored and distilled into a hybrid design (Appendix A.2).
- Novel Inter-block speculation in dLLMs. Enabled by bidirectional visibility, segmented parallel decoding across two blocks has no AR analogue and required new attention-mask design, verification logic, and correctness analysis. To make these distinctions clear, we will add a dedicated paragraph(as we shown in comment of Reviewer DaQs) in the revision explicitly contrasting BlockSpec with Eagle/Medusa-style AR speculative decoding.

2. Performance Improvements

Most reviewers recognized the strong empirical performance of BlockSpec, particularly its gains in decoding speed and parallel efficiency. We appreciate these comments and emphasize that BlockSpec delivers meaningful improvements even on top of an already optimized baseline (Fast-dLLM with Dual Cache).

Concretely, BlockSpec achieves:

- $\approx 40\%$ fewer denoising iterations,
- $\approx 65\%$ higher tokens-per-step (from 2.6 $\rightarrow$ 4.3),
- $\approx 30\%$ additional end-to-end latency reduction beyond Dual Cache.

These results demonstrate that our method scales parallelism effectively without sacrificing accuracy ($<1\%$ difference). Importantly, this improvement is orthogonal to Dual Cache:

- Dual Cache reduces per-step FLOPs,
- BlockSpec reduces the number of required steps.

Because the two components optimize different parts of the decoding pipeline, their benefits accumulate cleanly, making BlockSpec’s effect clearly visible even on an already accelerated system. We also highlight that, compared with concurrent speculative-decoding work for diffusion LLMs (e.g., dPAD, Spiffy), BlockSpec provides a complementary and practically meaningful speedup, particularly for low-confidence and mid-depth denoising stages where existing methods remain slow. Reviewers widely acknowledged these improvements as a key strength of our submission.

Add:

General Response (part 2)

[Official Comment](#) by Authors02 Dec 2025, 14:01Everyone

Comment:

3. Clarification of blockwise masks, suffix tokens, and KV caching

Multiple reviewers give concerns about Fig. 3(3) and the masking / visibility rules. We agree the earlier figure was misleading in the initial submission. In the revised manuscript, we have:

- redrew all attention-mask diagrams (Sec3.1, Appendix A.3,)
- clarified how prefix/suffix KV states are recomputed and cached at each block (Sec 3.4)
- explained draft-token visibility and isolation across branches
- redesigned the terminology to avoid conflation with causal attention
- added full descriptions of inter-block masking (Appendix A.3)

These additions resolve most of the reviewers' concerns.

4. Better revisions prompted by reviewers

We sincerely appreciate the reviewers' constructive dialogue throughout the rebuttal process. Their thoughtful questions and clarifications directly helped us refine both the presentation and empirical analysis, and the paper is significantly stronger as a result.

In response to the collective feedback, we have incorporated the following improvements:

- Added complete attention-mask diagrams for both intra-block and inter-block speculation.
- Expanded Appendix A.2 with detailed tree strategies and ablation results.
- Expanded Appendix A.3 with inter-block design discussions, diagrams, and pseudocode.
- Added long-sequence decoding experiments up to 2048 tokens (Appendix A.9).
- Clarified the baseline definition, dataset configurations, and figure captions (Sec. 4.1, Fig. 5).
- Added missing citations (Autospeculation, AccelDiffusion, AR speculative decoding, etc.).
- Improved terminology (e.g., "medium-confidence tokens", "degradation") for clarity.
- Fixed typos and enhanced overall figure readability.
- Formal exactness theorem proof and failure case study (Appendix A.10)

We are grateful to the reviewers—their deep and thoughtful engagement significantly strengthened the final submission, and many of the improvements in the revised paper were made possible through this productive and collegial dialogue.

Closing remarks

We are deeply grateful for the reviewers' thoughtful engagement and the constructive discussions throughout the review process. The detailed questions and insightful feedback greatly helped us refine conceptual framing, strengthen the empirical analysis, and significantly improve the clarity of the paper.

BlockSpec represents one of the earliest explorations of speculative decoding for diffusion LLMs—a setting fundamentally different from autoregressive models. As an initial step, our study sheds light on:

- how low-confidence degeneration emerges in dLLMs,
- how speculative trajectories interact with bidirectional attention,
- how blockwise and cross-block parallelism can be effectively realized, and
- how near-lossless decoding remains achievable even under semi-autoregressive constraints.

We hope that the insights, analyses, and design principles consolidated through this review process will contribute to future work on more principled and robust speculative decoding techniques for diffusion LLMs. We sincerely thank the reviewers again for their invaluable input and for helping us deliver a much stronger final submission.

Add:

Official Review of Submission6926 by Reviewer VDUL

[Official Review](#) by Reviewer VDUL01 Nov 2025, 01:01 (modified: 12 Nov 2025, 09:15)EveryoneRevisions

Summary:

The paper proposes Speculative Parallel Decoding (SPD) for diffusion-based LLMs, a topic gaining traction in the community. The method is training-free, maintains near-lossless performance on studied

benchmarks, and introduces a simple draft tree generation algorithm with verification. The speed-up gains are good

Soundness: 2: fair

Presentation: 2: fair

Contribution: 3: good

Strengths:

- Proposes SPD for diffusion LLMs, which is an emerging and relevant area.
- Training-free approach, reducing complexity and resource requirements.
- Maintains almost lossless performance across evaluated benchmarks.
- Introduces a simple draft tree generation algorithm combined with verification.
- Works effectively with multi-token unmasking while maintaining accuracy.

Weaknesses:

- Draft tree generation algorithm lacks clarity in certain cases (e.g., W3D3(6) configuration).
- Missing citations for speculative decoding and diffusion-related prior work.
- Some results and configurations are not fully explained (e.g., latency trade-offs, inter-block setup).

Please see questions for more details

Questions:

Draft Tree Generation: For W3D3(6), why is there no [1,4] node at step (or level) 2, given that the likelihood of [1,4] > [1,3,4] always?

Main Results: Could the authors clarify that W2D2(3) was chosen because it offers lower latency compared to W3D3(6) on A800 early on in the results section?

Figure 5: What does “base” refer to, and on which dataset are these results generated? Also, what is the difference between Fast-dLLM (dual cache) and the proposed method on H800, given that Fast-dLLM performs second best on A800?

Table 1: HumanEval(0) shows BlockSpec outperforming baseline methods, while in other benchmarks BlockSpec performs slightly worse (as expected due to lack of lossless generation). Is there any justification for this anomaly?

Table 3: Inter-block has the highest average tokens per step—what about its latency? Is the inter-block setup used only for 2 blocks? If extended to more than 2 blocks, does it start to hit TPS limits? Also, when doing inter-block, are draft trees for respective blocks generated independently?

Missing Citations: Citations for speculative decoding and diffusion-based approaches are missing. This idea essentially extends speculative decoding for images as discussed in prior works.

- Diffusion Speculation
 1. Diffusion Models are Secretly Exchangeable: Parallelizing DDPMs via Autospeculation Hengyuan Hu, Aniket Das, Dorsa Sadigh, Nima Anari arXiv:2505.03983
 2. Accelerated Diffusion Models via Speculative Sampling Valentin De Bortoli, Alexandre Galashov, Arthur Gretton, Arnaud Doucet arXiv:2501.05370
- Speculative Decoding
 1. Direct Alignment of Draft Model for Speculative Decoding with Chat-Fine-Tuned LLMs Raghav Goel, Mukul Gagrani, Wonseok Jeon, Junyoung Park, Mingu Lee, Christopher Lott arXiv:2403.00858
 2. Recursive Speculative Decoding: Accelerating LLM Inference via Sampling Without Replacement Wonseok Jeon, Mukul Gagrani, Raghav Goel, Junyoung Park, Mingu Lee, Christopher Lott arXiv:2402.14160

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:21 (modified: 19 Nov 2025, 15:22) Everyone Revisions

Comment:

Q1. Draft Tree Generation — For W3D3(6), why is there no [1,4] ... ?

Response to Q1: We sincerely appreciate the reviewer's insightful question. Indeed, this observation captures the core motivation behind our tree-based trajectory exploration. Conceptually, our design initially follows the same intuition. Nevertheless, extensive experiments show that exploring more nodes after level 2 (e.g., keeping all top-k children) yields marginal gains in decoding acceptance rate while introducing substantial computational overhead, which may reduce overall speedup.

In our final implementation, we empirically adopt a hybrid strategy: top-k expansion only at the root layer and top-1 expansion for all deeper layers, as detailed in Appendix A.2. Under this hybrid strategy, the reviewer's example is expected. When expanding from node [1], although draft [1,4] have a reasonably higher likelihood than [1,3,4], the model's top-1 continuation from [1] happens to be [1,3]. Therefore, the node [1,4] is intentionally pruned—not because its likelihood is low or invalid, but because deeper layers never include more than one child.

We especially thank the reviewer for raising this point and this provides a valuable opportunity for us to expose more of our design rationale and empirical analysis. Due to page-limit and clarity considerations, these design choices and ablation studies were omitted from the initial submission. In the revised version, we added these details to Section 3.2 (highlighted in yellow) and a full Appendix A.2, where we explicitly compare different tree expansion strategies.

Q2. Main Results — Could the authors clarify that W2D2(3)...?

Response for Q2: We sincerely thank the reviewer for this helpful suggestion. In the revised manuscript, we have added an explicit clarification in Sec. 4.1 (highlighted in blue) that W2D2(3) is chosen for A800 evaluations because it achieves the best latency–efficiency balance among all tested configurations.

In addition, we would like to explicitly highlight the underlying trade-off. While larger draft-tree structures such as W3D3(6) provide higher acceptance rates and reduce decoding iterations, they also incur heavier per-step verification and expansion overhead, which can outweigh the benefits and lead to worse end-to-end latency on A800. To make this trade-off clearer, we have revised the total Sec. 4.3, and explain how this trade-off inspire us to do experiments on higher-compute hardware (H800) in Sec.4.4 to demonstrate the potential of larger tree structures when more compute is available.

Q3. Figure 5 — What does “base” refer to, and on which dataset ...?

Response for Q3: Thanks for pointing out this ambiguity. In Figure 5, “base” refers to Fast-dLLM (Dual Cache), which is the baseline used for all BlockSpec experiments. The results in the figure are generated on the HumanEval benchmark using LLaDA-7B-Instruct.

Due to page-limit, our initial submission did not explicitly restate this baseline choice in the figure caption. In the revised version, we have clarified this in both Sec. 4.1 (yellow-highlighted) and Sec. 4.4 (yellow-highlighted), explicitly noting that Fast-dLLM (Dual Cache) is used as the baseline throughout all latency and throughput analysis.

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:25 Everyone

Comment:

Q4. Table 1: HumanEval(0) shows BlockSpec outperforming baseline methods, while in other benchmarks BlockSpec performs slightly worse ...?

Response for Q4: We appreciate the reviewer for raising this observation. In general, BlockSpec achieves higher parallel decoding efficiency by exploring a more aggressive decoding trajectories. This may occasionally introduce small accuracy drops—consistent with our results on GSM8K, MATH, and MBPP.

However, HumanEval behaves differently from the other benchmarks. We observed that diffusion LLMs tend to exhibit high output variance on HumanEval—even across different devices platform. Moreover, we also find that enabling KV-cache consistently improves performance on HumanEval, a trend that is aligned with the original Fast-dLLM paper. We believe that the distinct behavior on HumanEval largely stems from the small size and inherent noisiness of the dataset for diffusion-based models. In addition, the tasks in HumanEval are relatively simple, meaning that higher parallelism can produce more aggressive yet simpler decoding trajectories that may occasionally lead to better program-level correctness.

Fast-dLLM, cuda 12.3, 12.5, 12.7: 43.3

Fast-dLLM, cuda 12.2: 44.5

Fast-dLLM, cuda 12.8: 40.9

Fast-dLLM (dual cache): 45.7

In our further case study (visualizations included in the updated Appendix A.8), we found concrete cases where BlockSpec's more aggressive path corrected a token that the baseline mispredicted, resulting in a correct result—while in other datasets such isolated tokens do not have such high impact. This sensitivity explains why BlockSpec is slightly better on HumanEval despite being generally more “speculative” than Fast-dLLM.

Q5: Table 3: Inter-block has the highest average tokens per step—what about its latency? Is the inter-block setup used only for 2 blocks? If extended to more than 2 blocks, does it start to hit TPS limits? Also, when doing inter-block, are draft trees for respective blocks generated independently?

Response for Q5: Our direct answers are as follows: (1) Latency. We have added a dedicated analysis of inter-block latency in Sec. 4.3 and Appendix A.3, showing how higher tokens-per-step interacts with the additional verification cost. (2) Yes. Inter-block speculation is used only between two consecutive blocks. This design avoids the rapidly increasing verification overhead that would occur when extending to more blocks. We clarify this in Sec. 3.4 and provide visual explanations in Appendix A.3. (3) Draft-tree construction. Yes, the draft trees for the two blocks are generated independently, with details, examples, and visualizations added in Appendix A.3.

We thank the reviewer again for highlighting this. Their questions provided a valuable opportunity for us to clarify and more fully articulate the design of inter-block speculation. Motivated by these comments, we have substantially expanded the corresponding sections in Appendix A.3 in the revised manuscript. Appendix A.3 now includes additional design motivation, detailed visualizations of the inter-block workflow, attention-mask diagrams, pseudocode, and extended experimental comparisons. And we hope this revised presentation give readers a clearer and more complete understanding of the design of inter-block speculation.

Q6: Missing Citations: Citations for speculative decoding and diffusion-based approaches are missing. This idea essentially extends speculative decoding for images as discussed in prior works.

Response for Q6: Thanks for pointing out these references. We have added these suggested citations to the revised manuscript. We also would like to clarify that our work specifically targets discrete masked

diffusion language models, whereas the cited speculative-decoding methods for diffusion mainly focus on continuous denoising diffusion models. Although related in high-level motivation, the underlying formulations and verification mechanisms differ substantially. The omission in the paper was due to page-limit constraints, and we appreciate the reviewer's reminder, which helped us complete and broaden the contextualization of our contributions.

Add:

Official Review of Submission6926 by Reviewer VSE3

Official Review by Reviewer VSE331 Oct 2025, 12:38 (modified: 12 Nov 2025, 09:15)EveryoneRevisions

Summary:

Common unmasking strategies, such as 'confidence threshold' and 'top-k', are slower on low-confidence tokens since only a single token can be sampled per step. The authors argue that low confidence occurs when dLLMs consider multiple possible trajectories. The paper, therefore, proposes a novel self-speculative decoding method called BlockSpec, which simultaneously searches for and verifies possible trajectories to select multiple tokens. BlockSpec achieves much higher throughput than the baseline and is twice as fast as concurrent work.

Soundness: 2: fair

Presentation: 2: fair

Contribution: 2: fair

Strengths:

- BlockSpec is a customised, self-speculative decoding method designed specifically for dLLMs. It effectively addresses issues such as multi-token verification and changing the hidden state prefix.
- The paper also proposes a novel inter-block speculation method that substantially increases block efficiency.
- BlockSpec achieves a much higher TPS speedup compared to the baselines.
- BlockSpec is twice as fast as concurrent work.

Weaknesses:

- W1: There is no empirical evidence to support the argument that the low confidence tokens are due to plausible candidates. For example, the model would display low confidence if the problem was too complex for it to handle.
- W2: Since dLLMs use bidirectional attention, the hidden state of the prefix tokens varies as the model explores different nodes in the search tree. Therefore, the method is based on Fast-dLLM, which fixes the prefix token during the denoising process. However, since prefix caching leads to performance degradation, dependency on Fast-dLLM must be a weakness.
- W3: Despite its fast speed, BlockSpec shows similar performance to Fast-dLLM (dual cache). Formal Speculative Decoding methods maintain the original performance of the target models.
- W4: There has been no ablation study on the role of 'medium-confidence tokens'.
- W5: Short generation length. The generation lengths are fixed to 512, which is relatively short.

Questions:

- Typo in section 2 "When facing everal..."
- Is there any way to find an optimal tree configuration for arbitrary settings, such as model size, dataset, GPU FLOPS, and GPU memory bandwidth?
- W1: Could you show the results for more complex tasks, such as AIME?
- W3: Where does the performance degradation come from? I believe that the Fast-dLLM's performance would be the upper bound.

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:28 Everyone

Comment:

Q1: Typo in section 2 "When facing everal..."

Response to Q1: Thank you for catching this typo. We have corrected "everal" to "several" in Section 2 in the revised manuscript.

Q2: Is there any way to find an optimal tree configuration for arbitrary settings, such as model size, dataset, GPU FLOPS, and GPU memory bandwidth?

Response to Q2: Optimal tree configuration We thank the reviewer for raising this important question. It directly relates to how BlockSpec balances parallel efficiency and additional computation, and we appreciate the opportunity to clarify our design and search process.

Direct answer: Our current work identifies tree configurations through extensive empirical comparison rather than a closed-form optimization rule. In particular, our search is guided by a practical observation from our experiments: BlockSpec typically reduces the total number of decoding iterations by 30–40%. Therefore, as long as the per-step additional latency introduced by exploring extra draft blocks stays below roughly 30%, the overall method still yields a general speedup. We would like to further explain our search strategies for tree structure in the aspect of search space and search settings.

Search space. We conducted extensive exploration over a broad space of draft-tree expansion strategies—including different branching factors, depth limits, duplicated vs. non-duplicated nodes, and symmetric vs. left-biased structures. Through systematic comparisons, we arrived at a hybrid design that provides the most robust trade-off under typical inference settings. The revised manuscript now includes a full discussion of these strategies in Appendix A.2, together with ablation results explaining why the hybrid configuration is preferred. While our current search is empirical, we think an automated search procedure could be possible for future work.

Search settings. Instead of optimizing explicitly over FLOPS or memory bandwidth, we evaluate whether the single-step overhead stays within the acceptable range relative to the expected gain in parallel efficiency. This is precisely why we include the latency roofline analysis in Sec. 4.3, which directly visualizes how many additional blocks can be profitably explored before per-step latency outweighs the step-count reduction.

From these measurements: -On A800, the per-step overhead remains acceptable for roughly 4 draft blocks (≈ 128 tokens). -On H800, the overhead stays small even up to around 9 draft blocks (≈ 288 tokens). These findings explain why W2D2 is preferred under A800 settings, while larger structures such as W3D3 become advantageous on stronger hardware.

We also recognize that our explanation of compute overhead in initial submission was not sufficiently clear. Following the reviewer's suggestion, we have significantly refined the description in Sec. 4.3, making explicit the relationship between additional blocks, per-step latency, and the resulting parallel decoding efficiency. Thanks again for highlighting this point, which allowed us to better articulate both the practical search strategy and the underlying trade-off governing BlockSpec.

Q3: Could you show the results for more complex tasks, such as AIME?

Response to Q3: We appreciate the reviewer's suggestion. Following this comment, we conducted preliminary experiments on AIME using the same base models as in our main paper (LLaDA, Fast-dLLM, and our BlockSpec-accelerated variants), without adding complex CoT-style prompting or specific reasoning scaffolds. As a result, none of the models (neither the base diffusion LLMs nor our accelerated versions) were able to correctly solve a single AIME problem. We give a specific sample in a individual official comment below for more discussion if interested.

We believe this reveals an important characteristic of current discrete masked diffusion language models: they struggle to perform long-horizon, multi-step mathematical reasoning, even before considering decoding strategies. This aligns with the absence of such benchmarks in prior diffusion-LLM work, and highlights a potential future direction for strengthening these models, e.g., via reasoning-oriented training or reinforcement learning.

It is important to clarify that the focus of our work is on improving decoding efficiency for discrete diffusion LLMs, addressing limitations of applying speculative parallel decoding in this regime. Our method does not modify or enhance the underlying reasoning ability of the base models. The AIME results therefore reflect a limitation of the base models themselves rather than of BlockSpec.

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:31 Everyone

Comment:

Q4: Where does the performance degradation come from? I believe that the Fast-dLLM's performance would be the upper bound.

Response to Q4: We thank the reviewer for raising this important question. First, we clarify that our target model for speculative decoding is Fast-dLLM (Dual Cache), and all of our experiments are conducted on this baseline. Our initial submission did not emphasize this clearly, and we have added explicit statements in Sec. 4.1 (yellow-highlighted). We furthermore expanded Sec. 3.4 (Design of Suffix Tokens and KV Cache) to clarify how suffix [MASK] tokens and blockwise KV-cache mechanisms are handled in discrete diffusion LLMs.

By comparing with Fast-dLLM (Dual Cache), our method achieves a comparable results with a small accuracy gap ($<1\%$). The performance gap between Fast-dLLM (Dual Cache) with Fast-dLLM is mainly due to the use of blockwise cached attention mechanism: at the start of each block, all tokens perform one full-attention forward pass, and all prefix and suffix tokens' KV states are cached and remain fixed for the rest decoding steps of this block. This differs from full attention and introduces a small accuracy gap even before applying speculation.

We observe two main sources of the slight accuracy differences ($<1\%$) between our BlockSpec with Fast-dLLM (Dual Cache):

- Higher parallelism changes diffusion trajectories. Unlike autoregressive speculative decoding—which is strictly lossless because generation is left-to-right and logits align exactly—discrete diffusion LLMs update multiple masked positions per step. Changing the number of tokens decoded per step (i.e., increasing parallelism) changes the trajectory of updated tokens, which can slightly alter intermediate logits. As visualized in Appendix A.8, BlockSpec's more aggressive early-stage parallelism can cause small deviations at a few key tokens, leading to minor accuracy differences.
- Inter-block speculation introduces additional blockwise attention and KV-cache discrepancies. Fast-dLLM (Dual Cache) caches KV states only within a single block. BlockSpec additionally performs inter-block blockwise cached attention, where drafts of next block attend to the shared root of the current block as we illustrated in Appendix A.3. This differs slightly from the full-attention behavior used during training, and introduces another small source of deviation.

Across all datasets, both effects contribute only marginally, and the overall degradation remains $<1\%$, which is within the range expected for discrete diffusion decoding under modified trajectories.

We thank the reviewer again for prompting this clarification. In the revised manuscript, we added a concise explanation of these factors in the experimental analysis to provide a more complete discussion of accuracy differences.

Q5:W1: There is no empirical evidence to support the argument that the low confidence tokens are due to plausible candidates. For example, the model would display low confidence if the problem was too complex for it to handle.

Response to Q5:

We thank the reviewer for this insightful comment. We fully agree that low-confidence tokens may arise from multiple causes, including (i) the presence of several plausible candidates and (ii) inherently complex problems where the model is uncertain regardless of candidate ambiguity.

We would like to clarify that our method does not rely on low confidence being caused specifically by plausible alternatives. The mention of “plausible candidates” in the original submission was intended as a motivating example that inspired the design of our parallel speculative exploration. The core assumption behind BlockSpec is broader: even when a token has low confidence, it may still be correct, and verifying multiple possibilities in parallel can improve decoding efficiency without requiring confidence to be high.

Importantly, this assumption remains valid under the reviewer’s “complex problem” scenario as well. When the model faces a difficult reasoning step and produces low-confidence candidates, speculative parallel verification is still beneficial: the correct token—if present—can be efficiently validated and accepted, leading to the same decoding speedup.

We appreciate the reviewer for highlighting this point. In the revised version, if the reviewer doesn't mind, we have explicitly incorporated this broader interpretation of the causes of low confidence (including plausible candidates and complex problem) and clarified that our assumption in Sec1 (highlighted in yellow).

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:32 Everyone

Comment:

Q6:W2: .. on Fast-dLLM must be a weakness?

Response to Q6: We thank the reviewer for this insightful observation. Indeed, Fast-dLLM (Dual Cache) accelerates the original LLaDA model by introducing both prefix cache and suffix cache, effectively turning full attention at training time into a blockwise causal attention pattern at inference time. This train-test mismatch is empirically known to cause a small amount of performance degradation.

However, we believe using such blockwise causal attention as the baseline remains meaningful and valuable for the following reasons:

- The accuracy loss is minor and controlled. Both the Fast-dLLM paper and our experiments show that this attention gap leads to only about 1% accuracy drop, while bringing more than 7× speedup over the original LLaDA. In many practical settings, this trade-off between a minor loss in accuracy and a substantial gain in efficiency is acceptable. Also, the train-test mismatch can be further alleviated by updating the prefix and suffix KV caches more frequently, rather than only once at the beginning of each block as described in Sec.~3.4.
- Blockwise causal models are an active and promising direction. Recent works have demonstrated that starting from a full-attention model and then applying blockwise causal attention with building blockwise causal variants of autoregressive models[1][2][3] can yield models that jointly optimize

accuracy and speed. Our speculative decoding framework is fully compatible with these architectures and can be applied seamlessly to such blockwise causal diffusion or AR–diffusion hybrids.

In this sense, the dependency on Fast-dLLM should be viewed not as a specific weakness of our method, but as a design choice aligned with a broader and increasingly popular family of blockwise causal architectures, to which BlockSpec can naturally extend.

[1] Set Block Decoding is a Language Model Inference Accelerator

[2] Sequential Diffusion Language Models

[3] Fast-dLLM v2: Efficient Block-Diffusion LLM

Q7: W3: Despite its fast speed, BlockSpec shows similar performance to Fast-dLLM (dual cache). Formal Speculative Decoding methods maintain the original performance of the target models.

Response to Q7: Same with our Response to Q4

Q8: W4: There has been no ablation study on the role of 'medium-confidence tokens'.

Response to Q8: We thank the reviewer for pointing out this source of confusion. The phrase “medium-confidence tokens” in our original submission was a wording mistake. What we intended to describe are simply the top-k tokens among the low-confidence tokens, which in practice, often fall in a probability range around 0.3–0.6. These tokens were never meant to denote a distinct category separate from low-confidence tokens.

We appreciate the reviewer for highlighting this ambiguity. In the revised manuscript, we have removed the term “medium-confidence tokens” and replaced it with a clearer description of top-k candidates within low-confidence positions (highlighted in green). We hope this clarification resolves the misunderstanding and more accurately reflects our method’s design.

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:35 Everyone

Comment:

Q9: Short generation length. The generation lengths are fixed to 512, which is relatively short.

Response to Q9: Thanks for raising this concern. In the initial submission, we fixed the generation length to 512 primarily to ensure a controlled and fair comparison across all evaluated decoding methods. This setting follows common practice in recent diffusion-LLM work and matches the evaluation protocols of Fast-dLLM, LLaDA, Dream, and other related models (typically evaluated at lengths 256 or 512).

In the revised manuscript, we have added a new ablation study in Appendix A.9 evaluating BlockSpec and Fast-dLLM (Dual Cache) under longer generation lengths. Since our base model LLaDA is trained with a maximum context length of 4096 tokens (prompt + response), we tested generation lengths of 128, 256, 512, 1024, and 2048. Across all settings, BlockSpec consistently provides substantial throughput improvements over the baseline, demonstrating that the benefits of speculative parallel decoding are not restricted to short outputs. We present a quick result below. More explanation can be seen in Appendix A.9:

Fast-dLLM (Dual Cache): 128 (1.5s), 256 (3.2s), 512 (6.6s), 1024 (10.2s), 2048 (22.4s)

BlockSpec (W2D2): 128 (1.0s), 256 (2.3s), 512 (5.0s), 1024 (8.7s), 2048 (21.2s)

Across all generation-length settings, BlockSpec consistently delivers meaningful inference speedup. We also observe that the relative improvement becomes smaller as the generation length increases (e.g., at 1024 and 2048 tokens). This behavior arises from the characteristics of current discrete diffusion masked

models: after a certain number of blocks, the model typically predicts an [EOS] token, and all remaining positions are filled with placeholder [EOS] tokens. Once [EOS] is emitted, the decoding of each subsequent block requires only a single step, since the entire block is deterministically populated with [EOS].

Because BlockSpec does not modify the model's intrinsic denoising results, both BlockSpec and the baseline revert to a one-step-per-block pattern after [EOS] is generated. This naturally reduces the relative speedup advantage at longer sequence lengths, where a larger fraction of the decoding process occurs after [EOS] has already been produced.

To illustrate this observation, Appendix~A.9 also visualizes the average number of decoding steps across blocks (with block size = 32). As shown in the figure, BlockSpec consistently requires fewer steps in the earlier blocks—where meaningful tokens are generated—while the latter blocks collapse to a single decoding step due to early [EOS] emission.

We appreciate the reviewer for prompting this useful analysis, and we hope the expanded results more clearly illustrate how generation length interacts with speculative parallel decoding efficiency.

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:43 Everyone

Comment:

Following is a demo response of our method's for a AIME question. The correct result is 116 and all of baseline models (llada, Fast-dLLMs) and our model get a wrong result.

Question: Please solve the following math problem. Requirements:

1. Think step by step.
2. Show each step of calculation and derivation in detail.
3. The final answer must be placed in `\boxed{}`

Jen enters a lottery by picking distinct numbers from numbers are randomly chosen from She wins a prize if at least two of her numbers were of the randomly chosen numbers, and wins the grand prize if all four of her numbers were the randomly chosen numbers. The probability of her winning the grand prize given that she won a prize is

where and are relatively prime positive integers. Find .

Answer: To solve the problem, we need to determine the probability that Jen wins the grand prize given that she wins a prize. Let's break this down step by step.

Step 1: Calculate the total number of ways to choose 4 numbers from (S)

The set (S) has 10 elements. The number of ways to choose 4 numbers from 10 is given by the combination formula:
$$\binom{10}{4} = \frac{10!}{4!(10-4)!} = \frac{10 \times 9 \times 8 \times 7}{4 \times 3 \times 2 \times 1} = 210$$

Step 2: Calculate the number of ways to choose 4 numbers such that at least two of them are 2

We need to count the number of ways to choose 4 numbers such that at least two of them are 2. This can be done by subtracting the number of ways to choose 4 numbers such that none of them are 2 from the total number of ways to choose 4 numbers.

The number of ways to choose 4 numbers from the remaining 8 elements (since 2 is excluded) is:
$$\binom{8}{4} = \frac{8!}{4!(8-4)!} = \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2 \times 1} = 70$$

Thus, the number of ways to choose 4 numbers such that at least two of them are 2 is:
$$[210 - 70 = 140]$$

Step 3: Calculate the probability that Jen wins the grand prize given that she wins a prize
The probability that Jen wins the grand prize given that she wins a prize is the ratio of the number of favorable outcomes to the total number of outcomes: $\left[\frac{140}{210} = \frac{2}{3} \right]$

Step 4: Find $(m + n)$

Here, $(m = 2)$ and $(n = 3)$. Therefore, $(m + n = 2 + 3 = 5)$.

Final Answer

$\boxed{5}$

Add:

Official Comment by Reviewer VSE3

Official Comment by Reviewer VSE328 Nov 2025, 09:07Everyone

Comment:

Thank you for the detailed replies. Most of my concerns are now addressed. I have one remaining question.

Autoregressive speculative decoding is mathematically exact, but BlockSpec is not. I understand that the authors observe only small accuracy differences in the reported benchmarks. However, these benchmarks are not very difficult. This is why I asked about AIME. I accept that current diffusion LLMs cannot solve AIME, so experiments there are not meaningful.

Instead, I would like to ask on a high level: in what situations do the authors expect BlockSpec to struggle? For example:

- (1) On challenging reasoning tasks, could trajectory mismatch accumulate and cause larger performance declines?
- (2) When low-confidence tokens appear repeatedly, could the speculative process become unstable?
- (3) In long-horizon tasks, could inter-block speculation lead to additional errors?

If the authors have any expectations or intuitions about these potential failure cases, it would help me better understand the robustness of the method.

Thank you.

Add:

Replying to Official Comment by Reviewer VSE3

Official Comment by Authors

Official Comment by Authors02 Dec 2025, 13:17Everyone

Comment:

We sincerely thank the reviewer for the insightful follow-up question. This is an insightful point, and we are glad to further discuss the situations where BlockSpec may struggle. Our newly revised manuscript now includes an Exactness and Failure Analysis section in the Appendix, where we provide a formal exactness proof result under idealized conditions and an empirical failure case study. We summarize our intuitions and observations here.

High-level intuition: Our analysis shows that BlockSpec is mathematically exact under an idealized setting—cached prefix/suffix KV states, no inter-block speculation, and deterministic top-1 decoding. Under this condition, BlockSpec and Fast-dLLM (dual cache) produce identical trajectories. Therefore, all discrepancies between the two methods arise solely from differences in the parallel decoding trajectories introduced by speculative lookahead. This provides a clear reference point for understanding potential failure cases.

Empirical findings from Math (Levels 1–5). We evaluated BlockSpec across difficulty-separated Math tasks. As reported in the updated Appendix, BlockSpec and the baseline differ by only ~1% overall. However, failures concentrate on Level-4 problems rather than the easiest or the hardest ones. Our case study indicates these instances require both parallel decoding and mid-range multi-step reasoning—conditions under which speculative trajectories may diverge from those of the baseline.

Replies to the reviewer's three scenarios

(1) Challenging long-horizon reasoning tasks. Interestingly, our experiments show that very hard (Level-5) problems do not incur significant accuracy degradation. The intuition is that when reasoning becomes highly demanding, both BlockSpec and the baseline naturally collapse to low parallelism (near single-token decoding), bringing BlockSpec closer to the mathematically exact regime. Thus, although trajectory mismatch can accumulate in principle, we do not observe pronounced degradation on the hardest tasks.

(2) Repeated low-confidence tokens Repeated low-confidence positions do not destabilize BlockSpec. In fact, this is the regime where BlockSpec behaves most similarly to the baseline: when all positions exhibit low confidence, the tree becomes shallow; draft expansion collapses to top-1; the method reverts to near-exact behavior. Therefore, low-confidence streaks tend to reduce parallelism rather than introduce harmful deviations, and in our tests they do not lead to unstable decoding.

(3) Long-horizon tasks with inter-block speculation. Inter-block speculation can introduce additional trajectory differences, but empirically the effect is mild. From our failure analysis, most errors stem from a small number of reasoning-critical tokens being decoded differently; these errors can originate from both intra-block and inter-block speculation. We do not observe systematic degradation attributable specifically to inter-block interactions.

In short, BlockSpec tends to struggle not in fully hard reasoning tasks nor in low-confidence regions, but rather in medium-difficulty reasoning scenarios where:

- parallel decoding remains active,
- yet the reasoning chain is sensitive to perturbations in the trajectory.

Finally, we would like to thank the reviewer again for raising this question. Our preliminary analysis suggests that such failure cases illuminate a fundamental trade-off between parallel decoding and maintaining stable reasoning chains in diffusion LLMs. We hope that these insights will help guide future research on more robust and principled speculative decoding mechanisms for dLLMs.

Add:

Official Review of Submission6926 by Reviewer XYVg

Official Review by Reviewer XYVg24 Oct 2025, 13:49 (modified: 12 Nov 2025, 09:15)EveryoneRevisions

Summary:

This paper proposes BlockSpec, a training-free, blockwise speculative decoding method for diffusion language models (dLLMs). Through tree-structured trajectory generation and block-level parallel verification, the method achieves efficient decoding with high parallelism even under low-confidence conditions. The paper further introduces intra-block and inter-block speculation mechanisms, evaluates performance on reasoning and code-generation benchmarks, and analyzes computational overhead and limitations, reporting up to a 14× speedup over a vanilla dLLM.

Soundness: 2: fair

Presentation: 2: fair

Contribution: 1: poor

Strengths:

1. The proposed BlockSpec framework delivers substantial decoding acceleration, reducing iteration steps by up to 40% and achieving 7–14× speedups over vanilla diffusion LLMs, while maintaining comparable output quality on reasoning and code-generation benchmarks.
2. By combining tree-based trajectory generation with block-level parallel verification, the method effectively mitigates low-confidence degradation in diffusion LLMs, sustaining parallelism without compromising accuracy.

Weaknesses:

1. According to Fig. 3(3), all draft blocks are invisible to subsequent masked tokens. Thus, when unmasking a block, the process essentially reduces to dLLM-style generation with a generation length equal to the block size, which can lead to issues such as prematurely forcing an answer or emitting eos early. This suggests that the current blockwise attention mask is problematic.
2. Based on Tables 1 and 2, BlockSpec's high throughput (TPS) speedup seems attributable not only to fewer forward passes but also to reduced matrix computation induced by the specialized attention mechanism. However, the experiments and appendix do not clearly isolate or quantify this contribution.

Questions:

1. See Weakness 1.
2. What is the structure of the blockwise attention mask in inter-block speculation.
3. The experiments appear to show that a blockwise attention mask that breaks the bidirectional attention mechanism can still yield generation that is nearly lossless relative to full bidirectional attention. Have the authors investigated the underlying reasons for this?

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:38 Everyone

Comment:

Q1: According to Fig. 3(3), all draft blocks are invisible to subsequent masked tokens. Thus, when unmasking a block, the process essentially reduces to dLLM-style generation with a generation length equal to the block size, which can lead to issues such as prematurely forcing an answer or emitting eos early. This suggests that the current blockwise attention mask is problematic.

Response to Q1: We appreciate the reviewer for raising this important point. The concern is caused by a misleading illustration in Fig. 3(3), rather than by the actual design of our method. In our implementation, we strictly follow the Fast-dLLM (Dual Cache) blockwise KV-cache mechanism.

At the beginning of each block's speculative decoding, we run a full-attention forward pass and compute complete KV states for (i) all prefix tokens (prompt + decoded tokens), (ii) all suffix [MASK] tokens, and (iii) the current block. These KV states for prefix and suffix tokens are then cached and remain fixed throughout the entire block, just like Fast-dLLM(Dual Cache). During subsequent draft-tree expansions and verification, each draft token is visible to all cached prefix and suffix tokens, while only the KV entries associated with the explored draft tokens are updated. Therefore, draft decoding does not reduce to isolated dLLM-style generation; the figure in initial submission simply failed to depict this correctly.

In the revised manuscript, we have redrawn Fig. 3(3) to clearly show that:

- all draft tokens are visible to all cached prefix and suffix states, and

- draft tokens from different draft nodes are mutually invisible to avoid cross-node interference.

We also acknowledge that our earlier wording of “blockwise causal attention” may have contributed to the confusion. The term was not an accurate description of the mechanisms used in our method.

- In intra-block speculation, the correct term is blockwise individual attention: draft tokens are visible to all blockly-cached prefix and suffix tokens, but remain mutually invisible across draft branches.
- In inter-block speculation, we do not impose a causal dependency between consecutive blocks. Instead, the roots of two adjacent blocks retain full bidirectional visibility: all draft tokens of the next block are visible to the current block’s root, and all draft tokens of the current block are likewise visible to the next block’s root. This shared-root visibility is consistent with Fast-dLLM’s observation that the KV activations of prefix/suffix tokens remain highly similar across adjacent denoising steps.
- The only true causal structure in our masking scheme lies between the draft tokens of the decoding blocks and the blockly-cached prefix/suffix tokens. Prefix and suffix states are treated as the stable context(also cached) that each decoding block condition on, while all draft-level interactions (both intra- and inter-block) avoid introducing additional causal constraints.

To prevent further misunderstanding, we have updated the terminology in Sec. 3.1 and Sec. 3.3 and clarified the handling of suffix tokens and blockly-cached KV states. We also provide revised attention-mask diagrams in Appendix A.3 that accurately illustrate the intra-block, inter-block, and prefix/suffix visibility patterns.

Q2: What is the structure of the blockwise attention mask in inter-block speculation.

Response to Q2: Thanks for this suggestion. In the revised version, we have added a complete description of our inter-block speculation mechanism in Appendix A.3, including updated diagrams and detailed explanations of how visibility and KV caching are handled across adjacent blocks. The corresponding Inter-Block Attention Mask is also illustrated in Appendix A.3, providing a clear depiction of the masking rules used during inter-block draft expansion and verification.

A direct textual response is: The inter-block attention mask extends the blockwise formulation while preserving the bidirectional attention for inter-block roots. In our implementation, we do not impose a causal dependency between consecutive blocks. Instead, we retain full bidirectional visibility between the roots of the two adjacent blocks: all draft tokens of the next block are visible to the root of the current block, and all draft tokens of the current block are likewise visible to the root of the next block. In other words, drafts of two blocks share each other’s root Key/Value states, which is consistent with prior findings in Fast-dLLM that the KV activations of prefix and suffix tokens remain highly similar across adjacent denoising steps. Meanwhile, draft tokens across different blocks remain mutually invisible, ensuring that intra-block exploration is fully isolated.

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 15:38 Everyone

Comment:

Q3: The experiments appear to show that a blockwise attention mask that breaks the bidirectional attention mechanism can still yield generation that is nearly lossless relative to full bidirectional attention. Have the authors investigated the underlying reasons for this?

Response to Q3: The reviewer’s question is very perceptive and important. Indeed, in principle, a model with a clear train–test attention mismatch would be expected to suffer noticeable accuracy degradation. We have conducted some experiments, we are happy to share our explanations and findings here for more discussion.

As discussed in Q1 and Q2(also shown in AppendixA.3), our method actually preserves substantial bidirectional structure in both suffix handling and inter-block interactions. Our design does not replace full bidirectional attention with a fully blockwise-causal mask. Instead: 1) all draft tokens remain fully visible to blockly-cached prefix and suffix tokens, and 2) adjacent blocks retain full bidirectional visibility between their root states. These preserved components of the bidirectional structure are crucial for maintaining generation quality and explain why our accuracy closely matches the Fast-dLLM baseline.

This design choice is directly motivated by experiments that just confirming the reviewer’s insight --breaking bidirectional attention aggressively leads to dramatic degradation. We have tested two “fully causalized” alternatives: 1) applying strict blockwise causal attention (no bidirectional reuse of prefix/suffix KV), and 2) imposing a fully causal link between consecutive blocks (next block only sees current block → unidirectionally). Both variants suffered >15% accuracy drops, confirming that strong deviations from the bidirectional structure of diffusion LLMs severely impair model quality. These tests reinforced the necessity of retaining selective bidirectional visibility.

Our implementation is guided by Fast-dLLM’s empirical finding that prefix/suffix KV activations remain highly similar across adjacent denoising steps. Leveraging this property, our method preserves bidirectional context in two key places: a) Prefix/suffix tokens: At each block boundary, we perform a full-attention forward pass to obtain fresh KV states and treat them as blockly-cached KV, shared across all decoding steps within the block. b) Inter-block roots: We retain full bidirectional visibility between the roots of the two adjacent blocks, allowing both blocks’ draft tokens to condition on a shared, stable representation.

Together, these designs maintain much of the functional behavior of full bidirectional attention while still enabling efficient speculative parallel decoding. We appreciate the reviewer for raising this question. It aligns closely with our own diagnostic observations and motivates us to further elaborate these mechanisms in the revised manuscript.

Q4: W2: Based on Tables 1 and 2, BlockSpec’s high throughput (TPS) speedup seems attributable not only to fewer forward passes but also to reduced matrix computation induced by the specialized attention mechanism. However, the experiments and appendix do not clearly isolate or quantify this contribution.

Response to Q4: We appreciate the reviewer for this very accurate observation. Indeed, yes, the throughput improvements over vanilla lLaDA/dream model come from two sources: (1) fewer denoising steps enabled by our proposed blockwise speculative parallel decoding, and (2) faster per-step computation, actually inherited from the dual KV-cache mechanism introduced by Fast-dLLM (Dual Cache).

The latter is an important optimization by Fast-dLLM(Dual Cache): by reusing blockly-cached prefix and suffix KV states within each block, the attention computation becomes significantly cheaper than in the original LLaDA model. Since BlockSpec is built directly on top of Fast-dLLM (Dual Cache), our per-step efficiency benefits are essentially the same as those already established in that baseline. Our work focuses on improving parallel decoding efficiency, not introducing additional optimizations beyond the dual-cache design.

We acknowledge that our initial submission did not emphasize this baseline component clearly enough. In the revised version, we have: 1) added a detailed explanation of the dual KV-cache mechanism in Sec. 3.4, and 2) clearly highlighted Fast-dLLM (Dual Cache) as our baseline in Sec. 4.1 (yellow highlighted), to avoid further ambiguity.

To isolate the contribution of the KV-cache optimization, we can refer to the speed difference between Fast-dLLM and Fast-dLLM (Dual Cache) in the experiments. Since BlockSpec builds on the latter, we did not repeat this comparison in our ablations.

We thank the reviewer for pointing this out—it helped us improve the clarity of our presentation. We hope that the revised manuscript resolves the confusion regarding the role of dual KV caching and clarifies how BlockSpec builds on this mechanism.

Add:

Replying to Official Comment by Authors

Official Comment by Reviewer XYVg

Official Comment by Reviewer XYVg 25 Nov 2025, 13:16 Everyone

Comment:

Thank you for the detailed responses and clarifications. I appreciate the effort the authors have put into addressing the concerns. However, I still identify two major issues that remain after the rebuttal: 1. Incremental contribution beyond Fast-dLLM (Dual Cache). BlockSpec's core addition appears to be a tree-structured intra-block and inter-block speculative schedule, a design pattern that is conceptually borrowed from ARM-style speculative decoding. While such scheduling is reasonable, the methodological novelty is modest, and most of the technical foundation—including the blockwise KV caching, reduced attention cost, and bidirectional prefix/suffix reuse—comes directly from Fast-dLLM (Dual Cache). As a result, the overall contribution does not seem sufficiently substantial. 2. Most of the observed speedup appears to come from Dual Cache rather than BlockSpec itself. Based on the results in Tables 1 and 2 and the clarification in the rebuttal, the large throughput gains relative to vanilla dLLMs are largely inherited from the Dual Cache mechanism. The additional improvement brought by BlockSpec over Fast-dLLM (Dual Cache) is relatively small. This suggests that the core acceleration originates from Dual Cache, while BlockSpec's incremental benefit is limited. Given these remaining concerns, I am inclined to maintain my previous score.

Add:

Replying to Official Comment by Reviewer XYVg

Official Comment by Authors

Official Comment by Authors 26 Nov 2025, 12:50 Everyone

Comment:

We sincerely thank the reviewer for the thoughtful follow-up. These questions address the core of our contribution, and we appreciate the opportunity to clarify them more precisely.

Issue 1: Incremental contribution beyond Fast-dLLM (Dual Cache)

We agree that BlockSpec is built on Fast-dLLM (Dual Cache). The additional descriptions of prefix/suffix KV caching and blockwise attention in the revised version were added only to resolve earlier confusion and are not claimed as contributions. They simply clarify the baseline substrate on which our method operates.

Beyond these inherited components, BlockSpec introduces a new speculative decoding framework tailored specifically to diffusion LLMs, including several elements absent in Fast-dLLM and ARM-style speculative decoding:

- (a) Blockwise speculative decoding for arbitrary-order denoising. AR speculative decoding assumes left-to-right causal generation, while dLLMs must construct denoising trajectories over multi-token masked sequences. BlockSpec provides a new formalism that jointly handles token identity, position, and ordering under bidirectional attention.
- (b) A principled analysis and remedy for low-confidence degeneration. We show that Fast-dLLM frequently collapses to single-token decoding at low-confidence positions. This motivates our draft-token collection policy that explicitly targets these positions to sustain parallelism.
- (c) A carefully explored tree-structured draft strategy. Our tree design—root top-k expansion, deeper top-1, left-biased growth, duplication control, and depth-width trade-offs—results from extensive empirical

exploration (Appendix A.2). Although inspired by AR speculative patterns, adapting them to arbitrary-order trajectories and much higher verification cost in dLLMs required substantial redesign. We hope these findings offer guidance for future dLLM speculative frameworks.

(d) Inter-block speculation (a new form of segmented parallelism). Enabled by diffusion models' bidirectional visibility, this cross-block lookahead has no analogue in AR models. To our knowledge, BlockSpec is the first to introduce such a mechanism.

Issue 2: Most speedup comes from Dual Cache rather than BlockSpec

We agree that Fast-dLLM (Dual Cache) introduced the first major acceleration for dLLMs, and we do not claim it as our contribution. Our work builds upon this strong baseline—as do nearly all recent dLLM inference methods.

Our method sits on top of this baseline. Within this setting, BlockSpec provides additional speedup on an already highly optimized system, which is substantially more challenging than accelerating vanilla dLLMs. Despite this, BlockSpec achieves: 1) $\approx 40\%$ reduction in decoding steps, 2) $\approx 30\%$ decrease in end-to-end latency, 3) tokens-per-step from 2.6 $\rightarrow$ 4.3 (+65% increase)

This increased parallelism is meaningful, as tokens-per-step directly upper-bounds achievable speedup, and future optimized verification kernels can further amplify these benefits. We report speedups over vanilla models because they remain the standard comparison in diffusion LLM literature; e.g., DPad [1] achieves $\sim 4.14\text{--}10.32\times$, while BlockSpec reaches $7\text{--}14\times$ with similar accuracy.

Importantly, Dual Cache and BlockSpec optimize orthogonal dimensions: 1) Dual Cache reduces redundant computation (systems-level). 2) BlockSpec reduces required denoising iterations (algorithmic-level). Their gains accumulate, and BlockSpec's improvements remain visible even atop Fast-dLLM (Dual Cache).

We appreciate the reviewer's assessment and hope these clarifications more accurately reflect the novelty and value of BlockSpec. As an early attempt at speculative decoding for diffusion LLMs—centered on low-confidence degeneration, trajectory-space draft generation, and inter-block parallelism—we believe our contributions are meaningful and foundational for future research.

[1] DPad: Efficient Diffusion Language Models with Suffix Dropout

Add:

Official Review of Submission6926 by Reviewer DaQS

Official Review by Reviewer DaQS18 Oct 2025, 16:25 (modified: 12 Nov 2025, 09:16)EveryoneRevisions

Summary:

This paper introduces BlockSpec, a novel, training-free speculative decoding framework designed to mitigate the "low-confidence degradation" problem in diffusion-based Large Language Models (dLLMs), where parallel decoding efficiency collapses under uncertainty. BlockSpec addresses this by exploring multiple decoding trajectories simultaneously using a new tree-based generation strategy to create candidate token blocks in parallel. These blocks are then efficiently validated through a corresponding blockwise verification mechanism tailored for the any-order, bidirectional nature of dLLMs, with performance further enhanced by intra- and inter-block speculation. Experimental results demonstrate that this approach significantly reduces decoding steps, achieving up to a 14x speedup over vanilla dLLMs and a 1.3x improvement over strong baselines, establishing a more robust and efficient inference paradigm.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

The proposed BlockSpec in the paper is a novel training-free decoding acceleration strategy that attempts to consider more branches to alleviate the degradation problem in parallel decoding. Experimental results show that this method is effective on different datasets, different models, and different hardware.

Weaknesses:

1. In Section 3.3, the authors state that "...allows key-value caching on the prompt...". However, the rest of the paper (including the experimental settings) does not clarify whether KV caching was actually used. If KV cache was indeed applied, the authors should provide its design details and implementation description.
2. The stated motivation of the paper is to address the "low-confidence degradation problem of parallel decoding." However, intuitively, the proposed method should lead to better performance, which is not reflected in the experimental results. I suggest that the authors revise the Introduction to better align it with the presented method and findings.
3. In Figure 3(3), for the current block, the subsequent masked tokens (the rightmost three blocks with ellipses) should be invisible. Is this a plotting error? This visualization seems to indicate that the model is aware that this is the final block to be generated, which could substantially influence the effective generation length and thereby degrade model performance.

Questions:

1-2. See weakness 1 and 3. 3. Although Section 3.4 introduces the concept of inter-block speculation, it still lacks sufficient implementation details. For example, it does not appear in any of the provided pseudocode segments. My question is: if both the current block and the next block are generated simultaneously, and each block is supposed to attend to the prompt as well as all preceding blocks, then which branch of the current block should the next block attend to?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Comment by Authors

[Official Comment](#) by Authors 19 Nov 2025, 15:41 Everyone

Comment:

Q1: W1: In Section 3.3, the authors state that "...allows key-value caching on the prompt...". However, the rest of the paper (including the experimental settings) does not clarify whether KV caching was actually used. If KV cache was indeed applied, the authors should provide its design details and implementation description.

Response to Q1:

We thank the reviewer for raising this important clarification. Yes—our method does use KV caching, and the caching mechanism we adopt is essentially the same dual-cache design introduced in Fast-dLLM (Dual Cache), which serves as our baseline. In the original submission, we did not provide sufficient detail on this component due to space constraints, and we fully agree that this omission could lead to confusion.

In the revised manuscript, we have added a dedicated section (Sec. 3.4) that explicitly describes how suffix tokens and KV caching are handled in BlockSpec. To further highlight the relationship between our method and the dual-cache baseline, we also expanded the explanation of Fast-dLLM (Dual Cache) in Sec. 4.1 (highlighted in yellow).

Briefly, our KV-cache mechanism operates as follows: At the start of each block's speculative decoding, we recompute the full-attention KV states for 1) the prefix (prompt + previously decoded tokens), 2) the suffix (remaining [MASK] tokens), and 3) the current block root.

The prefix and suffix KV states are then cached and kept fixed throughout all decoding steps within the block. These cached states are reused for all draft expansions and verification passes, and only the KV entries corresponding to the actively explored draft tokens are iteratively updated. This behavior matches the dual-cache assumption of Fast-dLLM that prefix/suffix KV activations remain highly similar across adjacent denoising steps.

For full details—including diagrams and implementation notes—please refer to the newly added Sec. 3.4 in the revised version. We sincerely appreciate the opportunity to clarify our KV-cache design, and we hope the revised manuscript fully resolves any confusion regarding this component.

Q2: W3: In Figure 3(3), for the current block, the subsequent masked tokens (the rightmost three blocks with ellipses) should be invisible. Is this a plotting error? This visualization seems to indicate that the model is aware that this is the final block to be generated, which could substantially influence the effective generation length and thereby degrade model performance.

Response to Q2: We appreciate the reviewer for this accurate observation. Yes—this was indeed a plotting error. In our actual implementation, the model never hides or discards the suffix [MASK] tokens during blockwise speculative decoding. All remaining masked positions are preserved and included in the computation, exactly as in Fast-dLLM (Dual Cache). Their KV states are refreshed once at each block boundary and then reused throughout the block, contributing both to correctness and to per-step efficiency.

In the original figure, we omitted the suffix tokens for visual simplicity when illustrating the draft blocks, which unintentionally suggested that the model knew it had reached the final block. We have corrected this in the revised manuscript as: 1) Figure 3(3) has been updated to properly include all suffix [MASK] tokens, 2) Sec. 3.4 now explicitly describes our suffix-handling and KV-cache mechanism, and 3) Appendix A.3 provides an updated inter-block attention-mask diagram where suffix visibility is also shown.

We sincerely appreciate the reviewer for pointing out this issue and for giving us the opportunity to improve the correctness and clarity of our visualizations.

Add:

Official Comment by Authors

[Official Comment](#) by Authors 19 Nov 2025, 15:41 Everyone

Comment:

Q3: Although Section 3.4 introduces the concept of inter-block speculation, it still lacks sufficient implementation details. For example, it does not appear in any of the provided pseudocode segments. My question is: if both the current block and the next block are generated simultaneously, and each block is supposed to attend to the prompt as well as all preceding blocks, then which branch of the current block should the next block attend to?

Response to Q3: Appreciate for raising this question. The reviewer is absolutely right that the initial submission did not provide enough implementation detail for inter-block speculation, largely due to page-limit constraints. We have significantly expanded this part of the paper in the revised version.

In the updated manuscript, Appendix A.3 now provides a complete exposition of inter-block speculation, including its motivation, triggering conditions, draft-tree structure, step-by-step decoding demo, and the full attention-mask design. We have also added clarifying text in the main paper (Sec. 3.5).

Regarding the reviewer’s specific question—which branch of the current block does the next block attend to?—we apologize that the original text did not make this explicit. The answer is: The next block attends only to the root (i.e., the accepted version) of the current block, not to any draft branches.

All draft variants within the current block remain mutually invisible to the next block. The next block receives a single, stable root representation from the current block. This root–root connection is implemented as full bidirectional visibility, as detailed in Appendix A.3.

This design avoids ambiguity—there is only one representation from the current block that the next block can rely on—and preserves the stability required for accurate denoising. Prefix and suffix KV states remain blockly-cached and visible to all blocks, as described in Sec. 3.4.

We sincerely thank the reviewer for highlighting this gap. We hope the expanded description and newly added diagrams in Appendix A.3 can better clarify how inter-block speculation is implemented.

Q4: The stated motivation of the paper is to address the “low-confidence degradation problem of parallel decoding.” However, intuitively, the proposed method should lead to better performance, which is not reflected in the experimental results. I suggest that the authors revise the Introduction to better align it with the presented method and findings.

Response to Q4: We thank the reviewer for this valuable comment. Our motivation is indeed to address the “low-confidence degradation problem of parallel decoding”, but we realize that our original phrasing may have unintentionally suggested that low-confidence tokens harm generation quality. What we aim to resolve is a different issue: Low-confidence tokens cause a sharp drop in decoding parallelism, not a drop in model accuracy.

In existing diffusion-based parallel decoding methods, a low-confidence position typically forces the model to decode only one token per step, collapsing parallelism and significantly slowing down inference. BlockSpec is designed specifically to restore parallel decoding efficiency in these regions—from 1 token per forward pass to more than 2–4 tokens per forward pass depending on configuration—while keeping model accuracy.

We appreciate the reviewer for this advice. And we think that our wording “degradation” may have introduced confusion. A term like “degeneration of parallelism” may be more accurate. To better align the introduction with our actual method and findings, we have revised the relevant motivation statements (highlighted in green), clarified that the issue concerns efficiency rather than accuracy, and polished the introduction accordingly.

Add:

Replying to Official Comment by Authors

Official Comment by Reviewer DaQS

Official Comment by Reviewer DaQS 21 Nov 2025, 12:46 Everyone

Comment:

I have carefully reviewed the revised manuscript. The paper correctly highlights an important limitation of parallel decoding: in many cases, the model still effectively generates only one token at a time. However, such uncertainty does not necessarily imply incorrectness, and recognizing this distinction is a valuable insight. That said, the paper still has the following limitations:

1. **Tree verification.** Apart from replacing node-level tokens with blocks, the proposed technique appears to be a rather direct adaptation of speculative decoding methods used in autoregressive models such as Eagle and Medusa. I recommend that the authors include a dedicated paragraph clarifying the conceptual and technical differences between their method and prior work.
2. **Block-size constraints.** Because the block size cannot be set too large, the feasible decoding region becomes restricted. The original motivation of dLLMs is to break the strict left-to-right generation order of autoregressive models and enable arbitrary-order generation. However, in

BlockSpec, the block-size constraint forces generation to remain within the current or next block, and this size cannot be increased substantially (e.g., it is typically set to 32). As a result, the method still effectively enforces a left-to-right generation pattern, which undermines model performance. I suggest evaluating the approach on a global-planning task such as Sudoku to diagnose this issue.

Considering both the strengths and the remaining limitations of this work, I am inclined to maintain my previous score.

Add:

Replying to Official Comment by Reviewer DaQS

Official Comment by Authors

Official Comment by Authors 26 Nov 2025, 12:44 Everyone

Comment:

We sincerely thank the reviewer for these thoughtful comments and appreciate the opportunity to further clarify the two remaining limitations raised in the review.

Limitation 1: Tree verification and the relation to AR speculative decoding

We fully agree that speculative decoding in AR models (e.g., Eagle, Medusa) provides an important conceptual precedent, and clarifying the similarities and differences is valuable. While our work is indeed inspired by classical speculative decoding, its migration into diffusion LLMs is beyond a direct adaptation. dLLMs differ fundamentally from AR models due to bidirectional attention, multi-token parallel decoding, and blockwise generation. These properties require substantial redesign for speculative decoding to function at all.

In our method, the blockwise draft tree is only one of several dLLM-specific components. In particular, our framework introduces other three key innovations:

- a) Blockwise speculative decoding framework that supports arbitrary-order token speculation. Our draft construction operates over trajectories rather than left-to-right token chains, enabling exploration and parallel verification over any token order and arbitrary numbers of unresolved positions—capabilities that AR speculative decoding cannot express.
- b) A principled analysis of low-confidence degeneration and a tailored draft trajectory generation strategy. We identify that low-confidence positions systematically collapse parallel decoding in dLLMs. Based on this observation, our draft-generation mechanism explicitly collects and accelerates these positions while preserving correctness via parallel verification—this motivation is unique to diffusion models and parallel decoding methods.
- c) Inter-block speculation—a new form of segmented parallel decoding unique to dLLMs. Inter-block speculation leverages dLLMs' bidirectional visibility and blockwise denoising to enable parallel lookahead across blocks. This segmented multi-block parallelism has no analogue in AR speculative decoding and, to our knowledge, BlockSpec is the first work to introduce this strategy.

For these reasons, BlockSpec is not a simple transfer of AR speculative methods. Instead, our design originates from analyzing low-confidence degeneration in dLLMs and developing a speculative decoding framework that is compatible with arbitrary-order denoising and blockwise attention. We conducted extensive empirical exploration to derive the current speculation strategies. As one of the earliest attempts at speculative decoding for diffusion LLMs, we hope our findings can inform future works like improved draft structures, more efficient block verification, and more precise candidate selection strategies.

We appreciate the reviewer's comment, and in the revision we will add a dedicated paragraph (as shown below) clearly contrasting BlockSpec with AR speculative decoding and explaining why dLLMs require a different design.

Limitation 2. Block-size constraints and semi-autoregressive behavior The reviewer correctly notes that one motivation of dLLMs is to allow arbitrary-order generation. At the same time, prior work has consistently shown that inference-time semi-autoregressive scheduling—performing any-order denoising within blocks while moving left-to-right across blocks—is critical for stable performance.

As demonstrated in Large Language Diffusion Models [1], the gap is dramatic:

- LLaDA-Instruct, unrestricted any-order decoding: 12.9
- LLaDA-Instruct, semi-autoregressive decoding: 73.8

This substantial difference indicates that, despite being trained with arbitrary-order denoising, current dLLMs rely heavily on semi-autoregressive progression during inference. BlockSpec is fully aligned with this established decoding paradigm. Our method does not introduce new left-to-right constraints; rather, it operates within the same blockwise progression already used in existing dLLMs and strengthens parallelism inside that structure through speculative decoding.

We appreciate the reviewer’s suggestion to include a global-planning task such as Sudoku. While integrating such tasks is beyond the scope of our current study—which focuses on understanding and improving inference behavior under practical dLLM decoding paradigms—we agree it is an interesting direction, and we plan to explore it in future work.

Add:

Replying to Official Comment by Authors

Official Comment by Authors

Official Comment by Authors 26 Nov 2025, 12:45 Everyone

Comment:

New paragraph to be added in the revision (final version):

Relationship to AR speculative decoding. Speculative decoding in autoregressive (AR) models typically relies on fixed left-to-right generation and causal masking, allowing token-by-token draft construction and one-step parallel verification. However, these mechanisms do not directly transfer to diffusion LLMs, whose decoding is governed by bidirectional attention, multi-token denoising, and blockwise semi-autoregressive progression. BlockSpec introduces several design elements specifically tailored to this setting: (i) a blockwise speculative decoding framework capable of constructing and verifying arbitrary-order denoising trajectories; (ii) a draft-generation mechanism motivated by the low-confidence degeneration issue unique to dLLMs; and (iii) an inter-block speculative formulation that exploits bidirectional visibility across adjacent blocks to enable segmented multi-block lookahead—an ability absent in AR models. Together, these components constitute a speculative decoding framework fundamentally different from AR approaches, reflecting the architectural and behavioral differences of diffusion-based LLMs.

Add:

Official Comment by Reviewer DaQS

Official Comment by Reviewer DaQS 26 Nov 2025, 18:58 (modified: 26 Nov 2025, 19:14) Everyone Revisions

Comment:

Thank you for the new response and the revised manuscript. All of my previous concerns have been fully addressed. The current version is well written and presents its arguments clearly. At this point, the only remaining limitation seems to be that the number of tree nodes cannot be very large; for instance, according to Line 399, most cases contain only three nodes. The acceleration benefits of speculative decoding largely come from maximizing GPU utilization, and on less powerful GPUs, a small number of tree nodes may limit the achievable speedup. I would be willing to keep my positive evaluation.

Add:

[About OpenReview](#)

[Contact](#)

[FAQ](#)

[Hosting a Venue](#)

[Sponsors](#)

[Terms of Use / Privacy Policy](#)

[All Venues](#)

[Donate](#)

[News](#)

OpenReview is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026 OpenReview

[OpenReview.net](#)

[Login](#)



[Go to ICLR 2026 Conference homepage](#)

PARD: Accelerating LLM Inference with Low-Cost PARallel Draft Model Adaptation



Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, Emad Barsoum

Published: 26 Jan 2026, Last Modified: 11 Apr 2026 ICLR 2026 Poster Everyone Revisions BibTeX CC BY 4.0

Keywords: LLM, Speculative Decoding, AI Infra, Low Cost Training

Abstract:

The autoregressive nature of large language models (LLMs) fundamentally limits inference speed, as each forward pass generates only a single token and is often bottlenecked by memory bandwidth. Speculative decoding has emerged as a promising solution, adopting a draft-then-verify strategy to accelerate token generation. While the EAGLE series achieves strong acceleration, its requirement of training a separate draft head for each target model introduces substantial adaptation costs. In this work, we propose **PARD (PARallel Draft)**, a novel speculative decoding method featuring target-independence and parallel token prediction. Specifically, PARD enables a single draft model to be applied across an entire family of target models without requiring separate training for each variant, thereby minimizing adaptation costs. Meanwhile, PARD substantially accelerates inference by predicting multiple future tokens within a single forward pass of the draft phase. To further reduce the training adaptation cost of PARD, we propose a COnditional Drop-token (COD) mechanism based on the integrity of prefix key-value states, enabling autoregressive draft models to be adapted into parallel draft models at low-cost. Our experiments show that the proposed COD method improves draft model training efficiency by 3x compared with traditional masked prediction training. On the vLLM inference framework, PARD achieves up to 3.67x speedup on LLaMA3.1-8B, reaching 264.88 tokens per second, which is 1.15x faster than EAGLE-3. Our code is available at <https://github.com/AMD-AGI/PARD>.

Primary Area: generative models

Submission Number: 15276

Filter by reply type...

Filter by author...

Sort: Newest FirstSort: Oldest First

19 / 19 replies shown

Add:

Paper Decision

Decision by Program Chairs 26 Jan 2026, 14:15 (modified: 06 Feb 2026, 11:09) Everyone [Revisions](#)

Decision: Accept (Poster)

Add:

Meta Review of Submission15276 by Area Chair nwsp

Meta Review by Area Chair nwsp 13 Jan 2026, 15:05 (modified: 09 Feb 2026, 12:14) Everyone [Revisions](#)

Summary:

This paper proposes PARD, a speculative decoding method that combines target-independence with parallel token prediction using mask-based drafting. Key strengths include: (1) a single draft model can accelerate an entire family of target models without per-target retraining, significantly reducing deployment costs; (2) the COD (Conditional Drop-token) mechanism reduces training cost by 3x while maintaining inference performance; (3) strong empirical results showing 3.67x speedup on LLaMA3.1-8B and 1.15x faster than EAGLE-3 under vLLM; (4) the method is simple and well-motivated. Weaknesses include limited comparison with other parallel speculative decoding methods (ParallelSpec, BiTA, PaSS) due to lack of vLLM implementations, and questions about whether speedups stem from same-family alignment rather than the method itself.

Reviewer Concerns:

Addressed: (1) The family alignment concern was convincingly addressed with a cross-family experiment showing a Qwen2.5-0.5B draft model (adapted via vocabulary replacement) achieves 3.46x speedup on LLaMA3.1-8B, nearly identical to the same-family 3.49x, demonstrating PARD's gains are not tied to model homogeneity. (2) Acceptance rate degradation as K increases was analyzed, showing only ~4% drop at K=4 while reducing draft passes by K-fold. (3) COD mechanism was justified via expected acceptance-length analysis. (4) Comparison with EAGLE/EAGLE-3 under identical vLLM settings was provided. Outstanding: Direct experimental comparison with ParallelSpec, BiTA, PaSS remains missing (though justified by lack of vLLM support), and tree attention experiments were not included (vLLM limitation).

Reviewer Scores:

Reviewer 6iwh (6): Would likely maintain score. Did not engage post-rebuttal but their concern about missing baselines was addressed with methodological comparison. Reviewer Gfez (6): Maintains score. Explicitly stated they maintain their positive evaluation after the detailed acceptance rate analysis. Reviewer GWvT (2→raised): Explicitly raised score after rebuttal, stating concerns were partially addressed. Likely raised to 4 or 6 given the thorough responses on target-independence, COD justification, and cross-family experiments. Reviewer MJ3S (4): Would likely maintain score. Continued to express concerns about missing baselines and tree verification, though the cross-family experiment partially addressed the homogeneity concern.

Add:

Official Review of Submission15276 by Reviewer 6iwh

Official Review by Reviewer 6iwh 31 Oct 2025, 22:20 (modified: 12 Nov 2025, 11:02) Everyone [Revisions](#)

Summary:

The paper introduces PARD, a speculative decoding framework that accelerates LLM inference through a target-independent, parallel draft model. It combines mask-based prediction and a novel Conditional

Drop-token (COD) training strategy to enhance both training and inference efficiency. PARD achieves up to 3.67× speedup on LLaMA3.1-8B and outperforms the current state-of-the-art EAGLE-3 model in both speed and generalizability.

Soundness: 3: good

Presentation: 3: good

Contribution: 2: fair

Strengths:

1. Unlike methods like EAGLE, PARD can accelerate an entire family of target models with a single draft model, significantly reducing adaptation and deployment costs.
2. Moreover, the use of masked tokens allows PARD to predict multiple tokens in a single pass, improving inference speed without sacrificing quality.
3. Extensive benchmarks on HumanEval, GSM8K, and SpecBench across multiple LLMs demonstrate consistent speedups and high acceptance rates compared to EAGLE and vanilla SD.

Weaknesses:

The paper introduces PARD as a parallel and target-independent speculative decoding method. However, the experimental section lacks direct comparisons with other recent parallel speculative decoding methods such as ParallelSpec, BiTA, or PaSS.

Questions:

Could the authors justify the omission of comparing to other recent parallel speculative decoding methods, and if possible, include some comparative results to support the performance and efficiency claims of PARD in the parallel decoding category?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 20 Nov 2025, 18:42 (modified: 20 Nov 2025, 18:44) Everyone Revisions

Comment:

Dear reviewer,

Thank you for recognizing the strengths of our work and for raising this valuable question. We address your concern below.

W1,Q1: Why does the paper not include direct comparisons with other recent parallel speculative decoding methods (e.g., ParallelSpec, BiTA, PaSS), and can the authors justify or supplement this omission?

A: This is an excellent question. We first clarify the rationale behind our baseline selection in the experiments, and then provide a methodological comparison between PARD and other recent parallel speculative decoding approaches.

Baseline selection and evaluation environment: Our primary goal was to evaluate performance in realistic, production-level scenarios. Therefore, all experiments were conducted under **vLLM**, a widely used high-performance inference engine in modern LLM services. Appendix F of the paper compares vLLM and the standard *Transformers* inference environment commonly used in academic work:

- For autoregressive baseline decoding, vLLM achieves nearly **2× the decoding speed** compared to Transformers.

- For VSD-style decoding, Transformers only yields **1.36× speedup**, while vLLM achieves **2.02× speedup**.

These results indicate a large gap between academic results obtained under Transformers and real-world serving performance. Since methods such as ParallelSpec, BiTA, and PaSS do not provide vLLM implementations, including them only with Transformers would create an unfair comparison and would not reflect the practical deployment environment. Therefore, we selected **EAGLE and EAGLE-3**, which currently offer the strongest real-system improvements and support high-performance inference stacks, as our main baselines.

Methodological comparison: Although not included experimentally due to implementation constraints, we summarize below how PARD compares conceptually with other recent parallel speculative decoding methods:

Method	Draft Model Design	Target-Model Independence
Parallel Spec	Parallel draft head built base on the target model	✗ Requires separate tuning for each target
BiTA	Prefix tuning with additional learned prefix tokens	✗ Requires separate tuning for each target
PaSS	Look-ahead token embeddings trained with each target model	✗ Requires separate tuning for each target
PARD (Ours)	Fully decoupled small draft model	✓ One draft supports a family of target models

ParallelSpec modifies the draft head and still requires per-model optimization. BiTA adds trainable prefix tokens, again requiring customized tuning for each target. PaSS learns look-ahead embeddings that are tied to individual target models.

In contrast PARD fully decouples the draft model from the target model. A single small draft model can accelerate an entire *family* of target models without retraining, significantly reducing adaptation and deployment cost—one of the key motivations for the method.

We will include an expanded discussion of these differences in the revised version.

Thank you again for your constructive comments. We hope our clarifications adequately address your concerns and help improve your evaluation of the work.

Add:

Official Review of Submission15276 by Reviewer Gfez

Official Review by Reviewer Gfez31 Oct 2025, 22:10 (modified: 12 Nov 2025, 11:02)EveryoneRevisions

Summary:

This paper proposes PARD (Parallelized Accelerated Reasoning Decoding), a novel speculative decoding method featuring target-independence and parallel token prediction. PARD is highly generalizable: its target-independent design allows a single draft model to accelerate an entire family of target models. Through mask tokens, PARD substantially accelerates inference by predicting multiple future tokens within a single forward pass of the draft phase. And they propose a COnditional Drop-token (COD) strategy for the effient training of PARD. Leveraging the integrity of prefix key-value states, COD enables low-cost adaptation of autoregressive draft models into parallel ones, boosting training efficiency by up to 3× while

maintaining accuracy. In the end, the author integrates PARD into the high-performance inference framework vLLM.

Soundness: 3: good
Presentation: 3: good
Contribution: 3: good

Strengths:

1. This paper makes a pivotal conceptual shift by introducing a target-independent speculative decoding framework. Unlike dominant approaches like EAGLE and Medusa that train a draft model tightly coupled to a specific target model, PARD enables a single draft model to accelerate an entire family of target models.
2. The work innovates by integrating parallel token prediction into speculative decoding via masked language modeling. A key novelty is the Conditional Drop-token (COD) mechanism, which leverages the integrity of prefix key-value states to dramatically reduce training overhead. This allows efficient adaptation of autoregressive models into parallel draft models. The use of a shared mask token ID to enable extrapolation beyond the trained prediction length is another clever and inventive design choice that enhances practicality.
3. The integration with the industrial-grade vLLM framework underscores its immediate practical significance, providing a scalable and efficient solution that directly addresses real-world inference bottlenecks.

Weaknesses:

Limited Analysis of the Parallel Drafting Mechanism's Limitations: The paper celebrates the efficiency of parallel drafting but fails to critically analyze its inherent constraints. Generating K tokens in one forward pass necessarily uses less contextual information (relying on masks) than an autoregressive model, which may hurt prediction accuracy for long-range dependencies or complex reasoning steps. This likely explains the performance drop on certain tasks (e.g., PARD vs. EAGLE-3 on SpecBench for LLaMA3.3-70B). The work would be strengthened by a dedicated analysis—for instance, measuring how acceptance rate decays with K on different task types—to delineate the practical boundaries of its parallel approach.

Questions:

N/A

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Commentby Authors20 Nov 2025, 18:46Everyone

Comment:

Dear reviewer,

Thank you very much for your constructive feedback and for highlighting this important question.

W1: How does the parallel drafting mask mechanism affect accuracy and acceptance rate as the prediction length K increases?

A: This is an excellent question. While masked parallel prediction significantly improves decoding efficiency, it inevitably has an impact on acceptance rate as the prediction length increases. In the paper (Table 2), we report acceptance rates for EAGLE/EAGLE-3 and PARD. For greater clarity, we additionally

include the results from a vanilla autoregressive Speculative decoding draft (VSD, without masking) and organize the comparison below, where α denotes the average acceptance rate when the draft length is α :

Method	HumanEval 1- α	HumanEval 4- α	GSM8K 1- α	GSM8K 4- α
EAGLE	0.83	0.72	0.79	0.66
EAGLE-3	0.87	0.85	0.82	0.79
VSD	0.94	0.94	0.89	0.89
PARD	0.93	0.90	0.88	0.85

Interpretation and analysis: VSD achieves the highest acceptance rate among all methods, which is precisely one of the motivations behind PARD: leveraging VSD’s strong acceptance properties while accelerating inference via parallel masked prediction. Because VSD generates tokens autoregressively, each draft position yields identical acceptance behavior, making α effectively constant.

When VSD is converted into a PARD model:

- **For** $\alpha=1$, PARD and VSD have nearly identical acceptance rates. This is because substituting $K=1$ into Eq. (6) and Eq. (7) in the paper makes the prediction objectives equivalent:
- α are modeled the same way.
- **For** $\alpha>1$, the formulation becomes:
- where placeholders must stand in for future context. This makes the prediction task inherently more challenging than AR drafting without placeholders, which explains the gradual decay in acceptance rate.

Empirical outcome: Despite this, the degradation is relatively minor:

- Even at $\alpha=4$, masked parallel decoding reduces acceptance rate by only **~4%** on average.
- In return, it reduces the number of draft forward passes by a factor of α .

Thus, when balancing acceptance rate and draft speed, PARD consistently provides a higher net acceleration while maintaining strong accuracy, making it a practical solution for real-world parallel speculative decoding.

Thank you again for raising this insightful point. We hope this clarification addresses your concerns and helps improve your evaluation of our work.

Add:

Official Comment by Reviewer Gfez

Official Commentby Reviewer Gfez26 Nov 2025, 16:53Everyone

Comment:

Thanks for your detailed follow-up explanations. I'll maintain my original score, which still reflects a positive evaluation of your work.

Add:

Official Review of Submission15276 by Reviewer GWvT

Official Reviewby Reviewer GWvT31 Oct 2025, 17:17 (modified: 29 Nov 2025, 01:16)EveryoneRevisions

Summary:

The paper proposes PARD (PARallel Draft), a novel method for speculative decoding (SD) designed to accelerate Large Language Model (LLM) inference while featuring target-independence and parallel token prediction. PARD addresses the high adaptation cost of target-dependent methods like EAGLE by allowing a single, low-cost draft model to be applied across an entire family of target models. The core mechanism involves fine-tuning a small auto-regressive (AR) model into a parallel draft model, utilizing the Mask-Predict approach with special mask tokens ($\langle \text{MASK} \rangle$) to predict multiple candidate tokens in a single forward pass. This significantly reduces the draft model's latency from $\mathcal{O}(L)$ to $\mathcal{O}(L/d)$. To reduce the training cost for this parallel adaptation, the authors introduce the Conditional Drop-token (COD) mechanism. COD enables sparse sampling of training data while ensuring the integrity of prefix key-value states, boosting training efficiency by up to 10x compared to traditional masked prediction training. Experiments on the industrial vLLM framework show that PARD achieves a state-of-the-art speedup on LLaMA3.1-8B.

Soundness: 2: fair

Presentation: 2: fair

Contribution: 2: fair

Strengths:

1. This paper proposes PARD, which allows a single same-family draft model to accelerate an entire family of target models via multi-token prediction with look-head tokens.
2. This paper integrates a token-dropping scheme to reduce the draft model's training cost.

Weaknesses:

- The paper frames advanced SD methods as "target-dependent" and PARD as "target-independent", yet PARD crucially assumes the availability of a same-family small model—often unrealistic for fine-tuned production LLMs. In those settings, self-speculative decoding from the target model itself (e.g., LayerSkip/ACL'24, Self-SD/ACL'24, SWIFT/ICLR'25) seems strictly more applicable. The paper should explicitly discuss this constraint, compare against self-SD scheme, and clarify when PARD is preferable.
- Much of the method resembles transferring look-ahead token prediction to a small model and then boosting with fine-tuning. Given that small-model training is a one-off offline cost, the contribution of the token dropping mechanism reads like a patch rather than a core innovation.
- The concept of parallel decoding is not novel. For instance, [1] has proposed a similar idea based on a parallel-decoded drafter and a target model as a verifier for speculative decoding. Additionally, Medusa, Multi-Token Prediction (MTP) are existing parallel decoded speculation approaches. Compared with the existing works, this paper mainly differentiates in its draft model architecture design that leverages the $\langle \text{MASK} \rangle$ tokens as a placeholder to predict future tokens within one model forward pass. However, this design is similar to the approach proposed in [2] [3].
- PARD comes from converting the sequential prediction to a parallel prediction, reducing the time cost of the draft stage from $\mathcal{O}(L)$ to $\mathcal{O}(L/d)$.
- However, the unresolved issue is whether the T_{D^*} of the new draft model will change, and whether it is possible that T_{D^*} is less than T_{D^*} when the draft model is too large.
- The paper claims that its draft model is target-independent and can accelerate an entire family of target models. However, for target model families with a large range of parameter quantities, using the same draft model (lacking the adjustment of the draft model to fit the size of the target model) may lead to the problems mentioned in the last weakness. That is, universal draft model may perform slower in parallel decoding than the ideal draft model even if in sequential decoding. The evidence in the paper is insufficient to reveal or fix this issue.
- The Shared Mask Token ID Strategy in ablation study needs concrete details: how mask tokens are inserted across variants, which IDs are shared.
- Presentation issues:
 - Fig. 4 (especially subfigures (b) and (c) on training-time attention masks) is hard to parse. In addition, for Fig. 4(a), clarify the meaning of green/red. If green denotes permitted

- attention, the causal mask for query at position t should be upper-triangular over keys/values (e.g., "Q(story)" must attend to its history; the last column should be green).
- Equations (7) and (8) have an indexing bug: substituting $k=0$ (or $k=1$) yields $m_{\{-1\}}$; define base cases and boundary conditions to prevent invalid indices.
 - The core idea of COD ("earlier-subtask tokens are more critical; later ones can be dropped") lacks justification—why are later tokens less important? Is this theoretically motivated or empirically observed (e.g., uncertainty, acceptance rates)?

Questions:

Please refer to Weaknesses.

Flag For Ethics Review: No ethics review needed.

Rating: 2: reject, not good enough

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Part 1

Official Comment by Authors 20 Nov 2025, 18:48 (modified: 20 Nov 2025, 18:54) Everyone Revisions

Comment:

Dear reviewer,

Thank you very much for your detailed feedback and for raising many valuable and insightful concerns. We address each of your questions below.

W1: Clarify the advantages of being “target-independent” and compare against self-speculative methods (e.g., LayerSkip/ACL’24, Self-SD/ACL’24, SWIFT/ICLR’25).

A: This is an excellent question. We first clarify our definition of *target-independence*:

A draft model is *target-independent* if, once trained, it can be directly applied to accelerate multiple models within a family without any additional fine-tuning or retraining.

For example, a single PARD-LLaMA3-1B model can accelerate the entire LLaMA3 series. In contrast, methods such as EAGLE and Medusa rely on target-model hidden states as input and therefore require training a separate draft model per target model, making them inherently *target-dependent*.

Target-independence is an extremely valuable property because it dramatically reduces the cost of practical deployment. Take the LLaMA3 family as an example: the series includes L3-8B, L3-70B, L3.1-405B, and up to L3.3-70B, totaling eight models. If a user wishes to accelerate any one of these models using speculative decoding, target-dependent methods like EAGLE may not provide an available pretrained draft model, and training one from scratch is often prohibitively expensive for most users. Moreover, if the target model is further fine-tuned (e.g., via SFT or RL), the hidden states used as inputs to the EAGLE head will change, requiring the draft model to be retrained again. In contrast, PARD’s target-independent design eliminates this issue entirely: a single draft model can be used to accelerate the entire model family, even if the target models are fine-tuned afterward, significantly reducing operational and maintenance costs.

Relationship to self-speculative decoding: Self-speculative decoding is an important research direction, but whether a method is “self-speculative” and whether it is “target-independent” are orthogonal. We clarify with two representative examples:

- **LayerSkip (ACL’24):** Requires training models with layer dropout and later using subsets of layers as drafts. Each target model must be individually retrained to enable self-speculation, so it is **not target-independent**.

- **SWIFT (ICLR'25):** Requires Bayesian optimization to determine which layers to retain for each target model. Again, this process is repeated per target model, making it **not target-independent**. SWIFT is effective but the acceleration gain is modest (typically 1.3×–1.6×).

Self-speculative methods are valuable, and we will expand the Related Works section to discuss them. PARD targets a different use case: enabling a single universal draft model for an entire model family with no per-target adaptation cost, which is crucial for industrial deployments where one service must support multiple model variants simultaneously.

W2: The Importance of the Condition-Drop Token (COD) Mechanism.

A: Our method is designed to achieve high acceleration while minimizing the adaptation cost of the draft model. To this end, PARD is made **target-independent**, meaning that models within the same family require only a single adaptation. In addition, we introduce the **Condition-Drop Token (COD)** mechanism, which further reduces this one-time adaptation cost by **3×**. It is worth noting that even without COD, the draft model can still reach the same inference speedup, but COD enables us to achieve this performance at significantly lower training cost.

In implementation, COD also has its own ingenuity. As detailed in Appendix B, naively dropping condition tokens at random would result in incomplete preceding KV caches, causing a mismatch between training and inference. Our approach avoids this issue by ensuring that the KV cache before the condition remains complete, and only then performs token dropping. This allows PARD to be trained with acceleration benefits without introducing performance inconsistency.

Therefore, COD is an essential mechanism that makes PARD practical and cost-efficient for real-world deployment.

Add:

Part 2

Official Comment by Authors 20 Nov 2025, 18:49 (modified: 20 Nov 2025, 18:55) Everyone Revisions

Comment:

W3: Explain the relationship between PARD and parallel decoding, and how parallel decoding methods such as Medusa and Multi-Token Prediction relate to mask prediction.

A: Since the reviewer did not specify which prior works they were referring to, I will answer based on my understanding.

I address two aspects separately: **(1) the relationship between PARD and parallel decoding**, and **(2) how parallel decoding methods such as Medusa and multi-token prediction relate to mask prediction**.

Relationship between PARD and parallel decoding. PARD adopts a mask-prediction–based parallel decoding strategy, which falls under a category of multi-token speculative decoding widely used in recent works such as BiTA, SPACE, ParallelSpec, PaSS, and now PARD. Parallel decoding itself is not our main contribution. Instead, our primary innovations (Lines 103–131) focus on enabling a *high-speed draft model that remains broadly applicable while keeping training cost extremely low*. In particular:

- **Condition Drop Token (COD).** Mask-prediction-based parallel decoding typically increases the number of input tokens by a factor of k , leading to substantially higher training cost. Our COD mechanism selectively drops part of these conditioning tokens without degrading inference performance, resulting in a **3× reduction in training overhead**.
- **Target Independence.** Most speculative decoding approaches—including EAGLE, Medusa, BiTA, and SPACE—require training a separate draft model for each target model. This becomes increasingly costly and operationally inflexible when supporting a full model lineup (e.g., a LLaMA3 deployment might require eight separate draft models for LLaMA3-8B, 70B, LLaMA3.1-8B, 70B,

405B, LLaMA3.3-70B, etc.). In contrast, PARD is *target-independent*: a single draft model can be shared across the entire model family, significantly simplifying deployment and maintenance.

Difference between PARD and Medusa-style parallel decoding. Although both PARD and Medusa are parallel decoding, their computational properties are fundamentally different. In particular, PARD’s memory bandwidth consumption during decoding does not grow with the number of candidate tokens , whereas Medusa’s does, because each Medusa head maintains its own independent parameters. For example, using LLaMA3-70B as a reference:

- **PARD:** Regardless of the value of (k), the memory bandwidth cost of the draft model is constant and determined solely by the 1B-parameter draft model size.
- **Medusa:** Each prediction head is an independent set of parameters. With 5 parallel heads, the parameter size is approximately
- Since LLM decoding is **memory-bound**, this means Medusa’s memory bandwidth cost scales *linearly* with , while PARD’s cost stays fixed.

In summary, PARD leverages the advantages of Mask-prediction-based parallel decoding while introducing new mechanisms that make the draft model cheaper to train, easier to deploy, and more efficient to run in large real-world systems.

W4: In the modeling, does get affected by introducing a “new draft model”?

A: The execution of is identical in nature to , it performs masked multi-token prediction in a single forward pass. The assumption that multiple tokens can be produced in parallel without increasing latency is standard in speculative decoding literature, e.g.:

“Fast Inference from Transformers via Speculative Decoding”, Section 3.3.
Therefore, introducing PARD does not fundamentally change the latency model for .

W5: When using the same model to accelerate a series of target models, what impact does this have on target models of different sizes?

A: In Table 1 of the paper, we report the speedup achieved by using the same draft model to accelerate target models of different sizes. Specifically, L3.1-8B and L3.3-70B achieve speedups of 3.00× and 3.49×, respectively. In the table below, we re-evaluated these two models with respect to their accepted lengths, which are very close. This further demonstrates that a single draft model can effectively accelerate different target models.

Target Model	Benchmark	Speed-up	Accept Length
LLaMA3.1-8B	HumanEval	3.72×	5.72
LLaMA3.3-70B	HumanEval	4.15×	5.83

The similar acceptance lengths indicate that the same draft model generalizes well to different model sizes, empirically supporting the feasibility of universal drafting for model families.

Add:

Part 3

Official Commentby Authors20 Nov 2025, 18:50 (modified: 20 Nov 2025, 18:55)EveryoneRevisions
Comment:

W6: The Shared Mask Token ID strategy in the ablation study needs concrete details: how mask tokens are inserted across variants and which IDs are shared.

A: We appreciate the request for clarity. In practice, we simply reuse existing special token IDs in the draft model vocabulary:

Draft Model	Shared Token ID	Unshared token IDs
PARD-Qwen2.5	[151665]	[151665, 151666, ...]
PARD-LLaMA3	[128020]	[128020, 128021, ...]

We will add these configurations to the appendix for completeness.

W7: Presentation issues in Fig. 4 and indexing in Eqs. (7)(8).

A: Thank you very much for your careful observation. In Fig. 4, the KV and Q were mistakenly swapped, and we will correct this in the final version.

Regarding Equations (7) and (8), we originally wrote them following the code implementation, which may appear unclear when reading. We will revise them for clarity in the final version. For example, in Equation (7) from the paper, when $\alpha \rightarrow 0$, the first term on the right-hand side degenerates to a form without any α as input.

W8: Why are earlier-subtask tokens more critical, while later ones can be dropped?

A: This is an excellent question, and we would like to explain the motivation behind this design. In speculative decoding, the total acceptance length can be defined as:

where L is the total acceptance length, n is the number of candidate tokens generated by the draft model, p_i is the probability that the (i) -th candidate token is accepted, and α is the acceptance rate. Here, we assume that the acceptance rate is identical across different positions, which holds for vanilla SD.

We can interpret α as the contribution of the (i) -th candidate to the overall acceptance length. From the formula above, the probability of acceptance naturally decreases exponentially for later positions—the further the token, the lower its contribution.

In mask prediction, different training tasks correspond to ensuring the acceptance of the (i) -th candidate, i.e., p_i . It is therefore natural to allocate more training emphasis on earlier, more critical tokens, while reducing emphasis on later, less important tokens. Based on this analysis, the COD mechanism is designed according to the principle: **earlier-subtask tokens are more critical, while later ones can be dropped.**

Thank you again for your thoughtful and detailed reviews. We believe the clarifications above address your concerns, and we hope our responses improve your evaluation of the work.

Add:

Official Comment by Reviewer GWvT

Official Commentby Reviewer GWvT26 Nov 2025, 14:50Everyone

Comment:

Thanks to the authors for the details responses. The rebuttal has partially addressed my concerns. I would raise the score accordingly.

Add:

Official Review of Submission15276 by Reviewer MJ3S

Official Reviewby Reviewer MJ3S31 Oct 2025, 15:20 (modified: 12 Nov 2025, 11:02)EveryoneRevisions

Summary:

This paper proposes PARD, which accelerates autoregressive generation by training a target-independent draft model that generate drafts in parallel. Furthermore, the authors propose the CoD method to accelerate the training of the parallel inference model. Owing to parallel generation and target independence, PARD can quickly transfer across different model within the same series and generate drafts at a low cost, achieving greater acceleration compared to the autoregressive EAGLE.

Soundness: 2: fair

Presentation: 3: good

Contribution: 2: fair

Strengths:

1. The paper is well-structured, with clear and intuitive diagrams that effectively support the explanation.
2. It explores the concept of parallel draft generation, providing insightful and innovative perspectives on the topic.

Weaknesses:

1. The paper lacks a comparison with other similar draft acceleration methods, such as hierarchical acceleration [1,2] and draft models in diffusion models [3].
2. The experimental setup is not sufficiently detailed, with insufficient explanation of decoding strategies (e.g., chain or tree).

[1] Chen Z, Yang X, Lin J, et al. Cascade speculative drafting for even faster llm inference[J]. Advances in Neural Information Processing Systems, 2024, 37: 86226-86242.

[2] Zhao W, Huang Y, Han X, et al. Ouroboros: Speculative decoding with large model enhanced drafting[J]. arXiv preprint arXiv:2402.13720, 2024.

[3] Christopher J K, Bartoldson B R, Ben-Nun T, et al. Speculative diffusion decoding: Accelerating language generation through diffusion[J]. arXiv preprint arXiv:2408.05636, 2024.

Questions:

1. The experimental results in Table 1 lack reporting on the acceptance length, and the acceleration ratio of EAGLE is lower than that of other benchmark methods. It would be useful to clarify whether Tree validation is employed in this study.
2. In addition to the absence of a comparison with related methods, the related work section overlooks relevant literature on parallel generation techniques, which could offer valuable context for this study.

[1] Kou S, Hu L, He Z, et al. Clms: Consistency large language models[C]//Forty-first International Conference on Machine Learning. 2024.

[2] Wang Y, Luo X, Wei F, et al. Make some noise: Unlocking language model parallel inference capability through noisy training[J]. arXiv preprint arXiv:2406.17404, 2024.

[3] Gao X, Xie W, Xiang Y, et al. Falcon: Faster and parallel inference of large language models through enhanced semi-autoregressive drafting and custom-designed decoding tree[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2025, 39(22): 23933-23941.

3. As indicated in Table 2, the high acceptance rate of PARD primarily stems from its consistency with existing homologous models, rather than additional training. Consequently, it is essential to conduct comparisons between PARD and other draft model acceleration approaches to better assess its relative performance.

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Part 1

Official Comment by Authors 20 Nov 2025, 18:52 Everyone

Comment:

Dear reviewer,

Thank you very much for the positive feedback regarding the paper's structure and clarity, as well as for raising thoughtful and valuable questions. We address them in detail below.

Q1, W2: Experimental setup, chain vs. tree attention, and EAGLE testing details.

A: All experiments in the paper were run under the industrial inference stack **vLLM** and use **chain attention**. We adopted this environment to measure performance under realistic, production-like serving conditions. vLLM currently provides a chain-attention implementation only; consequently the reported results (Table 1, Table 2, Figure 1, etc.) reflect chain-attention behavior. Tree attention is an orthogonal decoding strategy and can be combined with PARD, but it was not used in our reported experiments.

EAGLE / EAGLE-3 testing details: All EAGLE / EAGLE-3 baselines were evaluated under the same vLLM + chain-attention setup and identical decoding settings used for PARD. The EAGLE/EAGLE-3 numbers reported in our paper are consistent with published EAGLE-3 evaluations (Eagle-3 [1], Section 4.4), when evaluated under comparable system-level settings.

[1] Li, Yuhui, et al. "Eagle-3: Scaling up inference acceleration of large language models via training-time test." arXiv preprint arXiv:2503.01840 (2025).

Q2, W1: Comparison with hierarchical acceleration, diffusion-based drafters, and other parallel works.

A: thank you for pointing out additional lines of related work.

Hierarchical acceleration approaches such as Cascade Speculative Drafting [2] and Ouroboros [3] introduce additional nested draft models (either smaller AR models or phrase-level decoders) to further accelerate drafting. Diffusion-based approaches [4] instead adopt a diffusion-style generator as the draft model to improve parallelism. Meanwhile, recent works such as CLLMs [5] and Noisy Parallel Decoding [6] employ Jacobi-style iterative refinement with noise augmentation, and Falcon [7] incorporates Coupled Sequential Glancing Distillation with a custom decoding tree to strengthen token dependency modeling.

PARD adopts a mask-prediction-based strategy, which falls under a category of multi-token speculative decoding widely used in recent works such as BiTA, SPACE, ParallelSpec, PaSS, and now PARD. Our primary innovations (Lines 103–131) focus on enabling a *high-speed draft model that remains broadly applicable while keeping training cost extremely low*. In particular:

- **Condition Drop Token (COD):** Mask-prediction-based method typically increases the number of input tokens by a factor of k , leading to substantially higher training cost. Our COD mechanism selectively drops part of these conditioning tokens without degrading inference performance, resulting in a 3× reduction in training overhead.
- **Target Independence:** Most speculative decoding approaches including EAGLE, Medusa, BiTA, and SPACE—require training a separate draft model for each target model. This becomes increasingly costly and operationally inflexible when supporting a full model lineup (e.g., a LLaMA3 deployment might require eight separate draft models for LLaMA3-8B, 70B, LLaMA3.1-8B, 70B,

405B, LLaMA3.3-70B, etc.). In contrast, PARD is target-independent: a single draft model can be shared across the entire model family, significantly simplifying deployment and maintenance.

We selected EAGLE-3 as our main baseline because it is a leading, widely adopted industrially-relevant speculative method and provides a fair and practical comparison under vLLM. Our experiments show that PARD delivers higher net acceleration than EAGLE-3 in the same real-system setting, demonstrating the method's effectiveness.

We will expand the Related Work section to include the cited works.

[2] Chen Z, Yang X, Lin J, et al. Cascade speculative drafting for even faster llm inference[J]. Advances in Neural Information Processing Systems, 2024, 37: 86226-86242.

[3] Zhao W, Huang Y, Han X, et al. Ouroboros: Speculative decoding with large model enhanced drafting[J]. arXiv preprint arXiv:2402.13720, 2024.

[4] Christopher J K, Bartoldson B R, Ben-Nun T, et al. Speculative diffusion decoding: Accelerating language generation through diffusion[J]. arXiv preprint arXiv:2408.05636, 2024.

[5] Kou S, Hu L, He Z, et al. Cllms: Consistency large language models[C]//Forty-first International Conference on Machine Learning. 2024.

[6] Wang Y, Luo X, Wei F, et al. Make some noise: Unlocking language model parallel inference capability through noisy training[J]. arXiv preprint arXiv:2406.17404, 2024.

[7] Gao X, Xie W, Xiang Y, et al. Falcon: Faster and parallel inference of large language models through enhanced semi-autoregressive drafting and custom-designed decoding tree[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2025, 39(22): 23933-23941.

Add:

Part 2

Official Comment by Authors 20 Nov 2025, 18:52 Everyone

Comment:

Q3: Homologous drafts models, and comparison to other speculative decoding approaches

A: As shown in Figure 1 of the paper, VSD demonstrates substantially higher acceptance rates compared to EAGLE, which directly motivates our work:

If we can reduce the draft cost of an AR-based VSD model, the resulting system can achieve significantly higher overall acceleration.

PARD contributes exactly in this direction by low-cost conversion of an autoregressive draft model into a parallel draft model, preserving VSD's high acceptance rate while greatly reducing draft latency.

For the comparison with other speculative decoding approaches. In the paper We compare PARD against vanilla speculative decoding and mainstream EAGLE / EAGLE-3 in the paper. Concretely, in our high-performance vLLM experiments (Table 1):

- On LLaMA3.1-8B, PARD yields higher wall-clock acceleration than EAGLE-3 (PARD is $\sim 1.15\times$ faster than EAGLE-3 on this workload). Eagle3 is a state-of-the-art method in the industry, and comparing against it demonstrates the effectiveness of our approach.
- PARD simultaneously provides **low training/adaptation cost**, **low draft latency**, **high draft acceptance**, and **target independence**, as summarized in Figure 1(c).

Thank you very much for your careful review and valuable feedback. We hope our clarifications help, and we would greatly appreciate your consideration in the final evaluation.

Add:

Replying to Part 1

Official Comment by Reviewer MJ3S

Official Comment by Reviewer MJ3S 27 Nov 2025, 08:21 Everyone

Comment:

Thank you for your response. My primary concerns remain centered on the following two points:

1. Comparison with Similar Methods

As noted previously, the current experimental setup employs an overly limited baseline, utilizing only the Vanilla Speculative Decoding (VSD) method. Given that the proposed PARD as a method for accelerating draft model decoding in a parallel manner, a comparison with other similar approaches is essential. Based on the current results, I cannot determine whether the observed performance improvement originates from the proposed method itself or from a better-aligned draft model within the same model family.

2. Fairness of EAGLE Results under Chain Verification

EAGLE's superior performance over Medusa is observed, particularly when utilizing Tree Verification. This advantage is attributable to EAGLE's autoregressive nature, which allows a significantly larger search space via the tree structure. In contrast, parallel decoding methods are limited to a Cartesian product approach. This consideration is equally relevant to the method proposed in this manuscript (PARL). I highly doubt that PARL will maintain its current advantage under a Tree Verification scheme.

As noted above, I remain concerned about whether PARL demonstrates a fundamental performance improvement compared to similar approaches (e.g., ParallelSpec). Consequently, I intend to maintain my current score.

Add:

Replying to Official Comment by Reviewer MJ3S

Official Comment by Authors

Official Comment by Authors 27 Nov 2025, 11:14 Everyone

Comment:

Dear reviewer,

Thank you for the thoughtful and constructive feedback. I address your concerns as follows.

Q1: Choice of Baselines and Comparison with Similar Methods

A: Our manuscript does not compare only against Vanilla Speculative Decoding (VSD), also compares PARL against **Eagle/Eagle3**, as shown in Figure 2 and Table 1. Since **Eagle3 is widely regarded as an industry-standard benchmark, demonstrating that PARL achieves 1.15× speedup over Eagle3 provides strong evidence of the effectiveness of our approach.**

A core strength of PARL lies in its target-independent design. **Target-independence is an extremely valuable property** because it drastically reduces practical deployment and maintenance costs. Consider the LLaMA3 family: L3-8B, L3-70B, L3.1-405B, L3.3-70B, and several intermediate versions—eight models in total. Under a target-dependent method like EAGLE:

- A pretrained draft model may not exist for each target model.
- Training a new draft model from scratch can be prohibitively expensive for most users.
- If the target model is later fine-tuned via SFT or RL, the hidden states used by the EAGLE head change, requiring draft retraining.

By contrast, **PARL's fully decoupled draft model avoids these issues altogether.** A single draft model can accelerate the entire model family—even after further fine-tuning—dramatically reducing operational costs.

Comparison with Other Mask-Prediction-Based Parallel Speculative Decoding Methods: To better approximate real-world inference scenarios, all experiments in our paper are conducted under the high-performance inference framework vLLM. Although ParallelSpec, BiTA, and PaSS are not integrated into our evaluation, we summarize below the conceptual comparison between PARD and these parallel speculative decoding methods:

Method	Draft Model Design	Target-Model Independence
Parallel Spec	Parallel draft head built directly on the target model	✗ Requires separate tuning for each target
BiTA	Prefix tuning using learned prefix tokens	✗ Requires separate tuning for each target
PaSS	Look-ahead token embeddings trained per target model	✗ Requires separate tuning for each target
PARD (Ours)	Fully decoupled small draft model	✓ One draft supports an entire family of target models

ParallelSpec modifies the draft head of each target model; BiTA adds trainable prefix tokens; PaSS learns target-specific look-ahead embeddings. All of them remain **target-dependent**.

PARD is fundamentally different: **It fully decouples the draft model from the target model**, enabling broad applicability and dramatically lowering adaptation cost—one of the key motivations behind our work.

Regarding whether the performance gains come from same-family alignment or from PARD itself: these two factors work synergistically. As shown in Figure 1, homologous models exhibit high acceptance rates. Combined with PARD’s architecture, this allows the draft model to achieve both **high acceptance** and **low latency**, which is a central contribution of our method.

Q2: Fairness of EAGLE Results under Chain/Tree Verification

A: We agree that tree attention can enhance decoding efficiency, and Eagle2’s dynamic tree selection further improves performance. **Importantly, PARD is fully compatible with Eagle2-style tree attention and can benefit from the same mechanisms.** To better reflect real-world usage scenarios, all evaluations in our paper are conducted using the vLLM framework. Since vLLM does not yet support tree attention, all experiments are performed under chain attention.

Below is an example illustrating how tree attention can be applied within PARD:

Consider input "" with . The draft model input token is: .

Suppose the token probability distributions (formatted as [token, probability]) after one forward pass are:

- First token:
- Second token:

As in Eagle2, we compute the probability of each token path, e.g.:

If a three-node draft tree is desired, the tree indices are .

For a six-node tree, the indices become .

After constructing the draft tree, verification proceeds exactly as in the EAGLE2 workflow. Thus, PARD can fully leverage dynamic tree-based verification rather than being limited to Cartesian-product expansion, and its performance is not tied to chain-only verification.

Thank you again for your valuable comments. We hope our clarifications address your concerns. We believe the properties above demonstrate that PARD offers both conceptual and practical advantages over existing similar methods.

Add:

Replying to Official Comment by Authors

Official Comment by Reviewer MJ3S

Official Comment by Reviewer MJ3S27 Nov 2025, 11:33Everyone

Comment:

I appreciate your timely feedback. I would like to further elaborate on my perspective regarding these two issues:

1. Comparison with Additional Baselines

While I acknowledge the importance of decoupling the target and draft models, several comparable methods operating under identical settings are currently overlooked. For instance, Ouroboros utilizes Lookahead decoding to accelerate the draft model in a training-free manner. It is essential to compare the proposed method against such approaches to demonstrate that performance gains stem from the efficacy of parallel generation, rather than merely from the superior alignment of Qwen. I think Eagle3 would achieve superior performance while maintaining a training budget comparable to that of the independent draft models.

2. Feasibility of Tree Verification

To clarify, I do not deny the compatibility between parallel decoding and tree verification. However, current parallel decoding methods—as you indicate—employ a Cartesian product approach for tree construction. The key point I wish to emphasize is that this yields a significantly smaller search space compared to the autoregressive Eagle method.

In your example, for the autoregressive draft model, the distributions for the second token are independent. Specifically, $\mathbf{p}(y_2 | \mathbf{y}_1)$ and $\mathbf{p}(y_2 | \mathbf{y}_1, \mathbf{y}_2)$ are sampled respectively from the outcomes of $\mathbf{p}(y_1)$ and $\mathbf{p}(y_1, y_2)$, rather than remaining within the same distribution as is currently the case.

Add:

Official Comment by Authors

Official Comment by Authors03 Dec 2025, 13:48 (modified: 03 Dec 2025, 14:53)EveryoneRevisions

Comment:

Thank you very much for your detailed and insightful feedback. We appreciate the opportunity to clarify our methodology. To address your comments more thoroughly, we address your concerns point by point and respond to each point individually. In addition, we have conducted supplementary experiments to further support our claims.

Q: Choice of Baselines & Tree Attention

A: To approximate real-world inference scenarios, all experiments in our paper are conducted under the high-performance **vLLM** framework. Some additional baselines (e.g., diffusion-based methods, Jacobi-style methods) and tree-attention-based verification are not yet supported in vLLM; this is the main reason they are not included in our experimental evaluation.

We agree that parallel-draft and autoregressive-draft methods expand their search trees differently. We will include in the appendix a brief explanation of how PARD can incorporate tree attention.

We selected **EAGLE/EAGLE-3** as baselines because vLLM supports them natively, and because they are widely considered **strong industry-standard benchmarks**. Demonstrating that **PARD achieves a 1.15×**

speedup over EAGLE-3 under identical conditions provides strong evidence of the effectiveness of our approach.

Q: Training Budget

A: As analyzed in **Appendix A**, using LLaMA3.3-70B as the target model, **PARD achieves 7× higher training efficiency than EAGLE and 10× higher than EAGLE-3.**

Beyond training efficiency, PARD offers **target independence**, meaning that a single draft model can accelerate an entire series of target models (e.g., LLaMA3-8B, 70B, 405B). EAGLE-style methods, by contrast, require training one draft model per target, which substantially increases adaptation and maintenance cost.

This efficiency difference explains why training a PARD draft model is substantially less costly in practice.

Q: Is the Speedup Mainly Due to Model Homogeneity?

****A:** ******This is an excellent question. It is true that using a same-family model—for example, LLaMA3.2-1B as the draft for LLaMA3.1-8B yields a high acceptance rate (as shown in Figure 1).

However, PARD’s acceleration does not rely on homogeneity. To demonstrate this, we conducted a controlled experiment using LLaMA3.1-8B as the target model on HumanEval, keeping all training and inference hyperparameters identical. The experiment compares:

- 1. **Baseline:** No speculative decoding
- 2. **PARD (same-family draft):** LLaMA3.2-1B → training to PARD_L3.2_1B
- 3. **PARD (random init):** Randomly initialized draft model → training to PARD_L3.2_1B_random_init
- 4. **PARD (cross-family draft):** Qwen2.5-0.5B adapted via vocabulary replacement → training to PARD_Q2.5_0.5B_vocab_modify

Because LLaMA and Qwen use different vocabularies, we first adapted Qwen2.5-0.5B by replacing its vocabulary with LLaMA’s. We found that the two vocabularies share **85% token overlap (109,566 / 128,256)**. The replacement rule is:

- If a token exists in both vocabularies, copy its original Qwen embedding.
- If it exists only in LLaMA’s vocabulary, initialize the embedding with a normal distribution.

Using this adapted model (“Q2.5_0.5B_modify”), we trained it under the PARD paradigm.

Index	Target	Draft	Draft Init Method	TPS	Speed up
1	LLaMA3.1-8B	—	—	72.70	1.00
2	LLaMA3.1-8B	PARD_L3.2_1B	Init from LLaMA3-1B	254.00	3.49
3	LLaMA3.1-8B	PARD_L3.2_1B_random_init	Random Init	94.26	1.30
4	LLaMA3.1-8B	PARD_Q2.5_0.5B_vocab_modify	Init from Qwen w/ vocab replacement	251.47	3.46

The cross-family draft model (Index 4) achieves a **3.46× speedup**, nearly identical to the same-family draft (3.49×), confirming that PARD’s benefits generalize beyond model-family homogeneity. From these results, we conclude that:

- **Homogeneity is not the primary factor driving PARD’s speedup**: even a small **Qwen** draft model can deliver comparable acceleration when applied to the **LLaMA3** family after PARD training.
- The final speedup is strongly correlated with the **intrinsic quality/capability of the draft model initialization**: small drafts from both the **Qwen** and **LLaMA3** families yield strong acceleration on **LLaMA3.1-8B**, whereas a randomly initialized draft performs substantially worse.
- The key contributors are **(1) baseline draft model quality + (2) the PARD training paradigm**, rather than shared model family.

We sincerely appreciate your constructive comments. Your feedback has helped us strengthen the clarity and rigor of the work, and we believe the additional analyses and experimental results address the key concerns raised.

Add:

[About OpenReview](#)
[Contact](#)
[FAQ](#)
[Hosting a Venue](#)
[Sponsors](#)
[Terms of Use / Privacy Policy](#)
[All Venues](#)
[Donate](#)
[News](#)

OpenReview is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026 OpenReview

OpenReview.net

Login



Go to [ICLR 2026 Conference homepage](#)

Speculative Speculative Decoding



Tanishq Kumar, Tri Dao, Avner May

Published: 26 Jan 2026, Last Modified: 05 May 2026ICLR 2026 PosterEveryoneRevisionsBibTeXCC BY 4.0

Keywords: inference, large language models, speculative decoding

TL;DR: We introduce an asynchronous speculative decoding algorithm wherein the draft model continuously speculates on top of anticipated verification outcomes, thus hiding drafting latency entirely.

Abstract:

Autoregressive decoding is bottlenecked by its sequential nature. Speculative decoding has become a standard way to accelerate inference by using a fast draft model to predict upcoming tokens from a slower target model, and then verifying them in parallel with a single target model forward pass. However, speculative decoding itself relies on a sequential dependence between speculation and verification. We introduce speculative speculative decoding (SSD) to parallelize these operations. While a verification is ongoing, the draft model predicts likely verification outcomes and prepares speculations pre-emptively for them. If the actual verification outcome is then in the predicted set, a speculation can be returned immediately, eliminating drafting overhead entirely. We identify three key challenges presented by speculative speculative decoding, and suggest principled methods to solve each. The result is Saguaro, an optimized SSD algorithm. Our implementation is on average 30% faster than optimized speculative decoding baselines and up to 5x faster than autoregressive decoding with open source inference engines.

Primary Area: foundation or frontier models, including LLMs

Submission Number: 9487

Filter by reply type...

Filter by author...

Sort: Newest FirstSort: Oldest First

16 / 16 replies shown

Add:

Paper Decision

Decisionby Program Chairs26 Jan 2026, 14:10 (modified: 06 Feb 2026, 11:07)EveryoneRevisions

Decision: Accept (Poster)

Add:

Meta Review of Submission9487 by Area Chair wC7Y

Meta Reviewby Area Chair wC7Y20 Dec 2025, 22:04 (modified: 09 Feb 2026, 12:09)EveryoneRevisions

Summary:

Paper summary. This paper proposes Speculative Speculative Decoding (SSD). The goal is to remove the remaining sequential dependency in speculative decoding: in standard SD, drafting and verification alternate, so drafting often sits idle while the target model verifies. SSD overlaps these two phases by having the draft model predict likely verification outcomes and pre-compute the next speculations asynchronously. The paper presents a concrete system (Saguaro), proofs for correctness, and end-to-end throughput gains on multiple model families and settings.

What happened in the discussion. Reviewers agreed the idea is interesting and the evaluation is broad, but raised three concrete issues: (1) the paper must compare to strong recent baselines (EAGLE-2/3, token-tree methods, other parallel SD systems), (2) the method increases draft-side compute and may stress memory/KV-cache, so overhead should be quantified with wall-time metrics, and (3) SSD may require an extra GPU/device for the speculator, which affects deployment cost. In the forum, the authors added comparisons (including an explicit EAGLE-3 comparison in an updated benchmark), added wall-clock metrics and a memory/overhead calculation, improved the paper organization/figure, and clarified how SSD can be combined with approximate drafting (they also outline an SSD+EAGLE-3 design and estimate expected gains).

My assessment as AC. The main contribution is a clean systems idea (asynchronous pre-speculation) backed by a detailed implementation and a solid correctness argument. The “extra device” requirement is a real trade-off, but the paper is transparent about it and provides enough data to judge the latency/throughput benefits. After the discussion, the biggest remaining weakness is that baseline coverage will never be fully complete in this fast-moving area; however, the added comparisons and

wall-time reporting make the story credible. I also think the revised structure/intuition addresses the “hard to follow” complaint.

Decision. Accept (poster). The method is technically sound, the empirical evidence is strong, and the authors responded directly to the key reviewer concerns with concrete additional results and clarifications.

Reviewer Concerns:

- **Reviewer QxMH (rating 6, confidence 4):** Asked for clearer quantification of extra compute/memory overhead and for stronger baseline comparisons; authors added an explicit overhead calculation and more baseline comparisons and discussion. **Status:** largely resolved.
- **Reviewer Jb3k (rating 6, confidence 4):** Raised paper flow/figure clarity and the “extra GPU/device” deployment cost ($2^n + 1$) question; authors reorganized the paper and clarified deployment and communication considerations. **Status:** partially resolved (cost trade-off remains, but is now clearly explained).
- **Reviewer h3mq (rating 6, confidence 4):** Asked for direct comparisons to EAGLE2/3 and token-tree SD, plus wall time and accepted-token metrics; authors added an EAGLE-3 comparison, added wall-clock measurements, and described SSD+EAGLE-3 combination plans. **Status:** mostly resolved.
- **Reviewer kXox (rating 2, confidence 3):** Found the algorithm hard to follow, unclear why it is lossless, and requested more baseline comparisons; authors added intuition for losslessness and improved organization/figures and comparisons. **Status:** partially resolved; readability is much improved but this reviewer may still dislike the systems trade-offs.

Reviewer Scores:

- **QxMH (6,4):** Likely unchanged (strong accept already).
- **Jb3k (6,4):** Likely unchanged.
- **h3mq (6,4):** Likely unchanged or slightly higher given added EAGLE-3 comparison and wall-clock table.
- **kXox (2,3):** I expect the score could move toward borderline (~4) after the added explanations/comparisons, but the reviewer did not explicitly confirm a score change.

Add:

Public Comment

Public Comment by Yuhao Shen 18 Nov 2025, 22:09 (modified: 18 Nov 2025, 22:17) Everyone Revisions

Comment:

Dear Authors,

I hope this email finds you well. I have carefully read your paper. I noticed that the motivation for SSD discussed in your introduction (specifically the three challenges) and the approach to multi-path draft generation bear a strong resemblance to an arXiv article [1]. Furthermore, I observed that both works conduct a theoretical analysis of rollback mechanisms for Parallel SD. However, I noticed that this paper [1] is not currently cited in your submission, nor is there a discussion or comparison with it. I believe SSD is an excellent piece of work on Parallel SD, supported by solid theoretical analysis. And if you explicitly cite this work and provide a detailed discussion or comparison regarding the differences, it would significantly clarify your contributions and strengthen the novelty of your paper.

Reference:

[1] "Speculative Decoding via Hybrid Drafting and Rollback-Aware Branch Parallelism." arXiv preprint arXiv:2506.01979 (2025).

Best regards,

Add:

Official Response from Authors

Official Comment by Authors 27 Nov 2025, 10:05 (modified: 27 Nov 2025, 10:06) Everyone Revisions

Comment:

Thank you for bringing this paper to our attention, and our apologies for not being aware of it and citing it in our submission—**we have now added this citation and discussion to our substantially expanded related work section** (see the newly attached manuscript, all changes highlighted in red). Indeed, SpecBranch has similar motivation to SSD, and some similar components: we can think of SpecBranch as (approximately) a special case of the SSD framework where only a single branching point is allowed, where the fallback speculator is equal to the regular speculator, and where the hyperparameters (branching point, number of branches, speculation length) are dynamically chosen.

SSD and Saguaro build upon the idea in SpecBranch of pre-speculating alternate token paths in the case of a rejection (in parallel to verification), though they take a somewhat different approach to doing so. The focus of most of our paper is in identifying and studying several design considerations for a parallel speculation system beyond the considerations in SpecBranch. These include:

- Allowing for an arbitrary number of branch points (thereby greatly increasing the probability of one of the branches being “correct” and thus improving cache hit rate) in Section 4.1.
- Identifying a new and extremely subtle tradeoff between cache hit rate and acceptance rate, and optimizes the draft sampling distribution in light of this tradeoff in Section 4.2. Exploiting this leads to large speedups at high temperature (Figure 6) when speculative decoding techniques usually fail.
- Carefully choosing the “fallback” strategy for the case where none of the branches were correct (important for batch size > 1 and temperature > 0) in Section 4.3.
- Providing theoretical results and bounds around what is the optimal strategy for each: predicting recovery tokens, recovering from cache misses, reasoning about the Pareto frontier in cache hit vs acceptance rate, and about the expected and maximal gains from each of these design choices both in isolation and end to end.

In the end, our algorithm and implementation attains a new state-of-the-art average of **~230 tok/sec** (Llama-70B/1B, TP=4, H100), compared to **~26 tok/sec** for SpecBranch in a similar but not identical setup (Llama-70B/8B, TP=4, A100). To achieve this, we had to implement our design choices in a way that works efficiently with staple parts of the modern inference stack: PagedAttention, continuous batching, CUDAGraphs, and torch compilation, each of which make a naïve implementation of asynchrony untenably slow. **We have added a substantial discussion of these important systems design details in Appendix B.1.** We will open source the engine we wrote at the time of conference decisions to avoid violating anonymity. We have definitely built on important prior work, and have significantly expanded our discussion of related work: thank you again for bringing this very interesting and relevant work to our attention, we are glad to cite and discuss it!

Add:

Public Comment by Yuhao Shen

Public Comment by Yuhao Shen 28 Nov 2025, 12:01 (modified: 28 Nov 2025, 12:07) Everyone Revisions

Comment:

I appreciate your detailed response. The revisions have elevated the paper's novelty and contribution. However, I noticed that SpecBranch does not utilize Tensor Parallelism (TP) for acceleration. The implementation appears to be based solely on the transformers. To my knowledge, nano-PEARL [1] is the only open-source system framework for parallel speculative decoding and I am one of the contributors. Therefore, I have a question regarding SSD: Upon which framework is your system implementation built? Did you complete a full system-level implementation (page attention, cuda graph, torch compile) entirely during the rebuttal phase? I look forward to the open-source of SSD.

[1]<https://github.com/smart-lty/nano-PEARL>

Add:

Official Comment by Authors

[Official Comment](#) by Authors 04 Dec 2025, 01:19 (modified: 04 Dec 2025, 01:43) Everyone [Revisions](#)

Comment:

We are glad to hear that you have appreciated our response and paper revisions, and how they better highlight the paper's novelty and contribution. We are glad to answer your follow-up questions here:

- **Regarding tensor parallelism:** We were referring to how inference in SpecBranch for the 70B model was performed across 4 GPUs, which is the same number of GPUs we use for SSD. We based this on the details from Appendices E.2 and E.3 of the SpecBranch paper, which specify that it runs “*70B models on four devices*” using “*bfloat16 format for optimized GPU inference without quantization*” on “*NVIDIA A100-PCIE-40G GPUs*”. Our implementation uses tensor parallelism; HuggingFace defaults to pipeline parallelism, but we believe this to be a fair comparison because both serve the same model at the same precision on the same number of devices.
- **Regarding the framework upon which SSD was built:** We implement our engine from scratch in pure PyTorch making use only of existing attention kernels like FlashAttention/FlashInfer. The entire engine (including support for PagedAttention, continuous batching, CUDAGraphs, torch compilation, etc) was written before submission, not during rebuttals. That is what was used for SSD tok/sec benchmarking. The systems design portion of the Appendix was also present in the originally submitted manuscript, we simply expanded it and highlighted it during the rebuttal to make it clearer.

Thank you so much for your interest in our paper and for your excitement about us open-sourcing our engine at the time of conference decisions.

Add:

Official Review of Submission9487 by Reviewer kXox

[Official Review](#) by Reviewer kXox 04 Nov 2025, 19:08 (modified: 12 Nov 2025, 09:47) Everyone [Revisions](#)

Summary:

This paper proposes to parallelize the draft and verifier models in speculative decoding by running them on separate hardware devices. The proposed approach demonstrates superior performance compared to traditional speculative decoding in the reported experiments.

Soundness: 2: fair

Presentation: 1: poor

Contribution: 2: fair

Strengths:

1. The proposed method outperforms standard speculative decoding (SD) in the reported experiments.
2. Introducing parallelism between the draft and verifier models represents an interesting and promising direction for advancing speculative decoding.

Weaknesses:

1. The algorithmic description is difficult to follow in the current paper organization. It is unclear how the proposed method preserves the original distribution, and while the paper provides extensive theoretical analysis, it offers limited explanation and intuition about the algorithm itself, particularly in Section 3.
2. The paper lacks baseline comparisons against other speculative decoding (SD) approaches, making it hard to assess the relative performance and contribution of the proposed method.

3. The paper does not include a dedicated experimental section—experiments appear only as a subsection—and there is insufficient detail regarding the setup, such as which models were evaluated and under what conditions.

Questions:

Could you improve the paper's structure to make the algorithm easier for readers to follow? Additionally, could you add more experiments to address the weaknesses noted in points 2 and 3, particularly by including comparisons with other speculative decoding methods and providing more detailed experimental setups? Finally, could you clarify how your method preserves the original distribution, both conceptually and mathematically?

Flag For Ethics Review: No ethics review needed.

Details Of Ethics Concerns:

NA

Rating: 2: reject, not good enough

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Response from Authors

[Official Comment](#) by Authors 27 Nov 2025, 10:07 (modified: 27 Nov 2025, 19:48) Everyone [Revisions](#)

Comment:

We thank the reviewer for their time. We have rewritten parts of the manuscript to explicitly address your feedback, and addressed some misconceptions below. Changes are highlighted in red in the new manuscript.

Weakness 1

The algorithmic description is difficult to follow in the current paper organization. It is unclear how the proposed method preserves the original distribution, and while the paper provides extensive theoretical analysis, it offers limited explanation and intuition about the algorithm itself, particularly in Section 3.

Thank you for sharing that you found the presentation of the SSD algorithm hard to understand, and that it was unclear to you why the algorithm is lossless (i.e., output distribution matches autoregressive decoding). **We have added intuition and further explanations to Section 3, which we also share here.**

To provide intuition about the algorithm, and better understand why it is lossless, it is useful to think about why the classical technique of speculative execution can speed up CPU performance without changing the behavior of the code. Speculative execution uses idle compute resources to preemptively execute certain conditional blocks or pre-fetch data before it is known if they'll be needed, such that if they are eventually needed, the work will already be done. SSD is doing something analogous with LLM forward passes relative to ordinary SD. SSD uses available compute resources to pre-speculate for many possible verification outcomes, such that if any of those outcomes does occur, the next speculated token sequence will be ready immediately. This token sequence is then verified in the same exact way SD performs verification, thereby maintaining the lossless properties of standard SD. In the case where SSD did not prepare for the verification outcome that actually occurred, a synchronous "fallback speculator" is used to generate the speculation, which also maintains the lossless property of SD.

We hope that this makes the algorithm much clearer, and we would be happy to answer any follow up questions. Thanks for bringing this up.

Weakness 2

The paper lacks baseline comparisons against other speculative decoding (SD) approaches, making it hard to assess the relative performance and contribution of the proposed method.

Our baselines are very strong: our implementation of ordinary speculative decoding as a baseline is faster than that of SGLang with SD, which was the fastest of all the popular inference engines (vLLM, SGLang, TRT-LLM) we tried. **To compare with an even stronger baseline, we have added a new comparison to SGLang with EAGLE-3 in Figure 8, and demonstrate that SSD outperforms this (previously) SOTA baseline by ~36% and ~44% on average for Llama-3.3-70B and Qwen3-32B, respectively. Also, it is possible to combine our method with EAGLE-3, and we are in the process of implementing this. We have added explicit discussion around how exactly the two methods can be implemented together in detail in Appendix E.**

Weakness 3

The paper does not include a dedicated experimental section—experiments appear only as a subsection—and there is insufficient detail regarding the setup, such as which models were evaluated and under what conditions.

Thank you for this feedback. **We have now moved our end-to-end results to a standalone section (Section 5) per your suggestion.** We have chosen, for now, to keep the more fine-grained experimental results validating the three main Saguaro optimizations embedded in their corresponding subsections, to make it easier for readers to assess the claims for that section while these (often complicated) ideas are fresh in the reader's mind.

To see details about our experimental setup, we'd like to refer you to our "Setup" section at the beginning of Section 4 (further details in Appendix B.2), which is written specifically to answer your questions. We have highlighted these in red in the new manuscript to emphasize this in case you want to take a look.

We believe we have addressed each concern or issue posed by the reviewer, and we would be delighted to answer any other outstanding questions, or otherwise invite the reviewer to consider raising their score. Thank you.

Add:

Official Review of Submission9487 by Reviewer h3mq

Official Review by Reviewer h3mq01 Nov 2025, 04:37 (modified: 12 Nov 2025, 09:47)EveryoneRevisions

Summary:

This paper introduces Speculative Speculative Decoding (SSD), a framework that parallelizes the drafting and verification stages of speculative decoding by having the draft model predict likely verification outcomes in advance and pre-speculate for them. The authors propose SAGUARO, an optimized SSD algorithm that addresses three key challenges: accurately predicting verification outcomes, balancing cache hit rate with token acceptance rate, and handling cache misses efficiently. SAGUARO achieves up to 2x speedup over standard speculative decoding and up to 5x over autoregressive decoding on open-source inference engines.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

1. The paper presents a principled and theoretically grounded approach to overlapping drafting and verification, effectively eliminating drafting latency through pre-speculation.
2. SAGUARO introduces practical and novel techniques—geometric fan-out allocation, cache-aware sampling, and adaptive fallback strategies—that collectively address core challenges in SSD.

3. The evaluation is comprehensive, covering diverse models, datasets, temperatures, and batch sizes, and shows significant end-to-end speedups while maintaining compatibility with existing acceleration methods.

Weaknesses:

1. The authors do not compare their method with existing SOTA methods (e.g., EAGLE2/3). They argue that their method is orthogonal to EAGLE and can be combined with it. However, the paper does not report the performance of their method when combined with EAGLE3. I would like to know what performance improvement could be achieved if they were combined.
2. The authors do not compare their method with existing Token Tree SPD methods, which I believe is a missing baseline. The proposed method (SSD) has the draft model send only a single speculative sequence to the verifier, and while the verifier validates that sequence, the draft model computes other possible branches in parallel. In contrast, Token Tree SPD methods have the draft model generate an entire "tree" structure at once, providing multiple candidate tokens at each position, and the verifier must validate all branches and nodes of this tree in a single forward pass. I consider these to be two similar solution approaches, and therefore the authors should compare these two types of methods. For example, they could replace the Token Tree SPD component in EAGLE3 with their SSD method and report whether the performance increases or decreases.

Questions:

1. I would like the authors to provide the wall time for their method. For example, while the target model is performing Verify, how many tokens can the draft model generate in that same amount of time?
2. I would like to know the number of accepted tokens for your method. Compared to EAGLE3, does your method accept more or fewer tokens per round?

I will adjust my score based on the author's rebuttal.

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Comment by Reviewer h3mq

Official Comment by Reviewer h3mq25 Nov 2025, 19:40Everyone

Comment:

Dear Authors,

Thank you again for your submission. I would like to follow up on the questions and concerns I raised in my previous review. So far, I have not received a response or clarification. To fairly assess the contribution of your paper, I would appreciate it if you could address these points.

In addition, I would also appreciate it if you could respond to the public comments related to your paper. These clarifications will help ensure a more accurate and fair evaluation. Otherwise, the lack of response may affect my final assessment and score.

Add:

Official Response by Authors (Part 1)

Official Comment by Authors27 Nov 2025, 10:11 (modified: 27 Nov 2025, 19:51)EveryoneRevisions

Comment:

Thank you for the detailed review and close-read of our paper! We respond to your weaknesses/questions inline below, and have made updates to the manuscript (highlighted in red in the new version) as a result of your feedback. Please also feel free to see our response to the public comment above about related work.

Weakness 1

The authors do not compare their method with existing SOTA methods (e.g., EAGLE2/3). They argue that their method is orthogonal to EAGLE and can be combined with it. However, the paper does not report the performance of their method when combined with EAGLE3. I would like to know what performance improvement could be achieved if they were combined.

We have added an explicit comparison with the SOTA EAGLE3 implementation in SGLang in Figure 8, finding that we significantly outperform it (by ~36% and ~44% on average for Llama-3.3-70B and Qwen3-32B, respectively). To substantiate our claim that our method can also be combined with EAGLE-3, we are in the process of reimplementing EAGLE-3 in our optimized inference engine in a way that works with SSD. This is nontrivial from an engineering perspective because it requires passing target activations to the draft over NCCL and doing careful accounting of KV cache context when asynchronously drafting to track whether we are conditioning on draft vs target activations. But it is in progress, thanks for bringing this up! **We have added a detailed systems design for exactly how EAGLE-3 and SSD are combined in Appendix E.**

We now provide a simple back-of-the-envelope calculation on the expected speedups from combining SSD and EAGLE-3, estimating that **we could attain at least ~33% speedup over SSD in the case of Llama-70B.** We note that EAGLE-3 often uses a single-layer draft model as opposed to our current Llama-3.2-1B draft with 16 layers. In our measurements timing the forward passes of both draft models, this can reduce the draft forward pass latency from ~2ms (current) to less than 1ms. This allows us to at least double (e.g., from 5 to 10 for Llama-70B) the number of draft forward passes we can do during the verification, and thus increase the expected number of accepted tokens from ~4.2 to ~5.6 for Llama-70B (assuming the acceptance rate of EAGLE-3 matches that of Llama-3.2-1B). Thus, even without leveraging the improved acceptance rates of EAGLE-3 relative to standalone draft models, we expect to theoretically attain at least a 33% speedup (over our already SOTA tokens/sec) over SSD end-to-end when combining it with EAGLE-3!

Weakness 2

The authors do not compare their method with existing Token Tree SPD methods, which I believe is a missing baseline. The proposed method (SSD) has the draft model send only a single speculative sequence to the verifier, and while the verifier validates that sequence, the draft model computes other possible branches in parallel. In contrast, Token Tree SPD methods have the draft model generate an entire "tree" structure at once, providing multiple candidate tokens at each position, and the verifier must validate all branches and nodes of this tree in a single forward pass. I consider these to be two similar solution approaches, and therefore the authors should compare these two types of methods. For example, they could replace the Token Tree SPD component in EAGLE3 with their SSD method and report whether the performance increases or decreases.

Thank you for pointing out the missing comparisons to token-tree verification SD methods, and the relationship between SSD and these methods.

As mentioned above, **we have now added an explicit comparison between SSD and SGLang with EAGLE-3 (a SOTA token-tree SD method + SOTA draft architecture), showing that SSD meaningfully outperforms this strong baseline (by ~36% and ~44% on average for Llama-3.3-70B and Qwen3-32B, respectively).** Additionally, we are in the process of implementing exactly the ablation you propose where we replace the token tree component of EAGLE3 with SSD to measure the performance gains we estimated above.

The key point to have in mind is that you can think of SSD as drafting a tree, but only verifying a linear sub-sequence within that tree during each target model forward pass. This means it sidesteps the need for expensive tree verification by the large and slow target model. **Our main point is that the overall framework is a general synchronous to asynchronous algorithm rather than one that specifically competes with or replaces the myriad of existing variations on (synchronous) speculative decoding that already exist.**

Add:

Official Response by Authors (Part 2)

Official Comment by Authors 27 Nov 2025, 10:14 (modified: 27 Nov 2025, 10:48) Everyone Revisions

Comment:

Question 1

I would like the authors to provide the wall time for their method. For example, while the target model is performing Verify, how many tokens can the draft model generate in that same amount of time?

We added a table with wall-clock metrics (like inter-token latency and tokens/sec) in Appendix B.3.

For example, we can attain between 200 and 257 tokens/sec for Llama-70B with SSD depending on the dataset. Here is a wall clock breakdown of what happens in one asynchronous speculation round under the hood in our current implementation so you have intuition.

- Verification latency + overhead: ~18ms (16 ms verification forward pass, 2 ms overhead)
- Draft does 5 forward passes in this time (hidden entirely on critical path while verification taking place):
- Expected number accepted tokens with 85% acceptance (measured, under greedy decoding): ~4.15
- Overall: 18ms for ~4.15 tokens → ~230 tok/s, which is what we see empirically.

Question 2

I would like to know the number of accepted tokens for your method. Compared to EAGLE3, does your method accept more or fewer tokens per round?

For Llama-70B, we attain around 4 accepted tokens per round, which sits between EAGLE-2 (3.5 reported) and EAGLE-3 (~5 reported). These differences come from the fact that EAGLE-3 can attain higher acceptance rates than Llama-1B, while also doing token tree drafting/verification to increase the expected number of accepted tokens. SSD has the same acceptance rate as ordinary SD but hides the draft latency entirely, so is still faster than EAGLE3 end to end. Two things to note here:

- Our draft model actually has longer to draft (~18ms above) than drafting usually takes in SD. This means in principle we can draft for more steps so that our expected tokens per round is higher than EAGLE3. However, in practice, the use of a custom attention mask leads to slower forward passes on the draft so that our implementation only fits in 5 draft forward passes per speculation round (comparable to the number of forward passes in SD). This is an engineering detail, not a limitation of the algorithm itself, and suggests there is room for further systems optimizations to push our inference speeds (already SOTA) even further.
- As mentioned earlier, SSD and EAGLE3 can be combined in a way we articulate in the new Appendix E, and are in the process of implementing.

Thank you for your thoughtful review. We hope this substantive response helps you better understand the nuances of the work and would be happy to engage further, and otherwise would appreciate if you consider raising your score. Thank you.

Add:

Official Review of Submission9487 by Reviewer Jb3k

Official Review by Reviewer Jb3k 31 Oct 2025, 21:05 (modified: 12 Nov 2025, 09:47) Everyone Revisions

Summary:

This paper introduces Speculative Speculative Decoding (SSD) — a new framework that parallelizes both speculation and verification in a distributed manner. While typical SD alternates executions between drafter and verifier, SSD removes this dependence and performs drafting and verification concurrently and asynchronously. In SSD, while verification for round T is ongoing, the draft model predicts the likely verification outcomes and pre-speculates corresponding continuations.

Soundness: 3: good

Presentation: 1: poor

Contribution: 3: good

Strengths:

- This paper has novelty in terms of serving the draft and target models asynchronously.
- This paper has clear derivations and proofs (Theorem 7, 12, 17), which make the framework analytically sound.
- The author has conducted extensive benchmarks on multiple model families, detailed ablations (fan-out, sampling, fallback), and clear visualizations.

Weaknesses:

- The flow of the paper can be improved, e.g. Section 4 shows the results first and then explains the algorithms
- Can give a better flowchart of the workflow in Figure 1, e.g. label the step index
- This method requires an independent GPU for the speculator, this can increase the cost of serving and system complexity (e.g. heterogeneous GPUs)

Questions:

- While the deployment of LLMs typically runs on GPUs, SSD actually requires GPUs. How can users deploy the SSD for serving efficiently? e.g. if using homogeneous GPUs, using a Hopper GPU for only the small draft model can be an overkill as much of the compute and memory are not saturated. If using heterogeneous GPUs (i.e. running the speculators on another type of GPU), how much impact will the communication between speculators and verifiers be?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Response by Authors

[Official Comment](#) by Authors 27 Nov 2025, 10:22 (modified: 27 Nov 2025, 19:54) Everyone Revisions

Comment:

Thank you for your careful review. We have updated our manuscript in response to your comments and highlighted new portions in red. We respond to your comments inline:

Weakness 1

The flow of the paper can be improved, e.g. Section 4 shows the results first and then explains the algorithms.

Thanks for this feedback. **We have now moved the end-to-end results (including new comparisons to SGLang with EAGLE-3) to Section 5, after the method has been introduced, as suggested.**

Weakness 2

Can give a better flowchart of the workflow in Figure 1, e.g. label the step index.

We have updated Figure 1 to now include the step index, as requested. Hopefully this makes the figure clearer. We'd love to hear any additional suggestions for how to improve this figure.

Weakness 3

This method requires an independent GPU for the speculator, this can increase the cost of serving and system complexity (e.g. heterogeneous GPUs)

Our method actually does much better in cost, where the relevant metric is tokens/s/GPU. To illustrate this, we measured the performance of SGLang w/ SD on both TP=4 and TP=8 with Llama-3.3-70B. TP=8 is actually slightly slower since the compute per GPU is already small and there is more communication overhead in the forward pass. Thus in the end the tok/s/GPU (cost) are:

- Llama-3.1-70B (SD, TP=4) $\sim 138/4 = 34.5$
- Llama-3.1-70B (SD, TP=8) $\sim 131/8 = 16.3$
- SSD (TP=4 + 1 draft GPU): $\sim 230/5 = 46$

As we can see, SSD is the most economical.

Question 1

While the deployment of LLMs typically runs on GPUs, SSD actually requires GPUs. How can users deploy the SSD for serving efficiently? e.g. if using homogeneous GPUs, using a Hopper GPU for only the small draft model can be an overkill as much of the compute and memory are not saturated. If using heterogeneous GPUs (i.e. running the speculators on another type of GPU), how much impact will the communication between speculators and verifiers be?

The GPU used for speculation is in fact saturated, even if we use an H100. This is because we leverage the full capacity of the GPU to pre-speculate in parallel for as many verification outcomes as possible, decoding many more tokens than are verified on the draft model. We have added a concrete calculation of the extra FLOPs the draft model must pay compared to the target in our setup in Appendix C to make this clearer. Thus, there is no specific pressure to use heterogeneous compute implied by our method.

Regarding communication, we find the overhead to be negligible, given that the only communication between draft and target is the verification outcome to the speculator once per round (tokens and logits).

We include an exact breakdown of how much data is sent and the time requirements in the new Appendix C.

Thank you for the perceptive questions. We believe we have revamped the diagrams/section to address presentation issues and explicitly addressed your technical concern. Let us know if we can help with anything else. Otherwise, we would appreciate it if you consider raising your score.

Add:

Official Review of Submission9487 by Reviewer QxMH

Official Review by Reviewer QxMH23 Oct 2025, 20:47 (modified: 12 Nov 2025, 09:47)EveryoneRevisions

Summary:

This paper introduces Speculative Speculative Decoding (SSD), a novel framework that parallelizes the drafting and verification phases in speculative decoding. By pre-speculating likely verification outcomes during ongoing target model verification, SSD eliminates the sequential dependency between drafting and verification, achieving significant latency reductions.

Soundness: 4: excellent

Presentation: 4: excellent

Contribution: 4: excellent

Strengths:

1 The idea of speculating on speculation is both creative and impactful. By leveraging idle draft-model compute during verification, SSD addresses a fundamental bottleneck in existing speculative decoding methods. The theoretical framework (Algorithm 1) is elegant and generalizable.

2 The paper provides strong theoretical analysis.

3 Comprehensive Empirical Validation.

Weaknesses:

1 While SSD reduces latency, the parallel speculation across multiple branches increases draft-model FLOPs. The paper should quantify this overhead more explicitly.

2 Although comparisons to standard SD are thorough, benchmarking against recent parallel SD methods (e.g., SwiftSpec) or tree-based decoding (e.g., SpecInfer) would strengthen the claims. Does SSD outperform these methods in high-batch or high-temperature regimes?

Questions:

1 How does SAGUARO's memory footprint scale with batch size and fan-out? Could cache eviction policies improve efficiency?

2 Could the pre-speculation idea be combined with approximate drafting (e.g., EAGLE-3) for further gains?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Add:

Official Response by Authors

Official Comment by Authors 27 Nov 2025, 10:29 (modified: 27 Nov 2025, 19:59) Everyone Revisions

Comment:

Thanks for your the awesome review! We're glad you felt the soundness, presentation and contribution of our work was excellent. We'll take your points line by line.

Weakness 1

While SSD reduces latency, the parallel speculation across multiple branches increases draft-model FLOPs. The paper should quantify this overhead more explicitly.

You are correct that SSD increases draft model FLOPS, and we include the exact calculation in the new Appendix C. In summary, the draft uses more FLOPs than regular SD drafting, where is the speculative lookahead, and the fan-out-factor for predicting recovery tokens. Importantly, however, LLM decoding is typically memory-bound, not compute-bound, which makes additional FLOPs essentially free in this regime.

Weakness 2

Although comparisons to standard SD are thorough, benchmarking against recent parallel SD methods (e.g., SwiftSpec) or tree-based decoding (e.g., SpecInfer) would strengthen the claims. Does SSD outperform these methods in high-batch or high-temperature regimes?

We are unable to compare directly to SwiftSpec as there is no code released and the technique relies on many proprietary kernels. **We now compare with EAGLE-3 in Figure 8, a SOTA token-tree method, and find SSD to be comfortably faster (by ~36% and ~44% on average for Llama-3.3-70B and Qwen3-32B).**

Regarding how SSD compares with these baselines across batch sizes and temperatures: SSD's speedups over existing parallel SD methods, as well as tree-based SD methods, increase as batch size and temperature vary for the reasons below.

Batch size. SwiftSpec uses a naive just-in-time fallback, which works at low batch sizes when the probability of at least one cache miss is low, but becomes suboptimal at higher batch sizes when this probability rises. **Corollary 16 and Theorem 17 in Section 4.3 of our paper make this suboptimality precise.** In the high-batch regime, token-tree methods like SpecInfer and EAGLE3 quickly struggle because verification time balloons as the compute done by the large verifier grows. SSD gets around these issues by adapting fallback strategies with batch size, and only increasing draft FLOPS, not verifier FLOPS.

Temperature. SwiftSpec only considers greedy decoding. As temperature rises, cache misses and an optimal fallback strategy become important considerations for the reasons above. SpecInfer struggles at low temperature because it can resample rejected tokens many times, using large amounts of verifier compute on these tokens without gain. **Saguaro sampling (Section 4.2) balances the fact that cache hit rates falls quickly at high temperature but acceptance rates fall slowly. The ability to trade them off in this subtle way allows for Saguaro to do very well at both low and very high temperatures (see Figure 6).**

Question 1

How does SAGUARO's memory footprint scale with batch size and fan-out? Could cache eviction policies improve efficiency?

The memory on the draft scales linearly with both batch-size and fan out, but the total size is very small overall (megabytes) compared to GPU HBM (gigabytes), and is already refreshed every round. **We have added an explicit calculation of the memory usage on the draft in the new Appendix C.**

Question 2

Could the pre-speculation idea be combined with approximate drafting (e.g., EAGLE-3) for further gains?

We can absolutely combine SSD with Eagle-3! In fact, we are in the middle of implementing this combination, and **have outlined the systems design for how to do this in the new Appendix E.** By our calculation, we should be able to attain at least 33% speedup over SSD by combining with EAGLE-3. We shared this calculation with Reviewer h3mq, and copy it here for convenience:

We now provide a simple back-of-the-envelope calculation on the expected speedups from combining SSD and EAGLE-3, estimating that we could attain at least ~33% speedup in the case of Llama-70B. We note that EAGLE-3 often uses a single-layer draft model as opposed to our current Llama-3.2-1B draft with 16 layers. In our measurements, this can reduce the draft forward pass latency from ~2ms (current) to less than 1ms. This allows us to at least double (e.g., from 5 to 10 for Llama-70B) the number of draft forward passes we can do during the verification, and thus increase the expected number of accepted tokens from ~4.2 to ~5.6 for Llama-70B (assuming the acceptance rate of EAGLE-3 matches that of Llama-3.2-1B). Thus, even without leveraging the improved acceptance rates of EAGLE-3 relative to standalone draft models, we expect to theoretically attain at least a 33% speedup (over our already SOTA tokens/sec) of our method end-to-end when SSD is combined with EAGLE-3!

Please let us know if we can answer any other questions, and we appreciate the review. If we have addressed the questions or concerns you had, we would be delighted if you consider raising your score.

Add:

[About OpenReview](#)

[Contact](#)

[FAQ](#)

[Hosting a Venue](#)

[Sponsors](#)
[Terms of Use / Privacy Policy](#)
[All Venues](#)
[Donate](#)
[News](#)

OpenReview is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026 OpenReview

OpenReview.net

Login



[Go to ICLR 2026 Conference homepage](#)

FOLD: Fast Correct Speculative Decoding



Haocheng Sun, Ruilin Wang, Yuxin Zhang, Yan Zhang, Tianyu Xie, Rongrong Ji, Xiawu Zheng

19 Sept 2025 (modified: 27 Jan 2026)ICLR 2026 Conference Withdrawn

SubmissionEveryoneRevisionsBibTeXCC BY 4.0

Keywords: speculative decoding, early exit, inference acceleration, large language models

TL;DR: We introduce FOLD(Fast cORrect specuLative Decoding) to accelerate the inference speed of Large Language Models (LLMs) by fast correcting wrong token.

Abstract:

Speculative decoding accelerates Large Language Model (LLM) inference by using a small, fast 'draft' model to propose tokens that a larger 'target' model then verifies in a single, parallel step. While this paradigm has become the standard for high-throughput inference, the community's focus has been almost entirely on a single metric: maximizing the acceptance rate of drafted tokens. We argue this is a critical oversight. The true bottleneck is not just acceptance, but the catastrophic computational cost of rejection. A single rejected token triggers a cascading failure, discarding all subsequent work and nullifying potential gains. We introduce Fast cORrect specuLative Decoding (FOLD), a framework that fundamentally reframes the problem from merely avoiding rejection to instantly recovering from it. FOLD transforms the verification step itself. Instead of a simple pass/fail check, our novel verifier uses an integrated Early Exit module to proactively generate high-probability alternative sequences in parallel. When the primary draft fails, FOLD doesn't discard the computation; it seamlessly pivots to a pre-computed, correct path. This turns a catastrophic failure into a minor course correction, salvaging the entire speculative branch. Extensive experiments show that by treating rejection as an opportunity for correction, not a point of failure, FOLD achieves up to a 4.09 speedup over Auto Regression decoding, setting a new bar for inference efficiency. We anonymously open-source our project at <https://anonymous.4open.science/r/iclr26-fold>.

Primary Area: generative models

Submission Number: 16674

Filter by reply type...

Filter by author...

Sort: Newest FirstSort: Oldest First
15 / 15 replies shown

Withdrawal by Authors

Withdrawal by Authors27 Jan 2026, 08:11Everyone

Withdrawal Confirmation: I have read and agree with the venue's withdrawal policy on behalf of myself and my co-authors.

Paper Decision

Decision by Program Chairs26 Jan 2026, 14:17 (modified: 12 Feb 2026, 21:29)EveryoneRevisions

Decision: Reject

Meta Review of Submission16674 by Area Chair EVZE

Meta Review by Area Chair EVZE28 Dec 2025, 00:27 (modified: 12 Feb 2026, 21:31)EveryoneRevisions

Summary:

This paper proposes a new method called FOLD (Fast cORrect specuLative Decoding) that utilizes a four stage process to proactively generates and verifies multiple alternative sequences using an Early Exit Module and Tree Attention. Compared with the default approach, this method does not waste all unselected tokens. The reviewers' main concerns are on the experiment settings. E.g., Some methodological details are missing and the experiments are on a relatively strong scale. Hence, I agree that this paper should be further improved to meet the bar.

Reviewer Concerns:

1. Not comprehensive enough experiments
2. The proposed method might introduce additional memory usage issues
3. The experiment scale is not big enough.

Reviewer Scores:

I have carefully read all rebuttals and reviewer opinions and do not see strong signals indicating a change in scores.

Official Review of Submission16674 by Reviewer qNND

Official Review by Reviewer qNND01 Nov 2025, 11:55 (modified: 12 Nov 2025, 11:22)EveryoneRevisions

Summary:

The paper presents FOLD (Fast cORrect specuLative Decoding), a novel framework that improves the efficiency of speculative decoding in Large Language Models by mitigating the high cost of token rejection. Unlike traditional methods that discard all computation after a rejected token, FOLD introduces a four-stage process—Early Draft, Early Verify, Draft Correct, and Final Verify—that proactively generates and verifies multiple alternative sequences using an Early Exit Module and Tree Attention. This allows the system to recover from errors without losing progress, turning rejection into correction. Compatible with existing approaches like Pearl, FOLD achieves up to a 4.09× speedup over standard auto-regressive decoding, marking a significant advancement in inference performance.

Soundness: 3: good

Presentation: 3: good

Contribution: 2: fair

Strengths:

- Reframes speculative decoding by treating token rejection as an opportunity for rapid correction instead of failure to improve efficiency.
- Introduces a four-stage framework with an Early Exit Module and Tree Attention, enabling parallel correction and preserving useful computation.
- Demonstrates up to a 4.09× speedup over standard auto-regressive decoding and is easily adaptable to existing methods like Pearl, proving both performance and practicality.

Weaknesses:

- Some methodological details, such as hyperparameter choices, are insufficiently explained, making the approach harder to reproduce or tune.
- The proposed Tree Attention mechanism, while innovative, may introduce additional memory overhead during inference, potentially limiting scalability on resource-constrained hardware.
- The experiments focus on short-context datasets, leaving uncertainty about FOLD's effectiveness and efficiency on longer or more complex tasks, which limits the demonstrated generalizability of the approach.

Questions:

- On line 186, there is " $\gamma \times K + 1$ draft branches", would that be " $\gamma \times k + 1$ "? Can you explain more on this?
- In the tree attention part, multiple branches will be generated, which increases the GPU memory usage/accesses at inference time. Considering the decoding stage in LLM inference, which is already memory-bound, will this tree attention further induce bottlenecks on limited bandwidth? Can you explain qualitatively or quantitatively?
- GSM8K and HumanEval are used in the experiment section. Does FOLD also work on datasets with longer prompt lengths, such as arxivSummary or other similar datasets, to further demonstrate the effectiveness of FOLD under different workloads?
- How does FOLD decide what values to be assigned for γ_1 and γ_2 at inference time? (i.e, does FOLD use fixed values for both across different prompts or does FOLD dynamically change values) For example, given two prompts of different difficulty levels (one is easy to answer/generate results, while the other is not), I assume the draft model in the Early Draft phase may be expected to generate different numbers of tokens (different γ_1). Can you elaborate more on this point qualitatively or quantitatively?

Minor typos: Line 88-89, farward -> forward Line 459, parameterkindirectly -> parameter k indirectly Line 461, differentkvalues -> different k values Line 461, parameterkincreases -> parameter k increases Line 464, increasingkto -> increasing k to

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Official Comment by Authors

Official Comment by Authors 20 Nov 2025, 19:40 Everyone

Comment:

Dear Reviewer,

Thank you for your thoughtful review and valuable suggestions. We sincerely appreciate the time you dedicated to evaluating our work, and we have carefully addressed all your concerns below. Your insights have significantly helped us improve the clarity and completeness of our methodology.

1. Clarification on " $\gamma \times K + 1$ " (Line 186)

To explain the "+1" term: During verification of a draft sequence of length γ , our Early Exit module generates logits across γ positions. For each position, we select the top- k probable candidates to create $\gamma \times k$ correction branches. Crucially, we retain the original draft branch since it may still be valid. Thus, the total branches become $\gamma \times k + 1$, combining correction paths with the original candidate.

2. Memory Overhead Analysis of Tree Attention

We appreciate your insightful question regarding memory bottlenecks. Our analysis confirms that Tree Attention **does not significantly increase memory-bound constraints**, as the additional overhead is marginal compared to baseline decoding costs. Below is a quantitative breakdown for **Llama2-7B**:

Component	Size Calculation	Memory Usage
Model Weights (per token)	7B params × 2 bytes	14 GB
KV Cache (per token)	32 layers × (4096 Key + 4096 Value) × 2 bytes	0.524 MB

The 14 GB weight transfer dominates memory bandwidth during token generation. While Tree Attention introduces extra tokens, their aggregated KV cache remains negligible (e.g., 100 tokens ≈ 52.4 MB) against the 14 GB baseline. Once a verification completed, only the KV cache of accepted tokens will be retained.

3. Evaluation on Long-Context Datasets

We conducted additional experiments on **SpecBench**'s **summarization** and **RAG** tasks (long-prompt scenarios). Results are summarized below:

Model	Method	Summary	Speedup	RAG	Speedup
Llama3-8B&70B	AR	14.85	1	14.85	1
	Pearl	42.16	2.84	40.79	2.75
	FOLD	43.53	2.93	39.13	2.64
Llama3-1B&70B	AR	14.85	1×	14.85	1
	Pearl	45.42	3.06	46.34	3.12
	FOLD	50.44	3.40	46.17	3.11
Llama2-7B&70B	AR	13.77	1	13.77	1
	Pearl	34.13	2.48	34.43	2.50
	FOLD	36.66	2.66	36.27	2.63

FOLD maintains competitive performance but may exhibit minor regressions in some RAG tasks with Llama3 series (e.g., -1.66% in RAG for Llama3-8B). This aligns with our discussion in Line 429: the current Early Exit module uses a simple design with simple training data for feasibility demonstration, not long-context optimization. In addition, there is a specific research direction focused on [long-context-speculative-decoding](#). Objectively speaking, we do not belong to this direction. Thank you for your understanding.

4. Hyperparameter Settings for γ_1 and γ_2

We thank you for this nuanced question. In our framework, γ_1 and γ_2 are static values determined by two factors:

1. Target/Draft model speed ratio.
2. Proportion of First N Layers (input to Early Exit).

For optimal balance between Early Exit accuracy and draft token volume:

If the speed ratio is odd, we adjust N to approach (but not exceed)

Regarding dynamic γ -values for varying prompt difficulties: we think it is an interesting direction. However, reliably quantifying complexity and difficulty of prefix sequence remains an open challenge, which is required by achieving dynamic and . If it is possible, there will be lots of methods to improve our framework like multi-draft structure.

We hope these explanations fully address your concerns. Thank you again for your invaluable feedback, which has strengthened our work. We look forward to incorporating these clarifications in the final manuscript.

Best regards, author.

Official Review of Submission16674 by Reviewer R4Ma

Official Review by Reviewer R4Ma01 Nov 2025, 10:01 (modified: 12 Nov 2025, 11:22)EveryoneRevisions

Summary:

This paper introduces FOLD, a novel method that enhances the speculative decoding process. It refines the two fundamental phases of speculative decoding—draft and verify—into a more granular four-step procedure: early draft, early verify, draft correct, and final verify. This refined, four-stage design is central to the method's effectiveness. It enables a subset of the tokens produced during the initial early draft phase to be supplemented and corrected using partial information from the target model, thereby substantially improving the quality of the draft tokens. Empirical results demonstrate the efficacy of the proposed method.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

1. The central idea of this paper is both novel and sound, and the experimental results validate the effectiveness of the proposed methodology.
2. The paper is well-written, and the figures and tables are clear and easy to follow.

Weaknesses:

1. I observe that all experimental results were derived using the 70B large-scale model. I am curious whether the proposed FOLD method remains applicable and effective at smaller model scales, such as 7B or 14B.
2. The comparison between FOLD and other baseline methods (like vanilla SD) appears to be potentially unfair, as the latter seemingly did not utilize the tree draft mechanism while FOLD did. A portion of FOLD's observed speedup directly stems from the tree draft. To strengthen the persuasiveness of the results, I recommend the authors enable the draft tree mechanism for all compared methods and then re-evaluate the performance accordingly.

- 3. Regarding the results presented in Table 2, I suggest the authors include the speed and the corresponding speedup data for Auto Regression decoding, similar to what is shown in Table 1. This would offer a more complete analysis and comparison of the results.
- 4. The theoretical speedup of this work is contingent upon a meticulously designed parallel mechanism. However, the authors' open-sourced repository seems only to contain the debug code for FOLD, omitting the implementation of the crucial parallel acceleration component. I kindly request the authors to either provide or clearly indicate the implementation details and location of FOLD's core parallel mechanism within the existing open-source code.

Questions:
Please see the weakness.

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Official Comment by Authors
Official Commentby Authors20 Nov 2025, 21:07Everyone

Comment:
Dear Reviewer, We deeply value your expertise in identifying these nuanced aspects, which have prompted us to refine our explanations and experimental presentation. Below are detailed responses to your additional queries.

1. FOLD Scalability to 13B Models

We appreciate your interest in larger-scale validation. We conducted experiments using **Vicuna-13B** (target) and **Vicuna-68M** (draft) with *top-k=4* on dual H100 GPUs. Results are summarized below:

Method	gsm8k	Speedup	humaneval	Speedup
ar	50.60	1.00	50.60	1.00
sd	65.82	1.30	69.05	1.36
pearl	76.76	1.52	76.22	1.51
fold	78.17	1.54	78.77	1.56

For models within the same series (e.g., Llama 3), manufacturers typically release a 7B model as the smallest variant. It is challenging to find suitable, smaller-scale than official 7B models to serve as draft models. Officially released smaller models, trained on similar data and with comparable methodologies, exhibit excellent consistency with their larger counterparts. This strong alignment leads to remarkable speedups in speculative sampling, as evidenced by the outstanding performance of the Llama3.2-1B and Llama3.1-70B pairing in Tables 1 and 2 of our paper.

In contrast, smaller models trained by third parties often show suboptimal consistency due to differing training data. Their performance, not only in terms of alignment but also in practical accuracy, frequently falls short of expectations. Consequently, pairing a 7B target model with a third-party small model yields less than ideal results. Even if FOLD corrects the current incorrect draft tokens, the draft model's

subsequent token predictions remain unreliable, necessitating frequent interventions from the early exit module. This is not an ideal scenario for showcasing the method's potential.

I would like to emphasize that this performance limitation stems primarily from the unavailability of a suitable draft model, rather than from a lack of generalizability in FOLD itself. On the contrary, as demonstrated by Equation 11 in our paper, FOLD is designed to unlock the theoretical upper bound of acceleration performance in current speculative sampling methods. We believe that with more appropriately paired models, the benefits of FOLD would be even more pronounced.

2. Baseline Implementation of Tree Attention

Because FOLD is very ingenious, I may need to explain to you why baselines do not and cannot use tree attention like FOLD does.

Technically, FOLD employs an early exit module to obtain multiple potential outcomes in advance, which are then used to correct draft tokens. This design ensures that each draft token generates only k additional branches, without creating extra branches for the subsequent draft sequence. In contrast, the baseline methods lack a pre-verification mechanism. This makes it impossible to determine the optimal moment to generate top- k branches or to directly construct a complete k -ary tree. As the draft sequence grows, complete k -ary tree would lead to an exponential increase in the number of tokens. Such an approach would not only preclude a fair performance comparison but also significantly increase latency and memory consumption due to being compute-bound, ultimately degrading performance.

Furthermore, I would like to add that, compared to other excellent and distinctive works, the combination of the early-exit module and tree attention in FOLD is both natural and elegant. Our work's distinctiveness, much like outstanding methods such as [Medusa](#) and [Eagle](#), lies in the appropriate and effective application of tree attention. Therefore, to ensure a fair and meaningful comparison, we regrettably chose to maintain the original configurations of the baselines in our experiments.

Official Comment by Authors

Official Comment by Authors 20 Nov 2025, 21:08 Everyone

Comment:

3. Auto-Regression Speedup in Table 2

While Auto-Regression is the benchmark for the target model's inference speed and the basis for our speedup ratio comparison (rather than being a speculative decoding method itself), including its metrics directly in Table 2 would have made the results more intuitive. Our initial reasoning was that since the Auto-Regression speed is a fixed constant, stating it in Table 1 would be sufficient. However, we overlooked how this might affect the reader's ability to make immediate comparisons.

We sincerely appreciate you pointing this out. We will add this data to the table in the revised version of our paper.

4. Code Implementation Details

The core inference logic is located in `src/engine.py` under the function `parallel_debug`. Our repository is a modified version based on the `ParallelSpeculativeDecoding` codebase (available at [smart-ity/ParallelSpeculativeDecoding](#)). To maintain compatibility with the original repository's evaluation commands, we have integrated the evaluation code for FOLD into this structure.

Furthermore, we undertook a significant effort to refactor the inference code for both the draft and target models to meet our specific requirements. These modifications can be found in `src/earlyexit/draft.py` and `src/earlyexit/modeling_llama_v4.51_target.py`, respectively. We understand that this amount of custom work can pose challenges for review, and we hope it has not caused undue difficulty in understanding our implementation.

As a final point, we have thoroughly verified the integrity of the anonymous code repository. We can confirm that it runs correctly and can be used to replicate our evaluation results.

Replying to Official Comment by Authors

Official Comment by Reviewer R4Ma

[Official Comment](#) by Reviewer R4Ma 21 Nov 2025, 08:38 Everyone

Comment:

Thank you for your detailed response; it has addressed some of my initial concerns. However, a few key issues remain that I believe warrant further clarification or analysis.

1. FOLD Scalability to 13B Models

The pairing of **Llama-3.1-8B-Instruct** (target) and **Llama-3.2-1B-Instruct** (draft) appears to be a highly suitable combination for achieving effective acceleration, given the authors' discussion on model family consistency. Could the authors please provide the experimental results for FOLD using this specific model pair?

2. Baseline Implementation of Tree Attention

I believe there are existing techniques to prevent the draft tree from becoming an exponentially increasing k-ary tree. For example, one could pre-define the tree structure and populate it with tokens during the drafting process [1], or implement dynamic pruning of the draft tree [2].

Essentially, my core point is as follows: FOLD's key contribution appears to be integrating the target model's early-exit outputs into the draft tree, resulting in more accurate draft tokens. Since using a draft tree itself can inherently accelerate generation, even if all draft tokens originate from the draft model, it is crucial to **decouple the separate contributions** to speedup provided by the *draft tree structure* and the *draft correction mechanism*. This distinction would better illuminate the unique benefits of FOLD.

[1] EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty

[2] EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test

Official Comment by Authors

[Official Comment](#) by Authors 28 Nov 2025, 12:35 (modified: 28 Nov 2025, 12:36) Everyone [Revisions](#)

Comment:

1. Regarding the Lack of Evaluation with Llama3-1B & 8B

Thank you for raising this important point regarding the potential pairing of Llama-3.1-8B-Instruct (target) and Llama-3.2-1B-Instruct (draft).

We did consider this model pairing early in our research and performed a preliminary analysis. However, we found that the inference speed difference between these two models is minimal—less than a 2x factor, which is not worth correcting wrong draft tokens. Unlike Eagle's draft model which features only a single decoder layer, the Llama3.2-1B architecture incorporates 16 decoder layers. Despite their comparable parameter sizes, the Llama3.2-1B with its increased number of decoder layers demonstrates less impressive performance in inference speed. For this reason, we concluded that this pairing would not serve as a representative or informative test case for our method's capabilities and thus did not proceed with a full set of experiments.

2. Regarding the Application of Tree Attention to Baselines

We explored experimental design possibilities but ultimately concluded that applying FOLD's tree structure to vanilla SD would not permit a fair comparative analysis to decouple the individual contributions of the draft tree structure and the draft correction mechanism. In FOLD, if the Early Exit module corrects a token, the verification of its subsequent tokens is deferred to the next cycle. If we were to apply the same tree structure to vanilla SD and allow it to verify these subsequent tokens within the current cycle, it would

effectively grant the baseline an unfair advantage by giving it a shorter verification loop (with only one verification round). See: [branches of FOLD](#) On the other hand, if omitting successor tokens with correction tokens, the behavior degenerates to standard target model rejection - indistinguishable from conventional autoregressive decoding. See: [branches of vanilla SD](#)

As a practical alternative to address this decoupling concern, **we have included an ablation study in the paper where FOLD operates in a top-1 setting**. This configuration minimizes the influence of the tree attention mechanism, thereby better isolating and highlighting the performance gains attributable primarily to our draft correction mechanism.

Furthermore, we observed that EAGLE compared the tree attention mechanism with chain-based approaches to evaluate the performance gain from tree attention. Such ablation studies are feasible for EAGLE due to two key characteristics:

1. All draft branches are generated by the draft model and verified within a single round, maintaining timing alignment comparable to other speculative sampling methods.
2. Building upon (1), the chain mechanism can be viewed as a top1 special case of tree attention, while tree attention allows flexible dynamic/static adjustment of topk configurations or tree topologies. In contrast, FOLD's early exit module for draft token correction introduces fundamentally distinct constraints:
3. Early exit branches undergo delayed validation (next round) temporally.
4. Early exit branches represent static multi-branch structures requiring integration of preliminarily verified tokens with draft tokens - structurally necessitating tree attention for subsequent sequence generation.

Official Review of Submission16674 by Reviewer RFZF

Official Review by Reviewer RFZF30 Oct 2025, 03:00 (modified: 12 Nov 2025, 11:22)EveryoneRevisions

Summary:

FOLD is a parallel speculative decoding framework. At the start of a given step, there are possible unverified tokens left over from a previous step. The draft model continues generation from these unverified tokens, while the target model with an early stopping module generates alternatives to the unverified tokens. Next, the full target model continues with verification of the unverified tokens while the draft model continues generation from its own branch (building on top of the unverified tokens) and branches from the early stopped target model (which are alternatives to the unverified tokens). When the full target model finishes verification, the branch with the most fully verified tokens is selected for the next round. Note that that branch will have some tokens not yet verified by the full model (i.e., those generated by the draft model in this round). The authors build FOLD into the PEARL speculative decoding framework with Kangaroo's early stopping module, and show speedups.

I had difficulty understanding the presentation of the method, so the above is a best-effort summary of the work. If I have misinterpreted the work, please let me know.

Soundness: 2: fair

Presentation: 1: poor

Contribution: 2: fair

Strengths:

- The core idea is an effective combination of Kangaroo's method of early stopping as a draft model, PEARL's method of parallelizing drafting and verification, and tree-based speculative decoding methods maintaining multiple drafts at once.
- Results are strong, showing reasonably consistent speedups compared to baselines across a variety of tasks.

Weaknesses:

The main weakness of the paper is its poor presentation, which makes it difficult to understand and evaluate. For example:

- Lines 58-64 in the intro (“Specifically, FOLD extends ... during Final Verify”) do little to explain what the four stages do and discuss concepts before they are introduced (e.g., what constitutes a “verification result”, where do branches come from, how are branches accepted).
- Figure 1 was confusing and did not help me understand the method. The legend was confusing (What’s a “correct wrong draft token?”), and I could not understand where each token came from, nor what the target tokens were. The caption was insufficient, and I did not find any references to the figure in the paper.
- The background was also insufficient. More discussion was needed of related works in Speculative decoding (e.g., the baselines presented in the results). More discussion of Pearl was also needed as it forms one of the core experiments of the paper (though that discussion could have been moved down to Section 4).
- Section 3.1, which describes how FOLD works, was very difficult for me to understand.
 - First, I think discussing the combination of Early Verify and Draft Correct when applying FOLD to Pearl (lines 126-129) this early only serves to confuse the reader.
 - The description of each stage is also unclear. The inputs, models used, methods applied, and outputs from each stage are not at all clear. For example, during Early Verify, it is not clear what tokens are being early verified. On my first read I assumed it was the output tokens from Early Draft, now I think it is unverified tokens from a previous step, but I am still not sure.
 - Figure 3 did not help either. On first read it is unclear how each token is obtained, how branches were built, why the target model re-generated input tokens, etc.
- Similar issues persisted into Section 4. In particular, there was essentially no explanation or discussion for Figure 4, which is a complicated figure key to understanding how FOLD was integrated into PEARL.
- More minor formatting issues:
 - The notations paragraph in the Background section is way earlier than it needs to be and confusingly refers to Early Draft and Draft Correct before they are introduced. Consider integrating it into section 3.1.
 - Also in that paragraph, “farward” should be “forward”.
 - Close quotes are used when open quotes should be used
 - Xu & McAuley, 2023a (line 97) and Xu & McAuley, 2023b (line 101) refer to the same paper, but are cited differently
 - In Line 160, refer to “Figure 3”, not just “3”.
 - Double comma in line 139 (“...draft tokens , , which means...”). These are just some of the presentation issues I had with the paper. I do not believe the paper is ready for publication in its current state. Significant editing and revising would be necessary to change my opinion.

And some comments on the results section:

- The authors characterize FOLD as a “training-free method” (line 339), but FOLD requires the training of the Early Exit Module. There is a training cost argument to make here the Early Exit Module may be much cheaper to train than a draft model, but training is still required.
- Authors should cite the benchmarks used.
- Experiments are restricted to Llama 3 and Llama 2. Results on models from other model providers would strengthen the results by demonstrating generalizability.
- No confidence intervals are provided for the speedup figures, making it difficult to contextualize the significance of the speedups.

Questions:

- Could you clarify the FOLD architecture in detail, specifying the inputs and outputs to each model at each stage, and across steps?
- What is the exact evaluation setup you used? Did all methods have access to the same computational resources?
- Related to this, it seems that FOLD would require more resources at any given time as both the draft model and target model need to run in parallel. Does FOLD actually end up using more GPU hours or does the faster generation offset this additional cost?

Flag For Ethics Review: No ethics review needed.

Rating: 2: reject, not good enough

Confidence: 4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

Code Of Conduct: Yes

Response to Reviewer RFZF with Question1(part 1/3)

Official Comment by Authors 21 Nov 2025, 19:34 (modified: 21 Nov 2025, 19:40) Everyone Revisions

Comment:

1. Clarification of the FOLD architecture, specifying the inputs and outputs at each stage and across steps.

Here are the figure that show the architecture <https://postimg.cc/w3RTnpVV> and pseudo code <https://i.postimg.cc/7PMcSdqn/FOLD-jia-gou-wei-dai-ma.png>. For pseudo code, we will incorporate it into the paper in the future.

FOLD consists of four main stages: **Early Draft**, **Early Verify**, **Draft Correct**, and **Final Verify**. I will detail the inputs, outputs, and roles of each stage below.

Let's assume we have an initial sequence to be verified, $\mathbf{x}$. We'll also assume the time required for a single inference step is in a 4:1 ratio for the target model to the draft model (i.e., the draft model can complete four inference steps in the time the target model completes one). Further, the time taken for the Early Exit module to produce an output is twice that of a single draft model inference step, as the early exit mechanism utilizes the hidden state from half of the target model's layers.

The workflow begins as follows:

1. **Stage 1: Early Draft & Early Verify (Parallel Execution)**
 - serves as the concurrent input for both the Early Draft and Early Verify stages.
 - **Early Draft:** The draft model takes $\mathbf{x}$ and autoregressively generates a draft token sequence $\mathbf{y}_d$. This takes two draft model time steps.
 - **Early Verify:** The target model's Early Exit module processes $\mathbf{x}$ and produces two outputs:
 - **Cached Hidden States (F):** A set of hidden states from the initial N layers of the target model, captured *before* the Early Exit block. will serve as the input for the Final Verify stage.
 - **Preliminary Logits (L):** A set of logits, $\mathbf{z}$, generated by the Early Exit module.
 - At this point, the two parallel processes synchronize, awaiting each other's completion to exchange and $\mathbf{y}_d$.
2. **Stage 2: Draft Correct & Final Verify (Parallel Execution)**
 - From the logits $\mathbf{z}$, we extract the top- k tokens for each position (e.g., for $\mathbf{z}_i$, we get $\mathbf{t}_i$).
 - $\mathbf{t}$. This forms a candidate set with a logical dimension of $k \times N$, which serves as a preliminary verification result for $\mathbf{y}_d$.
 - The sequences $\mathbf{y}_d$, $\mathbf{t}$, and the flattened tokens from $\mathbf{F}$ are concatenated to form $\mathbf{y}_c$. As illustrated in Figure 3, this is logically organized as a multi-branch structure.

- **Draft Correct:** This stage takes as input. Using an attention mask, it performs parallel inference to generate subsequent draft token sequences () for each branch. This is the **Tree Attention** mechanism shown in Figure 3 of our paper, a technique validated in prior work such as Medusa.
- **Final Verify:** This stage takes the cached hidden states as input and runs them through the remaining layers of the target model to produce the final, authoritative verification result,
- .

3. Final Verification and Path Selection

- The results are synchronized. The ground truth is used to validate the candidate paths formed by . The correct branch is selected, as shown in Figure 3.
- The subsequent draft tokens from the correct branch in will then be verified by the target model in the next forward pass. This ensures that the draft model's sequence to be verified always stays ahead of the target model's.

Response to Reviewer RFZF(part 2/3)

Official Comment by Authors 21 Nov 2025, 19:35 (modified: 21 Nov 2025, 19:37) Everyone Revisions

Comment:

2. What is the exact evaluation setup you used? Did all methods have access to the same computational resources?

Thank you for asking. Our setup was consistent for all model combinations and evaluations. All experiments were conducted using **4 x H100 GPUs**, and we set **top-k=4** for methods that require it. All methods had access to the exact same computational resources to ensure a fair comparison. If there are any other specific setup details you would like to know, please do not hesitate to ask, and I will be happy to provide them.

3. Does FOLD actually end up using more GPU hours?

Thank you for raising this important point. To clarify: (i) No, FOLD does not consume more GPU hours. (ii) Parallel execution does not necessarily require more GPUs.

If by 'GPU hours' you mean the product of inference duration and the number of GPUs, then our experiments do not result in higher GPU hours. All our experiments were conducted on the same hardware setup (4 x H100 GPUs), which is identical to the evaluation setup used in the PEARL paper. Therefore, in terms of results, the total GPU hours are directly comparable.

Regarding your question about whether parallel execution needs more GPUs, the multi-process parallelism is managed via the **accelerator**. In this setup, all allocated GPUs are visible and accessible to each process. Processes can efficiently share workloads on the same physical GPUs. This is supported by experiments in Appendix E of the PEARL paper, which discuss and validate the management of GPU resource contention in such setups.

4. About the *Weakness* section: The authors characterize FOLD as a “training-free method” (line 339), but FOLD requires the training of the Early Exit Module.

I apologize if my wording caused a misunderstanding. The term "training-free method" mentioned in line 339 refers to the **baselines** we compared against, not FOLD itself.

FOLD enhances the speculative decoding process by introducing a trainable Early Exit module, but it does not modify the original draft and target models. This module's primary role is to correct erroneous draft tokens, not to generate them. Under this condition, the ultimate speedup of any speculative sampling

method is still fundamentally determined by the consistency between the draft and target models. FOLD's contribution is to raise the theoretical performance ceiling for a *given* target model being accelerated.

Therefore, we concluded that comparing FOLD against "training-free" methods is fair because we are evaluating its ability to enhance a fixed model pairing. Furthermore, we did not shy away from comparing FOLD with methods that are not training-free. As shown in **Table 1**, we included the performance of [EAGLE2](#) as a reference and demonstrated that with a suitable model combination (e.g., Llama3-1B & 70B), FOLD can achieve a superior speedup.

Response to Reviewer RFZF(part 3/3)

[Official Comment](#)by Authors21 Nov 2025, 19:36Everyone

Comment:

5. Experiments are restricted to Llama 3 and Llama 2. Results on models from other model providers would strengthen the results by demonstrating generalizability.

We agree that demonstrating generalizability is crucial. In response, we have conducted additional experiments on the Vicuna model family, the results of which are available in our detailed response to **Reviewer R4Ma**. Despite the questionable quality of the Vicuna-68m model(as our response to Reviewer R4Ma in section [1. FOLD Scalability to 13B Models](#)), our results show that FOLD still provides a noticeable acceleration.

6. No confidence intervals are provided for the speedup figures, making it difficult to contextualize the significance of the speedups.

Thank you for this observation. As FOLD is a lossless speculative decoding acceleration method, we followed the convention established by nearly all prior work in this specific field, which typically does not report confidence intervals.

The speedup ratio itself is considered the most critical and central metric in this domain. It accurately reflects the consistency between the draft and target models under various scenarios. I would like to add that our reported speedup ratios are the average of multiple runs, and they provide a direct and intuitive measure of a method's superiority.

Replying to Response to Reviewer RFZF(part 3/3)

Official Comment by Reviewer RFZF

[Official Comment](#)by Reviewer RFZF24 Nov 2025, 22:15Everyone

Comment:

Thank you for the detailed response.

Clarifications [1-3]

I found the clarification of the FOLD architecture provided in your response to be much clearer than the description currently in the paper, and would recommend it be included in future revisions as well. The other clarifications are also helpful and should similarly be included.

Training-free Baselines [4]

The distinction drawn between the amount of training needed for the adapter in FOLD and methods like EAGLE could be made clearer. It would be good to baseline against methods that perform a comparable amount of training. For example, as you base the Early Exit module off of Kangaroo (Liu et al., 2024), that would be a good baseline. Further, I concur with Reviewer R4MA that comparisons with tree-based speculative decoding methods are necessary.

Other Model Families [5]

Thank you for your results on the Vicuna model family. I agree these results are evidence of FOLD's generalizability. If able, I encourage the authors to additionally evaluate on the Qwen 3 model family, as

the wider availability of model sizes can help address the authors' concerns about the weakness of the draft model.

Confidence Intervals [6]

While related work tends not to include confidence intervals, their inclusion would make for a stronger paper. Given that you are already reporting averages over multiple runs, hopefully further reporting the number of runs and/or the variance over those runs will not be too difficult.

Though the response addresses most of my concerns with the presentation of the work, given the scale of the revisions needed to integrate the clarifications, as well as my other concerns, I will keep my score.

[About OpenReview](#)

[Contact](#)

[FAQ](#)

[Hosting a Venue](#)

[Sponsors](#)

[Terms of Use / Privacy Policy](#)

[All Venues](#)

[Donate](#)

[News](#)

OpenReview is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026 OpenReview

FOLD: Fast Correct Speculative Decoding | OpenReview

[OpenReview.net](#)

[Login](#)



[Go to ICLR 2026 Conference homepage](#)

Cactus: Accelerating Auto-Regressive Decoding with Constrained Acceptance Speculative Sampling



Yongchang Hao, Lili Mou

Published: 26 Jan 2026, Last Modified: 11 Apr 2026 | ICLR 2026 Poster | EveryoneRevisionsBibTeXCC BY 4.0

Keywords: Large language models, Speculative sampling, Auto-regressive generation

TL;DR: We propose Cactus, a speculative sampling method that guarantees controlled divergence from the verifier distribution while increasing throughputs.

Abstract:

Speculative sampling (SpS) has been successful in accelerating the decoding throughput of auto-regressive large language models by leveraging smaller draft models. SpS strictly enforces the generated distribution to match that of the verifier LLM. This is unnecessarily restrictive as slight variations of the verifier's distribution, such as sampling with top- or temperature, would also be acceptable. Typical acceptance sampling (TAS) alleviates this issue by accepting more tokens using entropy-based heuristics. However, this approach distorts the verifier distribution, potentially

degrading output quality when the verifier encodes critical information. In this work, we formalize the speculative sampling algorithm through the lens of constrained optimization. Based on this formulation, we propose Cactus (**constrained acceptance speculative sampling**), a method that guarantees controlled divergence from the verifier distribution and increasing acceptance rates. Empirical results across a wide range of benchmarks confirm the effectiveness of our approach.

Primary Area: foundation or frontier models, including LLMs

Submission Number: 13094

Filter by reply type...

Filter by author...

Sort: Newest FirstSort: Oldest First

16 / 16 replies shown

Add:

Paper Decision

Decisionby Program Chairs26 Jan 2026, 14:13 (modified: 06 Feb 2026, 10:57)EveryoneRevisions

Decision: Accept (Poster)

Add:

Meta Review of Submission13094 by Area Chair ooug

Meta Reviewby Area Chair ooug10 Jan 2026, 09:26 (modified: 09 Feb 2026, 12:12)EveryoneRevisions

Summary:

The paper studies approximate speculative decoding (SD) through relaxing the acceptance criteria by solving a divergence constrained optimization problem. The paper complements the principled motivation with empirical evidence on a diverse set of benchmarks showing improved latency with neural quality on most datasets.

Modulo one reviewer with low-quality feedback, there is a positive consensus among the reviewers about the principled formulation of the problem and good empirical evidence. The reviewer raised concerns about the extensiveness of the experiments regarding the experimented models and benchmark dataset, as well as the robustness analysis on the hyperparameter choice, which are mostly addressed in the author's rebuttal.

One concern that the authors failed to address is that the paper doesn't contain experiments on more recent SD techniques such as EAGLE, which I think it is a nice experiment to have. However, I don't see this as a blocker for the paper's acceptance.

Overall, I would recommend the paper for acceptance. The authors should include the reviewers' suggestions to improve their final draft.

Reviewer Concerns:

See above discussion.

Reviewer Scores:

I think most reviewers will maintain their positive rating of the paper.

Add:

Official Review of Submission13094 by Reviewer sKn6

Official Reviewby Reviewer sKn604 Nov 2025, 15:29 (modified: 12 Nov 2025, 10:33)EveryoneRevisions

Summary:

This paper proposes CACTUS, a novel constrained acceptance speculative sampling algorithm that accelerates large language model decoding while maintaining output fidelity. The key idea is to cast speculative sampling as a constrained optimization problem that explicitly trades off divergence from the verifier distribution against acceptance rate. By leveraging an f-divergence–bounded objective, the authors derive a closed-form solution (with KL divergence) that ensures provably controlled deviation from the

verifier while allowing more draft tokens to be accepted. Extensive experiments on multiple model families (Qwen, Gemma, R1, LLaMA) and benchmarks (GSM8K, IFEval, GPQA) show that CACTUS consistently improves throughput and acceptance length without hurting accuracy. Compared with typical acceptance sampling (TAS), it demonstrates both higher acceptance rates and more stable quality, validating its theoretical motivation.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

The paper is well-written. The proposed approach is novel. The evaluation is comprehensive and analysis is in-depth.

Weaknesses:

1. The observed improvements of CACTUS appear to correlate strongly with the capability of the underlying LLMs. For instance, larger verifiers (e.g., 14B) consistently outperform smaller ones (e.g., 8B), and reasoning-tuned models achieve higher acceptance and throughput than non-reasoning counterparts. This raises the question of whether the proposed method inherently benefits from the stronger self-consistency and error-correction capacity of powerful models, rather than from the constrained optimization formulation itself. A more detailed analysis or discussion of this correlation would strengthen the empirical claims.
2. The evaluation primarily reports acceptance length and rejection ratio as proxies for throughput, but omits key metrics such as average generation length or tokens per completion. Without these, it is difficult to fully interpret both the quality improvements and the real speedup.

Questions:

See above

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

[Official Comment](#) by Authors 19 Nov 2025, 08:34 Everyone

Comment:

We thank the reviewer for the thoughtful feedback and positive assessment of our work. We're encouraged that you say our paper is "well-written", our "proposed approach is novel", and our "evaluation is comprehensive".

The observed improvements of CACTUS appear to correlate strongly with the capability of the underlying LLMs. For instance, larger verifiers (e.g., 14B) consistently outperform smaller ones (e.g., 8B), and reasoning-tuned models achieve higher acceptance and throughput than non-reasoning counterparts. This raises the question of whether the proposed method inherently benefits from the stronger self-consistency and error-correction capacity of powerful models, rather than from the constrained optimization formulation itself. A more detailed analysis or discussion of this correlation would strengthen the empirical claims. We appreciate the reviewer's observation on the correlation between LLM capabilities and Cactus performance. This is expected because Cactus is designed to retain the performance of the verifier model with our constrained optimization framework. The correlation in fact confirms the effectiveness of our method.

On the other hand, the acceptance rate could be higher when the draft and verifier models are more similar for all methods (including Cactus). We additionally conduct the following results to confirm that our method works across different selections of draft-verifier pairs:

Draft size	Score↑	AL↑	Rej↓
Verifier	91.71	-	-
0.6B	93.10	5.44	-32%
1.7B	92.50	6.78	-60%
4B	92.57	7.76	-76%

Here, we use Qwen 3 14B as the verifier with different sizes of draft models from 0.6B to 4B with on GSM8K. The results show that using a larger draft model substantially improves the average acceptance length and reduces the token rejection rate. Importantly, our method consistently outperforms the naive decoding baseline across all configurations in terms of efficiency. These findings suggest that Cactus is robust and compatible with a wide range of draft model sizes. We will incorporate the complete results with other datasets in the revised manuscript.

The evaluation primarily reports acceptance length and rejection ratio as proxies for throughput, but omits key metrics such as average generation length or tokens per completion. Without these, it is difficult to fully interpret both the quality improvements and the real speedup. We thank the reviewer for asking the question. We provide additional results on average generation length (counting CoT thinking tokens) on each task as below

Method	GSM8K	IFEval	GPQA
SpS	588.2	585.3	2484.6
TAS	591.3	670.2	2561.3
Cactus	608.1	632.3	2544.4

Here, we used the 0.6B+14B model pair. We see that all methods have a similar generation length, which is expected because our method is designed to have a similar distribution. This again shows the effectiveness of our method.

In addition, the generation length is not a metric of efficiency, given that different tasks require different lengths to complete (e.g. GPQA completions are much longer). In this paper, we mainly reported average acceptance length, rejected tokens, and throughput (wallclock time). They comprehensively evaluate the generation efficiency from different perspectives. As shown in Table 1 and Figure 3, Cactus consistently outperforms baselines in different aspects of efficiency.

Summary

We again thank the reviewer for the constructive comments and careful evaluation of our submission. In this response, we have (1) analyzed the interaction between verifier/draft capability and Cactus with new

experiments across draft sizes, and (2) reported average generation lengths while clarifying why acceptance length, rejected tokens, and wallclock throughput are used as efficiency indicators. We hope these clarifications address the concerns, and we would be grateful if the reviewer could raise the score in light of our responses. Thank you for your time and support!

Add:

Official Review of Submission13094 by Reviewer 6cgh

Official Review by Reviewer 6cgh03 Nov 2025, 21:11 (modified: 12 Nov 2025, 10:33)EveryoneRevisions

Summary:

This paper introduces Cactus, a novel speculative sampling method for accelerating auto-regressive decoding in large language models (LLMs). Building on prior work like vanilla speculative sampling (SpS) and typical acceptance sampling (TAS), the authors reformulate speculative sampling as a constrained optimization problem to trade off acceptance rates against divergence from the verifier model's distribution. They derive a closed-form approximation using KL divergence and provide theoretical guarantees on bounded divergence. Empirical evaluations on benchmarks like GSM8K, IFEval, and GPQA using models from Qwen, Gemma, R1, and LLaMA series show that Cactus achieves higher acceptance rates and throughput than baselines while preserving or improving output quality.

Soundness: 2: fair

Presentation: 3: good

Contribution: 2: fair

Strengths:

1. The paper is well-written and highly readable, with the authors providing a detailed introduction to the proposed method, along with in-depth theoretical contributions and proofs.
2. The "training-free" nature and minimal computational overhead of Cactus make it extremely easy to integrate into existing inference frameworks, endowing it with high practical value.
3. The experiments cover a variety of benchmark datasets as well as multiple model families and sizes. The results consistently show that Cactus outperforms SpS and TAS in terms of acceptance length and rejection rate, while maintaining stable accuracy.

Weaknesses:

1. It only includes three types of datasets: mathematical reasoning, instruction following, and scientific question answering, and lacks experimental results on Spec-Bench [1]—a more commonly used benchmark for speculative decoding. Spec-Bench covers multiple tasks such as multi-turn dialogue, translation, mathematical reasoning and so on.
2. The experiments only cover small-to-medium-sized models (with a maximum of 14B parameters) and do not involve larger models (e.g., 70B parameters) where memory bottlenecks are more prominent.
3. The baselines are relatively outdated. Key baseline methods such as Medusa [2] and EAGLE-2/3 [3, 4] are not comprehensively compared in the main results.
4. The code in the anonymous link (<https://anonymous.4open.science/r/Cactus-2E4D/>) is empty.

[1] Xia H, Yang Z, Dong Q, et al. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding [C]. Findings of the Association for Computational Linguistics ACL 2024. 2024: 7655-7671.

[2] Cai T, Li Y, Geng Z, et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads [C]. International Conference on Machine Learning. PMLR, 2024: 5209-5235.

[3] Li Y, Wei F, Zhang C, et al. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees [C]. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 7421-7432.

[4] Li Y, Wei F, Zhang C, et al. Eagle-3: Scaling up inference acceleration of large language models via training-time test [J]. arXiv preprint arXiv:2503.01840, 2025.

Questions:

1. How sensitive is Cactus to δ ? Can the authors provide ablation experiments?
2. How to select the optimal δ ? And how should δ be chosen for different tasks?

Flag For Ethics Review: No ethics review needed.

Rating: 4: marginally below the acceptance threshold. But would not mind if paper is accepted

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 08:35 Everyone

Comment:

We thank the reviewer for the careful and constructive assessment. We are glad that the reviewer appreciated the training-free design and high practical value of Cactus.

It only includes three types of datasets: mathematical reasoning, instruction following, and scientific question answering, and lacks experimental results on Spec-Bench [1]—a more commonly used benchmark for speculative decoding. Spec-Bench covers multiple tasks such as multi-turn dialogue, translation, mathematical reasoning and so on.

We thank the reviewer for suggesting SpecBench. The benchmarks in our paper (GSM8K, IFEval, and GPQA diamond) are extensively used to evaluate modern LLMs [1, 2, 3]. In fact, SpecBench (suggested by the reviewer) partially overlaps with our evaluation as it samples from GSM8K.

That being said, we're happy to conduct additional experiments on SpecBench, but it might take additional time for us to implement our method following the SpecBench's protocol. We would report back once we gather the results.

The experiments only cover small-to-medium-sized models (with a maximum of 14B parameters) and do not involve larger models (e.g., 70B parameters) where memory bottlenecks are more prominent.

Thanks for the suggestion. Again, conducting experiments on large-scale LLMs would take more time. We're running a 32B Qwen 3 model at this moment (as 70B is too big for us). We'll update the results when we have them.

The baselines are relatively outdated. Key baseline methods such as Medusa [2] and EAGLE-2/3 [3, 4] are not comprehensively compared in the main results.

We would like to clarify that Medusa and Eagle series are trained models, whereas our proposed method is an acceptance criterion without any training. In the paper, we have discussed this difference in Section 4. In our experiments, extensively compared with SpS and TAS, the underlying acceptance criteria used in Eagles and Medusa models. Given the scope of this paper, we leave the extension of Cactus to trainable tree-like attention modules to future work.

The code in the anonymous link (<https://anonymous.4open.science/r/Cactus-2E4D/>) is empty.

We apologize for the inconvenience. We have resolved the technical glitch and the repo should be viewable now.

How sensitive is Cactus to δ ? Can the authors provide ablation experiments? How to select the optimal δ ? And how should δ be chosen for different tasks?

Thanks for asking the questions. In Section 3.3, we conducted sweeping from 1 to 1e6. We empirically observe that usually yields stable text quality while largely accelerating the inference speed.

Summary

We again thank the reviewer for the thoughtful feedback and for pointing out several directions to further strengthen the work. We have clarified on every concern raised by the reviewer, and will report additional experiments once we gather the results.

- [1] Qwen3 Technical Report. arXiv 2505.09388.
- [2] The Llama 3 Herd of Models. arXiv 2407.21783.
- [3] DeepSeek-V3 Technical Report. arXiv 2412.19437.

Add:
Replying to Official Comment by Authors

Follow-up response with additional experiments

Official Commentby Authors25 Nov 2025, 11:24Everyone

Comment:

It only includes three types of datasets: mathematical reasoning, instruction following, and scientific question answering, and lacks experimental results on Spec-Bench [1]—a more commonly used benchmark for speculative decoding. Spec-Bench covers multiple tasks such as multi-turn dialogue, translation, mathematical reasoning and so on.

Following our previous response, we have now gathered the experiment results on Spec Bench as below.

	MT Bench	Translatio n	Summarizatio n	QA	Math	RAG	Accepte d	Overall I
SpS	2.01x	1.40x	1.92x	1.85 x	1.83 x	1.86 x	3.20	1.81x
Ours ()	2.09x	1.40x	2.04x	1.95 x	1.86	1.92 x	3.29	1.88x

In this experiment, the model combination is Qwen3 0.6B + 14B with a temperature of 0.6. Cactus is benchmarked without hyper-parameter tuning (i.e.,). It is evident that our method provides immediate acceleration over SpS without any training required. Across different domains, Cactus is consistently faster than SpS. Overall, Cactus achieves a +88% speed gain over the baseline, significantly reducing compute costs. Note that the speeds above are measured using the HuggingFace transformers framework, which is less optimized. The real-world performance gain could be even larger with more optimization (as justified by our other experiments using VLLM).

The experiments only cover small-to-medium-sized models (with a maximum of 14B parameters) and do not involve larger models (e.g., 70B parameters) where memory bottlenecks are more prominent. Following our previous response, we have also finished the experiments with larger models.

	GSM8K	GSM8K	IFEval	IFEval	GPQA	GPQA
	Score↑	AL↑	Score↑	AL↑	Score↑	AL↑
SpS	95.30	5.03	83.36	2.61	40.40	3.73
TAS	94.10	7.02	83.73	4.16	40.40	6.12

Ours ()	94.40	7.13	85.21	4.47	41.92	6.36
---------	-------	------	-------	------	-------	------

These experiments were conducted using the Qwen3 1.7B + 32B model pair. As shown in the table, Cactus not only has the longest acceptance length (AL) but also achieves the highest scores across 2 out of 3 tasks. This again confirms that the effectiveness of Cactus holds beyond smaller models and remains consistent even when both the drafter and the verifier are significantly larger.

Summary

We have completed the additional experiments as promised in our previous response. The results are now included in the appendix with proper citations. We believe these results provide strong evidence of the effectiveness of Cactus. We hope that all previous concerns have now been addressed, and we respectfully invite the reviewer to raise the score in light of our response. Please do not hesitate to let us know if you have any further questions. Thank you for your time and effort!

Add:

Official Review of Submission13094 by Reviewer Qb8d

Official Reviewby Reviewer Qb8d03 Nov 2025, 12:43 (modified: 12 Nov 2025, 10:33)EveryoneRevisions

Summary:

The core idea—casting speculative decoding as a constrained optimization and applying a KL-divergence bound with a second-order Taylor approximation—appears as an incremental re-interpretation of existing acceptance strategies rather than a fundamentally new algorithmic principle. Theoretical development largely repackages known mechanisms, while empirical gains emphasize acceptance/throughput and depend heavily on the hyperparameter δ ; accuracy robustness improvements are inconsistent across tasks. Overall, the contribution does not clear the bar for novelty and impact.

The manuscript exhibits noticeable typesetting glitches (e.g., unnecessary hyphenation and line breaks in the title/long tokens, inconsistent styles for algorithms/tables/captions, and minor inconsistencies in notation and reference formatting. These issues hurt readability and professional polish; a thorough pass against the official template is needed.

Soundness: 2: fair

Presentation: 2: fair

Contribution: 2: fair

Strengths:

The work introduced a principled, divergence-controlled acceptance rule (Cactus) that balances fidelity and efficienc.

Weaknesses:

The manuscript exhibits noticeable typesetting glitches (e.g., unnecessary hyphenation and line breaks in the title/long tokens, inconsistent styles for algorithms/tables/captions, and minor inconsistencies in notation and reference formatting. These issues hurt readability and professional polish; a thorough pass against the official template is needed.

Questions:

The manuscript exhibits noticeable typesetting glitches (e.g., unnecessary hyphenation and line breaks in the title/long tokens, inconsistent styles for algorithms/tables/captions, and minor inconsistencies in notation and reference formatting. These issues hurt readability and professional polish; a thorough pass against the official template is needed.

Flag For Ethics Review: No ethics review needed.

Rating: 2: reject, not good enough

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

Official Comment by Authors 19 Nov 2025, 08:36 Everyone

Comment:

Thanks for the comments.

The manuscript exhibits noticeable typesetting glitches (e.g., unnecessary hyphenation and line breaks in the title/long tokens, inconsistent styles for algorithms/tables/captions, and minor inconsistencies in notation and reference formatting. These issues hurt readability and professional polish; a thorough pass against the official template is needed.

Our template strictly follows the ICLR 2026 template. The hyphenation and line breaks are automatically generated by the LaTeX style file. On the contrary, all other reviewers rate our presentation score as 3. Reviewers sKn6 and 6cgh explicitly highlighted that our paper is well-written and highly readable. That being said, the formatting style does not hurt our central scientific contributions. **We urge the reviewer to revisit our paper and reassess our work accordingly.**

Add:

Official Review of Submission13094 by Reviewer CzSq

Official Review by Reviewer CzSq 01 Nov 2025, 04:36 (modified: 12 Nov 2025, 10:33) Everyone Revisions

Summary:

This paper proposes Cactus, a novel speculative sampling method that provides explicit control over divergence from the verifier's distribution to improve the token acceptance rate. It formulates speculative sampling as a constrained optimization problem that maximizes the draft token acceptance probability while keeping the target distribution within a bounded divergence from the verifier. Empirical results across multiple models and datasets show that Cactus achieves higher acceptance rates, often with improved accuracy compared to baseline methods.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

1. Proposes principled trade-off between efficiency and fidelity. The paper proposes viewing speculative sampling as choosing an appropriate distribution that achieves a desired trade-off between draft token acceptance rate and divergence from the verifier. This view allows methods including SpS and TAS to be viewed in a unified framework.
2. Practical implementation. Based on the constrained optimization formulation, the paper derives a closed-form solution with cost comparable to SpS.
3. Empirical evaluations. Experiments on multiple models and datasets show improved acceptance length compared to SpS and TAS.

Weaknesses:

1. Missing evaluations on drafter quality. Qwen 3 models are used as both the verifier and the drafter. Since these models belong to the same family, their output distributions are likely to be similar, making it unclear how the results would change with different levels of inherent divergence between the verifier and drafter. Also, how does performance vary when using drafters of different sizes?
2. Missing evaluations on interactions with temperature. For the main experiments, one set values for top-p, top-k, and temperature (fairly low) is used. Further analysis on how different methods

interact with sampling hyperparameters would be useful since the focus of the paper is on speculative .

3. Discussion on approximation validity. Cacus leverages second-order Taylor approximation assuming remains small. It is unclear whether values such as 0.75 justifies the approximation.

Questions:

Q. What are some possible explanations for the often better accuracies?

Q. The anonymous link to code seems broken?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

Code Of Conduct: Yes

Add:

Official Comment by Authors

[Official Comment](#) by Authors 19 Nov 2025, 08:42 (modified: 19 Nov 2025, 08:44) Everyone [Revisions](#)

Comment:

We are grateful to the reviewer for the careful reading and encouraging assessment. We are pleased that you found our method principled and recognize the practical implementation of Cactus.

Missing evaluations on drafter quality. Qwen 3 models are used as both the verifier and the drafter. Since these models belong to the same family, their output distributions are likely to be similar, making it unclear how the results would change with different levels of inherent divergence between the verifier and drafter. Also, how does performance vary when using drafters of different sizes?

In terms of model family, speculative sampling methods typically use the drafter and verifier from the same family [1, 2, 3], which is also our setup. This is because the draft and verifier models should have the same tokenization to evaluate the probability of the tokens. Mapping different tokenizations is an interesting research direction on its own, which is beyond the scope of our paper.

In terms of different draft model sizes, we provide the additional results as below

Draft size	Score↑	AL↑	Rej↓
Verifier	91.71	-	-
0.6B	93.10	5.44	-32%
1.7B	92.50	6.78	-60%
4B	92.57	7.76	-76%

Here, we use Qwen 3 14B as the verifier with different sizes of draft models from 0.6B to 4B with on GSM8K. The results show that using a larger draft model substantially improves the average acceptance length and reduces the token rejection rate. Importantly, our method consistently outperforms the naive decoding baseline across all configurations. These findings suggest that Cactus is robust and compatible with a wide range of draft model sizes. We will incorporate the complete results on other datasets in the revised manuscript.

Missing evaluations on interactions with temperature. For the main experiments, one set values for top-p, top-k, and temperature (fairly low) is used. Further analysis on how different methods interact with sampling hyperparameters would be useful since the focus of the paper is on speculative . Discussion on approximation validity. Cacus leverages second-order Taylor approximation assuming remains small. It is unclear whether values such as 0.75 justifies the approximation.

In the paper, we have provided results without top- and top- in Figure 2. The temperature is set to 1. This allows us to demonstrate the quality-efficiency tradeoff by setting different values. We hope the ablation study answers the reviewer’s question.

Q. What are some possible explanations for the often better accuracies?

Thanks for asking the question. We hypothesize that it’s an ensemble effect, where a “healthy” mix of two models could perform better than one model. We’ve provided additional discussion in Appendix C in the revision.

Q. The anonymous link to code seems broken?

We apologize for the issue. We’ve fixed it and our code is available at <https://anonymous.4open.science/r/Cactus-2E4D/>

Summary

Thank you again for the thoughtful comments and for pointing out several ways to strengthen our work. In the response, we’ve addressed all the points including conducting additional experiments. We hope our response addresses the reviewer’s concerns.

Add:

Official Review of Submission13094 by Reviewer PuX9

Official Reviewby Reviewer PuX931 Oct 2025, 04:05 (modified: 12 Nov 2025, 10:33)EveryoneRevisions

Summary:

This paper introduces Cactus, a constrained acceptance speculative sampling method aimed at accelerating auto-regressive decoding in large language models (LLMs) while preserving output distributional fidelity. The work frames speculative sampling as a constrained optimization problem balancing acceptance rate and divergence from a verifier model. Theoretical analysis yields a principled solution for constrained divergence (using KL divergence as a metric), and the authors propose a practical algorithm that maximizes accepted tokens per verification step. Empirical evaluation on multiple benchmarks (GSM8K, IFEval, and GPQA), using various pairs of drafting and verifying models, demonstrates improved throughput and competitive or superior accuracy versus classical speculative sampling and typical acceptance sampling approaches.

Soundness: 3: good

Presentation: 3: good

Contribution: 3: good

Strengths:

1. Formalization and Theoretical Rigor: The paper provides a thorough formalization of speculative sampling as a constrained optimization problem, with mathematically sound derivations (see Equations in Section 2 and supporting proofs in Appendix A). The use of -divergences, primarily KL, is well-motivated and leads to explicit solutions for the acceptance/recovery functions.
2. Principled Design: Compared to prior heuristics like typical acceptance sampling (TAS), Cactus mathematically guarantees bounded divergence from the verifier distribution. The method is grounded in strong theoretical justifications (see Theorem 3, Corollaries 6 and 7, and explicit treatment of the divergence-acceptance trade-off).
3. Empirical Breadth and Depth: The experiments include a wide variety of benchmarks, LLM architectures, and model sizes. Table 1 provides detailed results across tasks and models,

showing Cactus consistently offers higher acceptance rates (e.g., and Rej columns), often without sacrificing accuracy compared to SpS and TAS. Additional evaluations address wall-time throughput (Figure 3), cross-architecture robustness (Figure 4), and trade-offs with alternative approaches (interpolation and SpecCas).

4. **Clarity and Transparency:** Mathematical notation is mostly clear, with comprehensive appendices containing proofs and derivations. The design of the method is explained with both equations and pseudocode (Algorithm 1), and decision points are clearly highlighted.
5. **Practical Impact:** The method is training-free, requires only elementwise operations on output probabilities, and can be deployed to accelerate LLM inference in real-world conditions.
6. **Figure/Table Engagement:** Key quantitative claims are backed up by explicit references to figures and tables. For example, Table 1a and 1b directly show improvement of Cactus in both acceptance rate and quality, while Figure 1 graphically summarizes the accuracy-acceptance tradeoff across diverse settings.

Weaknesses:

1. **Empirical Baseline Coverage and Ablations:** While the paper includes SpS, TAS, Top-k, Mentored decoding, and SpecCas as baselines, it does not systematically evaluate against very recent tree-based decoding advancements (such as recursive or blockwise candidate strategies). This limits direct benchmarking of Cactus's relative value in the face of alternative parallel or hierarchical sampling frameworks. The omission can be seen in the lack of explicit mention of, e.g., "Recursive Speculative Decoding: Accelerating LLM Inference via Sampling Without Replacement" or similar tree/tree-of-N approaches, which could address the same efficiency-quality tradeoff. Including these could strengthen the empirical section.
2. **Scope of Generalization Claims:** Although Figure 4 and several model series are included, experiments mostly focus on models up to 14B parameters and a limited set of benchmarks. Claims about scalability to substantially larger backbones (hundreds of billions of parameters) or broader domains (e.g., multilingual or multimodal tasks) remain unproven, and the discussion in Appendix C defers these questions to future work.
3. **Selection and Tuning of (Divergence Constraint):** The hyperparameter critically determines the quality-throughput tradeoff. While the mechanism for grid search and choice is described, no principled or automated selection strategy is suggested, nor is the sensitivity to analyzed in detail beyond grid-search plots (see Figure 2). This may be a practical barrier for deployment, especially for practitioners unfamiliar with the KL/gamma tradeoff mechanics.

Questions:

1. The method currently relies on manual or grid-based tuning for the divergence parameter . Is there a theoretically principled or adaptive way to set or anneal based on runtime feedback (e.g., based on target throughput or accuracy)? What would be the pitfalls in automating this choice?
2. While Cactus aims to bound divergence and preserve verifier fidelity, do any empirical failure cases show semantic or correctness failures? For example, what types of errors arise in GSM8K or GPQA when is set too high?
3. How would the approach scale to extremely large verifiers (e.g., >100B parameters) with draft models that are structurally or statistically misaligned? Does the performance gain or accuracy gap increase as model size scales up?
4. Related to Figure 1 and Table 1: In specific cases (e.g., GPQA with Cactus 1.0), Cactus surpasses the verifier's accuracy. Is this genuine improvement or due to variance/noise in evaluation or artifacts of metric normalization?
5. Could the framework be generalized to allow for multi-token acceptance rules (e.g., chains or blocks of tokens) beyond per-token KL constraint, and what would the mathematical/algorithmic implications be?

Flag For Ethics Review: No ethics review needed.

Rating: 6: marginally above the acceptance threshold. But would not mind if paper is rejected

Confidence: 5: You are absolutely certain about your assessment. You are very familiar with the related work and checked the math/other details carefully.

Code Of Conduct: Yes

Add:

Official Comment by Authors (1/3)

Official Comment by Authors 19 Nov 2025, 08:46 Everyone

Comment:

We thank the reviewer for highlighting the theoretical rigor and practical impact of our method.

Empirical Baseline Coverage and Ablations: While the paper includes SpS, TAS, Top-k, Mentored decoding, and SpecCas as baselines, it does not systematically evaluate against very recent tree-based decoding advancements (such as recursive or blockwise candidate strategies). This limits direct benchmarking of Cactus's relative value in the face of alternative parallel or hierarchical sampling frameworks. The omission can be seen in the lack of explicit mention of, e.g., "Recursive Speculative Decoding: Accelerating LLM Inference via Sampling Without Replacement" or similar tree/tree-of-N approaches, which could address the same efficiency-quality tradeoff. Including these could strengthen the empirical section.

Thanks for the comment. The scope of our paper mainly focuses on proposing a new acceptance criterion as an alternative to SpS or TAS, which was used as the underlying acceptance function in the recursive accepting phase of the suggested paper. As a result, we believe Cactus can be naturally extended to tree-based decoding with recursion or blocks. We have added the suggested work and enhanced the discussion on this in the related work section .

Scope of Generalization Claims: Although Figure 4 and several model series are included, experiments mostly focus on models up to 14B parameters and a limited set of benchmarks. Claims about scalability to substantially larger backbones (hundreds of billions of parameters) or broader domains (e.g., multilingual or multimodal tasks) remain unproven, and the discussion in Appendix C defers these questions to future work. How would the approach scale to extremely large verifiers (e.g., >100B parameters) with draft models that are structurally or statistically misaligned? Does the performance gain or accuracy gap increase as model size scales up?

Thanks for the suggestion. We would like to conduct further experiments with large models (up to 32B due to limited resources) on more benchmarks. However, these experiments would take more time. We would report back once we gather the results.

Selection and Tuning of (Divergence Constraint): The hyperparameter critically determines the quality-throughput tradeoff. While the mechanism for grid search and choice is described, no principled or automated selection strategy is suggested, nor is the sensitivity to analyzed in detail beyond grid-search plots (see Figure 2). This may be a practical barrier for deployment, especially for practitioners unfamiliar with the KL/gamma tradeoff mechanics. The method currently relies on manual or grid-based tuning for the divergence parameter . Is there a theoretically principled or adaptive way to set or anneal based on runtime feedback (e.g., based on target throughput or accuracy)? What would be the pitfalls in automating this choice?

Thanks for the insightful comment. Given the nature of quality--efficiency trade-off, the suitable largely depends on what users prioritize. However, Figure 2 shows that it is not a sensitive hyperparameter, and a range of 0-10 works generally fine. Based on our experiments across different models and tasks, we find that choosing largely maintains the performance of the verifier model while accelerating the decoding.

Add:

Replying to Official Comment by Authors (1/3)

Official Comment by Authors (2/3)

Official Comment by Authors 19 Nov 2025, 08:48 Everyone

Comment:

While Cactus aims to bound divergence and preserve verifier fidelity, do any empirical failure cases show semantic or correctness failures? For example, what types of errors arise in GSM8K or GPQA when is set too high?

Yes. We observed that the performance drops when is large, especially in that the reasoning makes less sense. Here is a case study:

Question

Question: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

<think>\nOkay, let me try to figure out how much Janet makes each day at the farmers' market. Hmm, so the problem says she has ducks that lay 16 eggs per day. Then she eats three for breakfast and bakes muffins with four eggs each day. Then she sells the remaining eggs for \$2 each. Alright, let's break this down step by step.\n\nFirst, total eggs per day: 16 eggs. Then she uses some for breakfast and baking. Let's see, for breakfast, she eats 3 eggs. Then for baking, she uses 4 eggs. So total eggs used each day would be $3 + 4 = 7$ eggs. Therefore, the remaining eggs would be $16 - 7 = 9$ eggs. Then she sells those 9 eggs at \$2 each. So, how much does she make? 9 eggs multiplied by \$2, which is 18 dollars.\n\nWait, let me check again. If she uses 3 eggs for breakfast and 4 for baking, that's 7. 16 minus 7 is indeed 9. Then 9 times 2 is 18. Yeah, that seems right. So her daily earnings would be \$18. Hmm, is there any possibility I made a mistake here? Maybe I should re-read the problem to make sure I didn't miss anything.\n\nThe problem says she lays 16 eggs per day. Then she eats 3 for breakfast and bakes muffins each day with four. So the total used is $3+4=7$. Subtract that from 16, which leaves 9. Multiply by \$2 for selling, $9*2=18$. Yep, that's correct. I think that's the answer.\n</think>\n\nJanet's ducks lay 16 eggs per day. She uses 3 eggs for breakfast and 4 eggs for baking muffins, totaling 7 eggs used. The remaining eggs are $16 - 7 = 9$ \$. She sells these at \$2 per egg, earning $9 \times 2 = 18$ \$. \n\n#### 18

<think>\nOkay, so let's try to figure out how much Janet makes every day at the farmers' market. Let me start by breaking down the problem step by step. [lengthy CoT omitted] so total eggs used is $4 * 4 = 16$. Then, after eating 3 for breakfast, leftover is $16 - 3 = 13$, then she sells them at 2 each: $13 * 2 = \$26$. \n\nYes, that seems to make sense. Therefore, the answer should be 26. But let me check if the question is asking "how much money does she make every day at the farmers' market?" So yes, selling the remainder at \$2 per egg. Therefore, answer is \$26. \n\nTherefore, the answer is 26, which in the previous problems they had 5. This is a different case. So I think that's correct. \n</think>\n\nAnswer: 26\n\nThe problem states that Janet's ducks lay 16 eggs per day. She eats 3 eggs for breakfast and 4 eggs to make muffins daily. After these expenses, the remaining eggs are $16 - 3 - 4 = 9$ \$, which are sold at \$2 per egg. Therefore, the total money she earns at the farmers' market is $9 \times 2 = 18$ \$. \n\nHowever, the phrase "bakes muffins for her friends every day with four" might imply that she uses 4 eggs per muffin, meaning she makes 4 muffins. If she uses 4 eggs each muffin, the total eggs consumed would be $4 \times 4 = 16$ \$. Since she eats 3, the remaining eggs are $16 - 3 = 13$ \$, which are sold for $13 \times 2 = 26$ \$. Hence, the correct answer is **26**.\n\n#### 26

From the above case study, we can see the reasoning is poor and lengthy when is large (more divergence allowed). Consequently, the result is wrong due to the low-quality chain-of-thought. We've presented the case study in the revision.

Related to Figure 1 and Table 1: In specific cases (e.g., GPQA with Cactus 1.0), Cactus surpasses the verifier's accuracy. Is this genuine improvement or due to variance/noise in evaluation or artifacts of metric normalization?

We believe that it is a genuine improvement because (1) Cactus is often better by 2 sigma, and (2) the improvement is consistent across runs. Our hypothesis is that Cactus enables a "healthy" ensemble of two models, which performs better than one model. We're happy to explore ensemble effects in speculative decoding as future work (see discussion in Appendix C of the revision).

Add:

Replying to Official Comment by Authors (2/3)

Official Comment by Authors (3/3)

Official Comment by Authors 19 Nov 2025, 08:48 Everyone

Comment:

Could the framework be generalized to allow for multi-token acceptance rules (e.g., chains or blocks of tokens) beyond per-token KL constraint, and what would the mathematical/algorithmic implications be? Thanks for the insightful suggestion. Yes, we believe Cactus can be extended to multi-token acceptance rules given that it is an alternative to the SpS acceptance function. Therefore, one straightforward way is directly replacing the SpS/TAS acceptance functions in these methods with Cactus. However, we're also exploring more integrated algorithms that jointly consider multiple steps as future work.

Summary

We thank the reviewer again for supporting our submission and providing detailed comments. In our response, we (1) explained practical choices and potential adaptive strategies for , (2) discussed scalability to larger and more diverse models, and (3) clarified how Cactus complements recent tree-based / recursive decoding as a drop-in acceptance rule. We hope these clarifications address the reviewer's concerns. We're looking forward to your stronger support! Thank you.

Add:

Replying to Official Comment by Authors (3/3)

Follow-up response with additional experiments

Official Commentby Authors25 Nov 2025, 11:26Everyone

Comment:

Scope of Generalization Claims: Although Figure 4 and several model series are included, experiments mostly focus on models up to 14B parameters and a limited set of benchmarks. Claims about scalability to substantially larger backbones (hundreds of billions of parameters) or broader domains (e.g., multilingual or multimodal tasks) remain unproven, and the discussion in Appendix C defers these questions to future work. How would the approach scale to extremely large verifiers (e.g., >100B parameters) with draft models that are structurally or statistically misaligned? Does the performance gain or accuracy gap increase as model size scales up? Following our previous response, we have been conducting additional experiments with larger models, and we just obtained the new results.

	GSM8K	GSM8K	IFEval	IFEval	GPQA	GPQA
	Score↑	AL↑	Score↑	AL↑	Score↑	AL↑
SpS	95.30	5.03	83.36	2.61	40.40	3.73
TAS	94.10	7.02	83.73	4.16	40.40	6.12
Ours ()	94.40	7.13	85.21	4.47	41.92	6.36

These experiments were conducted using the Qwen3 1.7B + 32B model pair. As shown in the table, Cactus not only has the longest acceptance length (AL) but also achieves the highest scores across 2 out of 3 tasks. This again confirms that the effectiveness of Cactus holds beyond smaller models and remains consistent even when both the drafter and the verifier are significantly larger.

We hope our additional experiments further strengthen the empirical value of our work.

Add:

About OpenReview
Contact

[FAQ](#)

[Hosting a Venue](#)

[Sponsors](#)

[Terms of Use / Privacy Policy](#)

[All Venues](#)

[**Donate**](#)

[News](#)

[OpenReview](#) is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the [OpenReview Sponsors](#). © 2026
OpenReview